University of Naples Parthenope
School of Science, Engineering and Health
Master Degree in Applied Computer Science
Machine Learning & Big Data



Master Degree Thesis

Explainable Contrastive Learning for Spatiotemporal Representation of In Vitro Fertilization Embryo Videos

Supervisors
Prof. Angelo Ciaramella
Prof. Emanuel Di Nardo

Candidate
Attilio Di Vicino
Student ID 0120000331

Academic Year 2024/2025

"Ai miei genitori"

Abstract

Selecting the most viable embryo for transfer is one of the most consequential decisions in assisted reproduction, yet it still relies heavily on the subjective judgment of the embryologist. Time-lapse imaging has made continuous observation of embryonic development possible without disturbing the culture environment. Subtle morphokinetic events such as cytoplasmic waves, transient patterns of internal fluid movement associated with embryo viability, are particularly difficult to detect and quantify by visual inspection alone.

This thesis presents a self-supervised deep learning framework for the spatiotemporal analysis of embryo time-lapse videos, designed to operate under the strict data constraints that characterise the clinical setting. A 3D convolutional backbone extracts spatial features from preprocessed frames and optical flow fields, which are then aggregated temporally by modular GRU, LSTM, or Transformer encoders trained with contrastive objectives. The learned representations are shown to encode class-discriminative structure without requiring any manual annotation. Explainability is addressed through a modified Integrated Gradients formulation anchored to a temporally grounded baseline, and through temporal occlusion analysis, both confirming that the model's attention is directed towards genuine biological events rather than imaging artefacts.

The framework offers a reproducible, label-efficient, and interpretable path towards automated cytoplasmic wave detection, with direct implications for embryo selection in clinical IVF practice.

Contents

Introduction **1**

1 Theoretical background **4**

1.1 Assisted Reproductive Technology 4

1.1.1 Time-Lapse Monitoring 5

1.1.2 The EXPREMIA Project 6

1.1.3 Biomarkers 6

1.2 Optical Flow 7

1.2.1 Motion Field and Optical Flow 8

1.2.2 The Optical Flow Equation 8

1.2.3 The Aperture Problem 9

1.2.4 Deep Learning Approaches 9

1.3 Contrastive Learning 10

1.3.1 Triplet Loss 11

1.3.2 SimCLR 12

1.3.3 InfoNCE Loss 13

1.3.4 NT-Xent Loss 14

1.3.5 VICReg 14

1.4 Deep Learning 15

1.4.1 Autoencoder 16

1.4.2 Convolutional Neural Network 16

1.4.3 Long Short-Term Memory 17

1.4.4 Gated Recurrent Unit 18

1.4.5 Transformer 19

1.5 Explainable Artificial Intelligence 20

1.5.1 A Taxonomy of XAI Methods . . . 21
1.5.2 XAI in the Medical Domain . . . 22
1.5.3 Integrated Gradients . . . 23
1.5.4 Temporal Occlusion . . . 25

2 Realized Application **27**
2.1 Loss Functions . . . 27
2.1.1 Contrastive Loss Functions . . . 28
2.1.2 Reconstruction Loss . . . 29
2.1.3 Combined Loss . . . 30
2.2 Contrastive Encoder Architecture . . . 30
2.2.1 3D Convolutional Backbone . . . 31
2.2.2 Temporal Aggregation Module . . . 32
2.2.3 Projection Head . . . 33
2.3 Decoder Architecture . . . 34
2.3.1 General Architecture . . . 34
2.3.2 Wave Probability Decoder . . . 35
2.3.3 Grayscale Reconstruction Decoder . . . 36
2.4 Explainability Methods . . . 37
2.4.1 Integrated Gradients . . . 38
2.4.2 Temporal Occlusion . . . 40

3 Experimental Results **42**
3.1 Dataset and Data Preprocessing . . . 43
3.1.1 Preprocessing Pipeline . . . 43
3.1.2 Optical Flow Extraction . . . 44
3.1.3 Clip Generation for Contrastive Learning . . . 45
3.2 Baseline Model Configuration . . . 45
3.3 Ablation Studies . . . 46
3.3.1 Contrastive Loss Function . . . 47
3.3.2 Input Modality . . . 47
3.3.3 Temporal Model Architecture and Batch Size . . . 49
3.3.4 Training Dynamics . . . 51

3.4 VICReg as Batch-Size-Free Alternative 54
3.4.1 Architecture Comparison under VICReg 54
3.4.2 Validation Stability: GRU vs. Transformer 55
3.4.3 Sensitivity Analysis: VICReg Coefficients on GRU 57
3.4.4 NT-Xent vs. VICReg 58
3.4.5 Data Augmentation 58
3.5 Results on Augmented Datasets 60
3.5.1 NT-Xent + Triplet 24-video dataset 60
3.5.2 NT-Xent + Triplet 60-video dataset 61
3.5.3 VICReg with GRU 60-video dataset 62
3.6 Summary of Training Results 63
3.7 Latent Space Separability Analysis 64
3.7.1 NT-Xent with GRU and Batch Size 16 65
3.7.2 NT-Xent with GRU and Batch Size 128 66
3.7.3 VICReg with GRU 67
3.8 Latent Space Separability on Augmented Datasets 69
3.8.1 NT-Xent + Triplet, $\lambda_{\text{triplet}} = 0.1$, 24-video dataset 69
3.8.2 NT-Xent + Triplet, $\lambda_{\text{triplet}} = 0.5$, 24-video dataset 70
3.8.3 NT-Xent + Triplet, $\lambda_{\text{triplet}} = 0.5$, 60-video dataset 71
3.8.4 VICReg, 60-video dataset 72
3.9 Clustering Evaluation 73
3.10 Explainability Analysis 76
3.10.1 Integrated Gradients 76
3.10.2 Temporal Occlusion 80

4 Discussion and Conclusions **89**
4.1 Discussion 89
4.2 Conclusions 93
4.3 Future Work 93

A **100**
A.1 Contrastive Loss Functions 100
A.2 Reconstruction and Combined Loss 102

A.3 Encoder 105
A.4 Decoder 111
A.5 Explainability 114
A.5.1 Spatial Attribution 114
A.5.2 Temporal Occlusion 117

List of Figures

1.1 Topological wave comparison 7

2.1 Architecture of the contrastive encoder 34
2.2 Architecture of the decoder 37

3.1 Preprocessing pipeline used before model training 45
3.2 Training curves for four contrastive loss functions 47
3.3 NT-Xent training curves for the three input configurations 48
3.4 NT-Xent encoder loss curves 49
3.5 Final NT-Xent loss at epoch 200 50
3.6 Training loss curves at batch size 128 51
3.7 Training and validation loss curves for the GRU model 52
3.8 VICReg training loss curves for LSTM, GRU, and Transformer . . 55
3.9 Train and validation loss curves with VICReg 56
3.10 GRU training loss under four VICReg coefficient configurations . 57
3.11 Training and validation loss curves for NT-Xent + Triplet on 24-video dataset 60
3.12 Training and validation loss curves for NT-Xent + Triplet on 24-video dataset (lambda = 0.5) 61
3.13 Training and validation loss curves for NT-Xent + Triplet on 60-video dataset (lambda = 0.5) 62
3.14 VICReg training and validation loss curves on 60-video dataset . . 63
3.15 Latent space projections for NT-Xent GRU trained at batch size 16 65
3.16 Latent space projections for NT-Xent GRU trained at batch size 128 66
3.17 Latent space projections for VICReg GRU 68

3.18 Latent space projections for NT-Xent + Triplet on 24-video dataset (lambda triplet = 0.1) 69
3.19 Latent space projections for NT-Xent + Triplet on 24-video dataset (lambda triplet = 0.5) 70
3.20 Latent space projections for NT-Xent + Triplet on 60-video dataset (lambda triplet = 0.5) 71
3.21 Latent space projections for VICReg on 60-video dataset 72
3.22 Clustering performance metrics (ARI, NMI, Silhouette) across all configurations 74
3.23 Integrated Gradients for NT-Xent GRU batch size 16 77
3.24 Integrated Gradients for NT-Xent GRU batch size 128 78
3.25 Integrated Gradients for VICReg GRU 79
3.26 Integrated Gradients for NT-Xent + Triplet, 60-video dataset . . 80
3.27 Temporal occlusion profiles for NT-Xent GRU at batch size 16 . . 81
3.28 Temporal occlusion profiles (blur/mean) for NT-Xent GRU at batch size 128 83
3.29 Temporal occlusion profiles (mean/blur) for VICReg GRU 85
3.30 Temporal occlusion profiles for NT-Xent + Triplet 60-video dataset 87

List of Tables

3.1 Baseline model hyperparameters used across all ablation studies. . 46
3.2 Complete ablation results 53
3.3 Final selected configuration resulting from the ablation studies. . 54
3.4 Comparative results for NT-Xent and VICReg 58
3.5 Summary of training results across all configurations (loss comparison table) 64
3.6 Clustering evaluation results across all configurations (ARI, NMI, Silhouette) 75

Introduction

The treatment of human infertility has been profoundly shaped by the development of In Vitro Fertilization, a technology that has made parenthood possible for millions of couples worldwide. Over the decades, clinical protocols have evolved considerably: where it was once common to transfer multiple embryos simultaneously to maximise the chances of a successful pregnancy, modern reproductive medicine now favours the elective transfer of a single embryo. This shift has greatly reduced the risks associated with multiple gestations, but it has introduced a demanding selection problem. Among a cohort of candidate embryos, clinicians must identify the one with the highest implantation potential before the transfer window closes.

To support this decision, most IVF laboratories rely on time-lapse imaging systems, which are incubators equipped with high-resolution cameras that record the embryo's development continuously over several days without disturbing the culture environment. The result is a rich spatiotemporal record of developmental events, including cytoplasmic waves, which are transient patterns of internal fluid movement observed in the hours following fertilisation and recognised as meaningful indicators of embryo viability. The problem is that reviewing this footage manually is time-consuming, subjective, and prone to variability between operators. There is therefore a genuine clinical need for computational tools capable of detecting and characterising these events automatically.

From a machine learning perspective, the challenge is not straightforward. Supervised deep learning has achieved impressive results in medical imaging, but it depends on large quantities of annotated data, which are exceptionally difficult to obtain in the IVF setting. Pixel-level annotations of embryo videos are costly to produce, require specialist expertise, and are subject to strict data privacy

regulations.

This thesis addresses the problem through a self-supervised contrastive learning framework designed specifically for the spatiotemporal analysis of embryo time-lapse videos. Rather than relying on human-provided labels, the proposed architecture learns meaningful representations directly from the structure of the data, combining preprocessed image frames with optical flow fields that make cytoplasmic motion explicit. The system pairs a 3D convolutional backbone with modular temporal aggregation modules, namely GRU, LSTM, and Transformer variants, and is trained under several contrastive objectives, including NT-Xent, VICReg, and a joint NT-Xent + Triplet formulation, to identify which configuration produces the most discriminative latent representations under the constraints of a small clinical dataset. A convolutional decoder is trained jointly to reconstruct motion masks from the latent space, providing an additional supervisory signal and enabling qualitative inspection of what the encoder has learned.

To investigate the effect of data availability, a dedicated offline augmentation pipeline was developed to generate physically coherent augmented copies of the original videos, preserving the alignment between image frames and optical flow fields across all geometric transformations, and yielding two expanded datasets of 24 and 60 balanced videos respectively. The experiments on these augmented sets show consistent improvements in both latent space geometry and clustering quality: the joint NT-Xent + Triplet objective on the 60-video dataset achieves an Adjusted Rand Index of 0.81 and a Normalised Mutual Information of 0.76 when evaluated with Gaussian Mixture Models, with the 24-video balanced configuration reaching an ARI of 0.92, the highest recorded across all experiments. These results confirm that even modest dataset expansion, when combined with a margin-enforcing contrastive objective, substantially improves the unsupervised recoverability of the class structure.

Finally, the framework incorporates two complementary explainability methods: a modified Integrated Gradients approach with a temporally grounded baseline, and a temporal occlusion analysis, both designed to verify that the model's attention is anchored to genuine biological events rather than to imaging artefacts. The spatial attribution maps confirm that the encoder consistently attends to the interior of the oocyte, tracing the internal cytoplasmic dynamics across

frames. The temporal occlusion profiles show that the highest-importance windows align with the intervals where the wave is expected to occur and return to zero once it subsides, a behaviour that matches clinical intuition and provides a degree of interpretability that is essential for deployment in a medical context.

The remainder of this thesis is organised as follows. Chapter 1 provides the theoretical background, covering the relevant concepts from assisted reproductive technology, optical flow, contrastive learning, and explainable AI. Chapter 2 describes the architecture and implementation of the proposed system. Chapter 3 presents the experimental results, including ablation studies on loss functions, input modalities, and temporal architectures, as well as the latent space, clustering, and explainability analyses. Chapter 4 draws conclusions and outlines directions for future work.

Chapter 1

Theoretical background

This chapter provides the conceptual foundations needed to understand the contributions of this thesis. It is organised in four parts. The first covers the clinical and biological context: the IVF procedure, the role of time-lapse imaging in modern embryology, and the significance of cytoplasmic waves as a viability biomarker. The second introduces optical flow, the computer vision technique used to make cytoplasmic motion explicit in the input data. The third covers the self-supervised and contrastive learning frameworks that form the core of the proposed methodology, from InfoNCE and NT-Xent to VICReg and the triplet loss. The fourth addresses the deep learning architectures used throughout the system, namely CNNs, LSTMs, GRUs, and Transformers, and closes with a discussion of Explainable AI methods, with particular attention to Integrated Gradients and Temporal Occlusion.

1.1 Assisted Reproductive Technology

The fight against human infertility has seen remarkable scientific progress over the past few decades. The development of In Vitro Fertilization (IVF) led to the birth of the first "test-tube baby" in 1978, an achievement that eventually earned Robert G. Edwards the Nobel Prize in Physiology or Medicine in 2010 [1]. Since then, the use of Assisted Reproductive Technology (ART) has grown exponentially worldwide, and this upward trend is expected to continue as both technology and clinical protocols keep improving.

While pregnancy rates following ART have shown steady improvement over

time, modern reproductive medicine still faces a critical challenge. Historically, to maximise the chances of a successful pregnancy, it was common practice to transfer multiple embryos into the patient's uterus. This approach, however, led to a high rate of multiple gestations, such as twins or triplets, which carry serious health risks for both the mother and the infants. For this reason, modern IVF has gradually moved towards an elective Single Embryo Transfer (eSET) policy, with the goal of achieving the birth of a single, healthy baby while drastically reducing the occurrence of multiple pregnancies [2].

This shift introduces a critical selection problem: if only one embryo is to be transferred, clinical staff must be able to identify the single most viable candidate, that is, the one with the highest implantation potential.

1.1.1 Time-Lapse Monitoring

The in vitro development from oocyte to blastocyst typically spans the first 5 to 6 days following fertilization. The journey begins with the fertilized oocyte (zygote) and progresses through a series of cleavage stages, dividing into 2 cells (t2), 3 cells (t3), and continuing until it compacts into a morula, finally expanding into a blastocyst ready for implantation [3].

Traditionally, monitoring this delicate process required embryologists to manually remove the embryos from the incubator to observe them under a microscope. This static, point-in-time evaluation is not only time-consuming but also potentially harmful, as it exposes the embryos to suboptimal environmental conditions outside their controlled culture.

To overcome these limitations, modern biological laboratories have introduced advanced Time-Lapse Systems (TLS) integrated directly into specialized incubators, such as the Geri system. These devices consist of six individualized incubation chambers equipped with cameras that capture images at regular intervals (e.g., one frame every 5 minutes). This technology allows embryologists to monitor the developmental progress continuously, remotely, and 24/7, without ever disturbing the culture environment.

1.1.2 The EXPREMIA Project

For the field of Computer Science, the widespread adoption of TLS represents a fundamental paradigm shift. The IVF embryo selection process has evolved from a purely biological assessment into a large-scale spatiotemporal data analysis problem.

Because of this strong need for automated and objective support tools, the EXPREMIA project was created. The core objective of EXPREMIA is the identification and automated selection of reliable biomarkers associated with embryo development and implantation competence. By leveraging deep learning and computer vision, the project aims to improve blastocyst selection, thereby increasing the efficiency of the eSET protocol.

1.1.3 Biomarkers

To accurately characterize the embryo's transformation and predict its viability, several morphological and morphokinetic features are extracted from the time-lapse videos. The most frequently analyzed parameters include cleavage timing, the appearance of a cytoplasmic halo, pronuclear morphology, blastomere symmetry, direct cleavage events, overall dimensions, and cytoplasmic waves.

Among these, the *cytoplasmic wave* is a particularly crucial biomarker. Given that the in vitro culture medium is completely static and devoid of physical fluid currents, these visible movements are internally driven. They are biologically associated with intracellular calcium (Ca^{2+}) oscillations that occur shortly after fertilization, which are fundamental for oocyte activation [4].

Recent clinical observations distinguish between two main topologies of cytoplasmic waves based on their origin: *central* waves and *peripheral* waves. As illustrated in Figure 1.1, empirical results from a cohort of $n = 178$ embryos reveal a statistically difference in wave topology between blastocysts and arrested embryos ($p < 0.01$). Specifically, blastocysts exhibit an higher frequency of central waves compared to arrested embryos, whereas the peripheral wave frequency follows the opposite trend, being significantly lower in blastocysts than in arrested embryos. Consequently, the presence of a central cytoplasmic wave is hypothesized to be a strong indicator of high embryonic quality, making its automated

detection a primary focus of the computational pipeline.

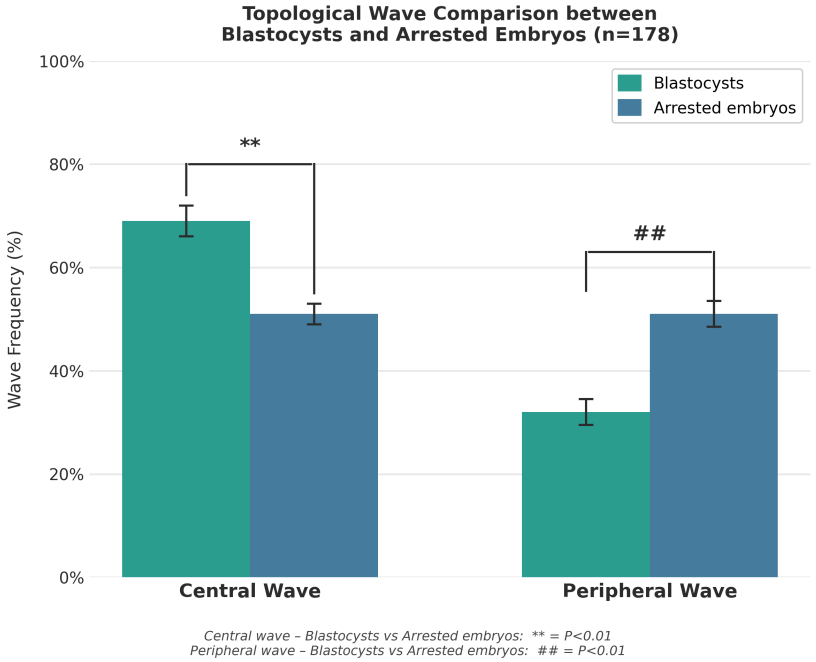


Figure 1.1: Topological wave comparison between blastocysts and arrested embryos ($n = 178$). Blastocysts show a significantly higher frequency of central waves compared to arrested embryos, while the opposite pattern is observed for peripheral waves. Statistical significance: $^{**}p < 0.01$ (central wave, blastocysts vs. arrested embryos); $^{\#\#}p < 0.01$ (peripheral wave, blastocysts vs. arrested embryos).

1.2 Optical Flow

In the context of computer vision, a video can be thought of as a sequence of frames sampled over time, where each frame is a function of spatial coordinates (x, y), the channel c, and the time instant t, written as $I(x, y, c, t)$. Understanding how the content of a scene changes between consecutive frames is a fundamental problem in many computer vision applications, and optical flow is one of the main tools developed to address it.

1.2.1 Motion Field and Optical Flow

Before introducing the optical flow equation, it is important to distinguish between two related but conceptually different quantities: the *motion field* and the *optical flow*.

The motion field is a purely geometric concept. When a point p_0 moves through a 3D scene, its trajectory projects onto the image plane, producing a corresponding 2D velocity vector. This projected velocity is what constitutes the motion field: it describes the actual physical displacement of scene points as seen by the camera.

Optical flow, on the other hand, is not a geometric quantity but a photometric one. It is defined as the apparent motion of brightness patterns in the image, and it is what can actually be estimated from pixel intensities alone. Under ideal conditions, optical flow and the motion field coincide, but in practice they can diverge significantly [5].

1.2.2 The Optical Flow Equation

Classical optical flow estimation is built upon two working assumptions. The first is *brightness constancy*: the intensity of a pixel is assumed to remain unchanged as it moves from one frame to the next, so that $I(x, y, t) = I(x + u,\, y + v,\, t + 1)$, where (u, v) denotes the displacement vector. The second is *small motion*: displacements between consecutive frames are assumed to be small enough for a first-order Taylor expansion to hold.

Combining these two assumptions and discarding higher-order terms yields the optical flow constraint equation:

$$I_x u + I_y v + I_t = 0 \tag{1.1}$$

where I_x and I_y are the spatial image gradients, and I_t is the temporal gradient between consecutive frames. This single equation relates the two unknown flow components (u, v) to quantities that can be computed directly from the image data [6].

1.2.3 The Aperture Problem

Equation (1.1) provides one constraint per pixel but leaves two unknowns underdetermined. This fundamental ambiguity is known as the *aperture problem*: within a small local window, only the component of motion perpendicular to the local image gradient, called *normal flow*, can be recovered. The component parallel to the gradient, the *parallel flow*, remains entirely invisible, because any displacement along the direction of a straight edge produces no change in brightness.

The Lucas-Kanade method [7] resolves this ambiguity by introducing the additional assumption of *spatial coherence*: all pixels within a small neighbourhood around a given point are assumed to share the same flow vector (u, v). Using a 5×5 window, for instance, yields 25 equations for the same two unknowns, forming an overdetermined system that can be solved in the least-squares sense as $\mathbf{u} = (A^\top A)^{-1} A^\top \mathbf{b}$. The matrix $A^\top A$ has the same structure as the second-moment matrix used in the Harris corner detector, and its eigenvalues determine where reliable flow estimates can be obtained. Regions with low texture or along edges remain problematic, whereas corners and high-texture regions with two large eigenvalues yield well-conditioned estimates.

A further practical limitation of the basic formulation is the small motion assumption. When displacements exceed roughly one pixel, the Taylor approximation breaks down and the linear constraint no longer holds. The standard solution is a *coarse-to-fine* strategy based on a Gaussian image pyramid: optical flow is first estimated at a coarse, low-resolution scale where large displacements appear small, and the result is then progressively refined at finer scales by warping one image toward the other using the estimate from the level above [6].

1.2.4 Deep Learning Approaches

Although classical methods such as Lucas-Kanade remain relevant and widely used, the state of the art in optical flow estimation has shifted towards learned approaches. Convolutional neural networks can be trained end-to-end to predict a dense flow field directly from two consecutive RGB frames stacked into a six-channel input, bypassing the hand-crafted assumptions of brightness constancy

and spatial coherence entirely. FlowNet [8] was among the first architectures to demonstrate this, and subsequent work such as FlowNet 2.0 [9] showed that deeper and more sophisticated multi-stage architectures substantially improve accuracy, closing the gap with classical methods on standard benchmarks such as MPI-Sintel while maintaining competitive runtime. These deep models are typically trained on large synthetic datasets, such as the Flying Chairs dataset, where ground-truth flow fields are available by construction, and then fine-tuned or evaluated on real-world sequences [6].

1.3 Contrastive Learning

The success of deep learning models is strongly tied to the availability of large quantities of labeled data. In fully supervised learning, each training sample must be paired with a manually annotated ground-truth label, a process that is not only expensive and time-consuming but also limited in scale and prone to inconsistency. This dependency on human annotation becomes particularly problematic in specialized domains such as medical imaging, where expert knowledge is required to produce reliable labels and the annotation bottleneck can severely constrain model development.

To overcome this limitation, the machine learning community has explored a range of alternative approaches. Unsupervised learning attempts to discover structure in data without any labels, while semi-supervised learning seeks to make use of a small labeled set alongside a larger pool of unlabeled examples. More recently, Self-Supervised Learning (SSL) has emerged as a highly effective middle ground: the model is trained using a supervisory signal derived directly from the intrinsic structure of the data itself, with no need for external annotation. The key intuition is that a model forced to solve a well-designed *pretext task*, such as predicting the relative position of image patches, recovering a masked region, or identifying whether two views originate from the same sample, must necessarily learn rich and transferable representations of the underlying data. Once pre-trained in this way, the model can be fine-tuned on a downstream task with only a small number of labeled examples, dramatically reducing the annotation cost.

Among self-supervised techniques, **Contrastive Learning** has achieved par-

ticularly strong results, especially for learning representations from visual and spatiotemporal data [10]. The core idea behind contrastive learning is to shape a meaningful latent space by enforcing a simple geometric principle: representations of semantically similar samples, called *positive pairs*, should be pulled close together, while representations of dissimilar samples, called *negative pairs*, should be pushed apart. Positive pairs are typically constructed through data augmentation, where applying two different stochastic transformations to the same input produces two views that, by construction, should carry the same semantic content and therefore yield similar representations.

1.3.1 Triplet Loss

Before the emergence of the modern contrastive learning frameworks described above, metric learning laid much of the conceptual groundwork for representation learning through similarity. The Triplet Loss, introduced by Schroff et al. [11] in the context of face verification, formalises this idea through a simple but effective geometric constraint. Rather than operating on pairs of samples, the loss is defined over *triplets* (a, p, n), where a is an anchor sample, p is a positive example from the same class, and n is a negative example from a different class. The objective enforces that the anchor is closer in embedding space to the positive than to the negative by at least a fixed margin α:

$$\mathcal{L}_{\text{triplet}} = \sum_{(a,p,n)\in\mathcal{T}} \max\Big(0,\ \|f(a) - f(p)\|_2^2 - \|f(a) - f(n)\|_2^2 + \alpha\Big) \tag{1.2}$$

where $\mathcal{T}$ is the set of valid triplets and $f(\cdot)$ denotes the encoder. The margin $\alpha > 0$ acts as a safety buffer: the loss is zero only when the positive is already closer to the anchor than the negative by at least α, preventing the trivial solution of collapsing all embeddings to a single point.

A central challenge in triplet-based training is the selection of informative triplets. If negatives are chosen at random, the vast majority of triplets will satisfy the margin constraint from the very first epochs, producing zero gradients and stalling learning. This motivates the use of *hard negative mining* strategies, which select the negatives that most violate the margin, the so-called hardest negatives,

to keep the loss informative throughout training [11]. While effective, this strategy introduces non-trivial implementation complexity and can destabilise training if mining is too aggressive, causing the encoder to collapse onto degenerate solutions.

Compared to NT-Xent, which implicitly mines negatives from the entire batch at every step, the triplet loss requires explicit triplet construction and is generally more sensitive to the quality of the negative selection procedure. NT-Xent also benefits from a richer supervisory signal per update, since all $2(N-1)$ non-positive samples in the batch serve as negatives simultaneously, whereas a single triplet provides only one negative per anchor.

Notwithstanding these differences, the two objectives are not mutually exclusive. Several works have shown that combining them in a joint training objective can exploit complementary properties: NT-Xent provides dense, batch-level discrimination and fast convergence, while the triplet loss enforces an explicit geometric margin that can sharpen cluster boundaries in the latent space. Concretely, Yuan et al. [12] demonstrated that this combination yields more structured embedding spaces than either loss alone on visual recognition benchmarks, and similar findings have been reported in the medical imaging domain by Chaitanya et al. [13], where the margin-based component of the triplet term proved beneficial for separating visually similar pathological classes. These results suggest that incorporating a triplet regularisation term alongside NT-Xent is a natural extension to explore in future work, particularly if the dataset is expanded to a point where hard negative mining becomes computationally tractable.

1.3.2 SimCLR

One of the most influential and widely adopted frameworks for contrastive learning is SimCLR (Simple Framework for Contrastive Learning of Visual Representations), introduced by Chen et al. [14]. Despite its conceptual simplicity, SimCLR showed that a carefully designed pipeline could match or even surpass the performance of more complex self-supervised methods. The framework consists of four components working in sequence.

First, a stochastic data augmentation module applies two independent random transformations to each input sample, generating a pair of correlated views (x_i, x_j) that form a positive pair. Typical augmentations include random cropping, color

jittering, Gaussian blur, and horizontal flipping; in the video domain, temporal shifts and frame sampling strategies are also commonly used.

Second, a shared base encoder $f(\cdot)$, usually a ResNet [15], maps each augmented view to a representation vector $h = f(x)$ in a high-dimensional feature space.

Third, a projection head $g(\cdot)$, implemented as a small Multi-Layer Perceptron (MLP), further maps h into a lower-dimensional embedding $z = g(h)$ where the contrastive loss is computed. Notably, Chen et al. [14] found that discarding the projection head after pre-training and using the encoder representations h directly for downstream tasks consistently improves performance. This suggests that the projection head tends to absorb information that is useful for the pretext task but does not transfer well to other settings.

Fourth, a contrastive loss function maximises the agreement between the embeddings z_i and z_j of the positive pair, while treating all other embeddings in the mini-batch as negatives.

1.3.3 InfoNCE Loss

The theoretical foundation underlying most contrastive objectives is the InfoNCE loss (Noise-Contrastive Estimation of Mutual Information), originally proposed by van den Oord et al. [16]. Given a query representation q and a set of $K+1$ keys $\{k_0, k_1, \ldots, k_K\}$, where exactly one key k^+ is the positive match and the remaining K are negatives, the loss is defined as:

$$\mathcal{L}_{\text{InfoNCE}} = -\mathbb{E}\left[\log \frac{\exp(q \cdot k^+/\tau)}{\sum_{i=0}^{K} \exp(q \cdot k_i/\tau)}\right] \tag{1.3}$$

where $\tau > 0$ is a scalar temperature hyperparameter controlling the sharpness of the distribution over keys. Minimizing this objective is equivalent to maximizing a lower bound on the mutual information between the query and its positive key [16], providing a rigorous information-theoretic justification for the contrastive approach. The temperature τ plays a crucial role: lower values produce a more peaked distribution, forcing the model to be more discriminative,

while higher values result in a softer objective that is more tolerant of imperfect negative sets.

1.3.4 NT-Xent Loss

Within the SimCLR framework, the InfoNCE objective is instantiated as the NT-Xent loss (Normalized Temperature-scaled Cross-Entropy) [14]. Given a mini-batch of N original samples, the augmentation module generates $2N$ views. For each augmented sample i, its positive counterpart j is the other view derived from the same original input; the remaining $2(N-1)$ views in the batch are treated as negatives. Denoting the cosine similarity between two vectors as $\mathrm{sim}(u, v) = u^\top v/(\|u\|\|v\|)$, the loss for a positive pair (i, j) is:

$$\ell_{i,j} = -\log \frac{\exp(\mathrm{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k\neq i]} \exp(\mathrm{sim}(z_i, z_k)/\tau)} \tag{1.4}$$

where $\mathbb{1}_{[k\neq i]} \in \{0, 1\}$ is an indicator that excludes the query itself from the denominator. The total loss is computed symmetrically over all positive pairs in the batch, averaging $\ell_{i,j}$ and $\ell_{j,i}$ across all N pairs.

A well-known limitation of NT-Xent and contrastive methods in general is their sensitivity to the number of negatives: large batch sizes are required to populate the denominator with sufficiently diverse and hard negatives, which imposes substantial computational and memory costs during training [14].

1.3.5 VICReg

To overcome the reliance on large batches of negative samples, a family of *non-contrastive* methods has been proposed that prevent representational collapse through explicit regularization rather than repulsion. VICReg (Variance-Invariance-Covariance Regularization), introduced by Bardes et al. [17], is one of the most principled approaches in this category.

Given two batches of embeddings $Z = \{z_1, \ldots, z_N\}$ and $Z' = \{z'_1, \ldots, z'_N\}$ produced by two augmented views of the same samples, VICReg minimizes a joint objective comprising three terms:

$$\mathcal{L}_{\mathrm{VICReg}} = \lambda\, \mathcal{L}_{\mathrm{inv}} + \mu\, \mathcal{L}_{\mathrm{var}} + \nu\, \mathcal{L}_{\mathrm{cov}} \tag{1.5}$$

The **Invariance** term $\mathcal{L}_{\text{inv}}$ is the mean squared error between the two embedding batches, encouraging the representations of the same sample under different augmentations to be identical:

$$\mathcal{L}_{\text{inv}} = \frac{1}{N} \sum_{i=1}^{N} \|z_i - z_i'\|_2^2 \tag{1.6}$$

The **Variance** term $\mathcal{L}_{\text{var}}$ penalizes any embedding dimension whose standard deviation across the batch falls below a threshold γ, ensuring that the network does not collapse all representations onto a constant vector:

$$\mathcal{L}_{\text{var}} = \frac{1}{d} \sum_{j=1}^{d} \max\Big(0,\, \gamma - \text{Std}(z^j)\Big) \tag{1.7}$$

where d is the embedding dimensionality and $\text{Std}(z^j)$ denotes the standard deviation of the j-th dimension over the batch.

The **Covariance** term $\mathcal{L}_{\text{cov}}$ decorrelates the different dimensions of the embedding by penalizing the squared off-diagonal entries of the empirical covariance matrix $C(Z)$:

$$\mathcal{L}_{\text{cov}} = \frac{1}{d} \sum_{i \neq j} [C(Z)]_{ij}^2 \tag{1.8}$$

By jointly enforcing invariance to augmentation, sufficient variance across samples, and decorrelation among dimensions, VICReg achieves competitive representational quality without requiring any negative pairs, large batch sizes, or momentum encoders. This makes it particularly attractive for settings where computational resources are constrained or where the definition of meaningful negatives is non-trivial [17].

1.4 Deep Learning

In recent years, Deep Learning (DL) has fundamentally transformed the landscape of computer vision and sequential data analysis. Unlike classical machine learning approaches that require manual feature engineering, deep learning models learn hierarchical representations directly from raw data, discovering increasingly abstract and task-relevant patterns at each layer of the network. This property makes them particularly well-suited for the analysis of time-lapse embryo videos, where useful information spans multiple scales: from fine-grained

morphological details within individual frames to long-range temporal dynamics unfolding over several days of development.

1.4.1 Autoencoder

An Autoencoder is an unsupervised neural network architecture designed to learn compact, low-dimensional representations of unlabelled data [18]. The network is structured around two symmetric components: an *encoder* f_θ that compresses the input x into a latent code $z = f_\theta(x)$, and a *decoder* g_ϕ that attempts to reconstruct the original input from that code, $\hat{x} = g_\phi(z)$. Training minimizes the reconstruction error, typically the Mean Squared Error (MSE):

$$\mathcal{L}_{\text{rec}} = \frac{1}{n} \sum_{i=1}^{n} \|x_i - \hat{x}_i\|_2^2 \tag{1.9}$$

The bottleneck structure is the key inductive bias of autoencoders: by constraining the dimensionality of z to be much smaller than that of x, the network is compelled to discard noise and redundancy while preserving the most informative structure in the data. In the context of video analysis, spatiotemporal autoencoders extend this principle to sequences, encoding entire clips into compact latent vectors that capture the dominant morphokinetic patterns. Beyond compression, autoencoders serve as pre-training backbones for downstream tasks: the encoder learned in an unsupervised fashion can be subsequently fine-tuned with a small number of labeled examples, making them a natural fit for the label-scarce setting of clinical embryology.

1.4.2 Convolutional Neural Network

Convolutional Neural Networks (CNNs) are the standard architecture for processing data with grid-like topology, most notably images [19]. The fundamental building block is the convolutional layer, which applies a set of learnable filters, also called kernels, to the input by computing a local dot product at each spatial position. Given an input feature map X and a kernel W of size $k \times k$, the output at position (i, j) is:

$$(X * W)_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X_{i+m,\, j+n} \cdot W_{m,n} \tag{1.10}$$

This operation exploits two key structural properties of natural images: *local connectivity*, since nearby pixels tend to be strongly correlated, and *weight sharing*, since the same filter is applied uniformly across the spatial domain. As a result, the network learns spatially equivariant features, such as edges, textures, and eventually semantically meaningful shapes, while using far fewer parameters than a fully connected layer of equivalent capacity.

Successive convolutional layers are typically interleaved with non-linear activations such as the Rectified Linear Unit (ReLU), $\mathrm{ReLU}(x) = \max(0, x)$, and spatial downsampling operations such as max-pooling, which progressively reduce the feature map resolution while increasing the receptive field and the number of channels. This hierarchical structure allows deep CNNs to build increasingly abstract representations: early layers detect low-level features such as oriented edges, while deeper layers respond to complex patterns such as object parts or anatomical structures.

1.4.3 Long Short-Term Memory

Standard Recurrent Neural Networks (RNNs) process sequential data by maintaining a hidden state that is updated at each time step as a function of the current input and the previous state. In principle, this allows them to capture temporal dependencies of arbitrary length; in practice, training via backpropagation through time causes gradients to either vanish or explode over long sequences, making it extremely difficult for standard RNNs to learn dependencies spanning more than a few time steps [20].

Long Short-Term Memory (LSTM) networks were specifically designed to overcome this limitation [21]. The key innovation is the introduction of a dedicated *cell state* C_t, a separate memory line that runs alongside the hidden state h_t and is protected from the gradient-degrading effects of repeated non-linear transformations. Access to the cell state is regulated by three learned gating mechanisms:

- **Forget gate:** $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$, which determines which information in C_{t-1} should be discarded.

- **Input gate:** $i_t = \sigma(W_i[h_{t-1},\, x_t] + b_i)$, which controls how much of the new candidate $\tilde{C}_t = \tanh(W_C[h_{t-1},\, x_t] + b_C)$ is written into the cell state.
- **Output gate:** $o_t = \sigma(W_o[h_{t-1},\, x_t] + b_o)$, which filters the cell state through a tanh to produce the hidden state $h_t = o_t \odot \tanh(C_t)$.

The cell state update combines these gates as:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{1.11}$$

The additive nature of this update allows gradients to flow back through time with minimal attenuation, enabling LSTMs to learn temporal dependencies spanning hundreds of time steps.

1.4.4 Gated Recurrent Unit

The Gated Recurrent Unit (GRU), introduced by Cho et al. [22], is a streamlined alternative to the LSTM that consolidates the gating mechanism into a simpler, more parameter-efficient design. The GRU eliminates the separate cell state entirely, relying instead on a single hidden state h_t managed by two gates:

- **Reset gate:** $r_t = \sigma(W_r[h_{t-1},\, x_t])$, which controls how much of the previous hidden state is exposed when computing the candidate update, allowing the model to selectively ignore past information.
- **Update gate:** $z_t = \sigma(W_z[h_{t-1},\, x_t])$, which interpolates between the previous hidden state and the new candidate, playing the combined role of the LSTM's forget and input gates.

The candidate hidden state and the final update are:

$$\tilde{h}_t = \tanh\Big(W\Big[r_t \odot h_{t-1},\; x_t\Big]\Big), \qquad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{1.12}$$

By merging the two memory streams of the LSTM into one and reducing the number of gate computations from three to two, GRUs train faster and require less memory while often achieving comparable sequential modelling performance [23].

1.4.5 Transformer

The Transformer architecture, originally proposed for neural machine translation by Vaswani et al. [24], departs entirely from the recurrent processing paradigm. Rather than updating a hidden state sequentially at each time step, Transformers process all positions in a sequence simultaneously through a *self-attention* mechanism that directly models pairwise relationships between every pair of elements, regardless of their distance in the sequence.

Given an input sequence represented as a matrix of n token embeddings, the model computes three projections, namely queries Q, keys K, and values V, through learned linear maps, and produces an output as a weighted sum of the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1.13}$$

where d_k is the dimensionality of the key vectors. The scaling factor $1/\sqrt{d_k}$ prevents the dot products from growing too large in magnitude, keeping the softmax gradients in a numerically stable range. In practice, multiple attention heads are computed in parallel, each learning to focus on different relational patterns in the sequence; their outputs are concatenated and linearly projected, a mechanism known as *Multi-Head Attention*. Each encoder block then pairs this multi-head attention module with a position-wise feed-forward network, applying residual connections and layer normalization after each sub-layer [24].

The elimination of recurrence gives Transformers two significant practical advantages. First, all positions are processed in parallel during training, enabling far more efficient use of modern hardware accelerators compared to sequential RNNs. Second, the attention mechanism provides a direct, learned path between any two positions in the sequence, making it structurally easier to capture long-range dependencies without the gradient degradation that affects LSTMs and GRUs.

Since their introduction in NLP, Transformer-based architectures have been successfully adapted to vision tasks through the Vision Transformer (ViT) [25], which treats an image as a sequence of non-overlapping patches, and subsequently extended to video through models such as TimeSformer [26], which factorizes spatial and temporal attention for efficient spatiotemporal reasoning. These prop-

erties make the Transformer a natural choice for analysing embryo development sequences, where capturing dependencies across distant time steps is essential for predicting developmental outcome.

1.5 Explainable Artificial Intelligence

The remarkable predictive performance achieved by modern deep learning models comes at a significant cost: as architectures grow deeper and more complex, their internal decision-making process becomes increasingly opaque. A model that maps an input to an output through millions of non-linear transformations offers no intuitive account of why a particular prediction was made. This opacity is captured by the metaphor of the *black box*, a system whose inputs and outputs are observable, but whose internal logic is not.

In many application domains, this limitation is a manageable inconvenience. In high-stakes domains such as medicine, finance, and law, however, it becomes a fundamental barrier. A clinician who cannot understand why a model flagged an embryo as non-viable cannot meaningfully validate, override, or take responsibility for that recommendation. Regulatory frameworks, including the European Union's General Data Protection Regulation (GDPR), have begun to codify a *right to explanation* for automated decisions affecting individuals [27], further elevating interpretability from a research desideratum to a legal requirement.

Explainable Artificial Intelligence (XAI) is the field concerned with developing tools, frameworks, and methodologies that make the reasoning of machine learning models understandable to human stakeholders. The goals of XAI are multiple and distinct: *transparency* refers to the ability to understand the global logic of a model; *interpretability* refers to the ability to understand individual predictions; *fairness* refers to the ability to detect and correct systematic biases; and *trust* refers to the degree to which a user can rely on a model's output in novel situations [28]. These goals are not always mutually achievable, and different application contexts impose different trade-offs between them.

1.5.1 A Taxonomy of XAI Methods

XAI methods can be organized along several orthogonal axes, and understanding these distinctions is essential for selecting the appropriate technique for a given problem [28].

Ante-hoc vs. post-hoc. The first and most fundamental distinction concerns *when* interpretability is built into the modelling pipeline. *Ante-hoc* methods, also called intrinsically interpretable models, are designed so that their decision process is transparent by construction. Decision trees, linear regression, logistic regression, and rule-based systems all belong to this category: for these models, the prediction mechanism is directly readable from the model parameters, without any additional analysis. The key advantage of ante-hoc methods is that the explanation is exact and global; the limitation is that the model family may lack the representational capacity needed to capture complex patterns in high-dimensional data such as images or video sequences.

Post-hoc methods, by contrast, are applied after a model has already been trained, with the goal of approximating or probing its behaviour without modifying it. Post-hoc methods make no assumptions about the internal architecture of the model, which allows them to be applied to arbitrarily complex systems including deep neural networks. The trade-off is that post-hoc explanations are inherently approximate: they describe the model's behaviour in a local neighbourhood or under specific perturbations, rather than providing a global, exact account of its decision logic [29].

Model-agnostic vs. model-specific. A second axis distinguishes methods that treat the model as a black box from those that exploit knowledge of its specific architecture. *Model-agnostic* methods, such as LIME [30] and SHAP [31], interact with the model only through input-output queries, making them universally applicable but potentially less precise. *Model-specific* methods, such as Grad-CAM [32] and Integrated Gradients [33], exploit the internal computational structure of the model, for instance gradient information or layer activations, to produce more faithful and detailed attributions, at the cost of being tied to a particular architecture family [34].

Local vs. global. A third axis concerns the *scope* of the explanation. *Local* methods explain a single prediction: given a specific input, they identify which features were most responsible for the output. This is often the most relevant form of explanation in clinical practice, where a physician wants to understand why a particular patient or embryo received a particular score. *Global* methods, on the other hand, characterise the overall behaviour of the model, for instance by identifying which input dimensions are systematically most influential across the entire dataset. Global explanations are more useful for auditing and debugging, but harder to compute and interpret for complex models [29].

The complexity–interpretability trade-off. A recurring theme across all these distinctions is the fundamental tension between model complexity and interpretability. For simple, low-dimensional data, linear models provide exact and interpretable predictions; their coefficients directly quantify the marginal contribution of each feature. As data become higher-dimensional and the underlying relationships become more non-linear, more expressive models are required, but at the cost of a less transparent decision process. Ensemble methods such as Random Forests [35] occupy an intermediate position: they achieve strong predictive performance through the aggregation of many simple trees, but the resulting model is no longer directly readable. Deep neural networks push this trade-off to its extreme: they achieve state-of-the-art performance on virtually every perceptual task, but their decision process involves millions of non-linear operations that resist direct interpretation [36].

1.5.2 XAI in the Medical Domain

The medical domain represents one of the most demanding contexts for XAI, combining the full complexity of deep learning systems with a set of domain-specific requirements that go well beyond standard software engineering concerns [37].

First, the *stakes* of incorrect predictions are uniquely high. A misclassification in an image recognition benchmark is a statistic; a misclassification in a diagnostic system can directly affect patient health outcomes. This asymmetry makes it essential not only that a model achieves high predictive accuracy, but also that

clinicians can identify the cases where the model is likely to be wrong, which in turn requires understanding the features the model relies upon.

Second, medical AI systems must contend with *distribution shift*: a model trained on data from one hospital or imaging system may behave differently when deployed in a different clinical environment. Without interpretability tools, such shifts are difficult to detect before they cause harm. Post-hoc attribution methods can expose whether a model has learned clinically meaningful features or has instead latched onto spurious correlates, such as scanner-specific artefacts or demographic confounders, that happen to be predictive in the training distribution but not in deployment [31].

Third, there is the question of *regulatory acceptance* and professional accountability. In most jurisdictions, a clinician who acts on the recommendation of an AI system remains legally and ethically responsible for that decision. This creates a strong practical demand for systems that can provide a post-hoc justification that a trained clinician can evaluate, challenge, and document [27].

Finally, medical XAI must navigate the gap between *statistical relevance* and *clinical relevance*. A feature that is statistically important for a model's prediction is not necessarily the feature that a domain expert would recognise as biologically or clinically meaningful. Bridging this gap requires close collaboration between machine learning researchers and medical professionals, using XAI tools not as a final answer but as a probe for iterative model validation and refinement [37].

1.5.3 Integrated Gradients

Integrated Gradients (IG) is an axiomatic attribution method that quantifies the contribution of each input feature to a model's prediction by integrating the gradients of the output with respect to the input along a straight path from a reference baseline to the actual input [33].

Formally, let $F : \mathbb{R}^n \to \mathbb{R}$ be a differentiable model, $x \in \mathbb{R}^n$ the input of interest, and $x' \in \mathbb{R}^n$ a baseline representing a neutral or uninformative reference, typically a black image, a zero vector, or the mean of the training distribution. The integrated gradient for the i-th input feature is defined as:

$$\mathrm{IG}_i(x) = (x_i - x'_i) \int_0^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} \, \mathrm{d}\alpha \tag{1.14}$$

The integral is computed along the linear interpolation between x' and x, parameterized by $\alpha \in [0, 1]$. In practice, the integral is approximated via a Riemann sum over a finite number of steps m:

$$\mathrm{IG}_i(x) \approx (x_i - x_i') \cdot \frac{1}{m} \sum_{k=1}^{m} \frac{\partial F\Big(x' + \frac{k}{m}(x - x')\Big)}{\partial x_i} \tag{1.15}$$

The theoretical foundation of IG rests on two axioms that together guarantee the faithfulness of the attribution. The *Sensitivity* axiom states that if the model's output differs between the input and the baseline, and that difference can be attributed to a single feature, then that feature must receive a non-zero attribution score. The *Implementation Invariance* axiom states that two models that are functionally equivalent, meaning they produce identical outputs for all inputs, must yield identical attributions regardless of their internal implementation. Together, these axioms distinguish IG from simpler gradient-based methods such as plain saliency maps, which violate implementation invariance, and from perturbation-based methods, which can violate sensitivity [33].

A further useful property of IG is *completeness*: the sum of all attributions across input features exactly equals the difference in model output between the input and the baseline, $\sum_i \mathrm{IG}_i(x) = F(x) - F(x')$. This makes the attribution scores internally consistent and directly interpretable as a budget allocated across features.

The choice of baseline is critical and domain-dependent. In image-based models, a black image with all zeros is commonly used; in sequential models, a zero-vector sequence or a sequence of mean embeddings is more appropriate. The baseline should represent a point of maximal uninformativeness, a signal that the model cannot use to distinguish between classes, so that all attributable signal is captured by the difference $x - x'$.

In the context of spatiotemporal models for embryo analysis, IG can be applied to identify which spatial regions in which frames most heavily influenced the viability prediction. When applied to the output of a temporal encoder, it can additionally reveal which time steps in the developmental sequence carry the most predictive information, providing a form of temporal saliency that is grounded in the model's actual gradient structure rather than in heuristic perturbations.

1.5.4 Temporal Occlusion

Occlusion sensitivity is a perturbation-based XAI technique that probes a model's reliance on specific input regions by systematically masking portions of the input and measuring the resulting change in the model's output confidence [38]. In its original formulation, the method slides a patch of fixed size across a 2D image, replacing each region with a neutral value such as zeros or the mean pixel intensity, and recording the drop in classification score. Regions whose occlusion causes a large drop in confidence are deemed highly relevant for the prediction.

Temporal Occlusion extends this paradigm from the spatial to the temporal dimension, making it applicable to sequential and video data [39]. Rather than masking spatial patches within a single frame, the method systematically masks contiguous temporal segments of the input sequence, replacing entire time windows with a neutral token or a zero embedding, and measures the corresponding change in model output. Let $F(x_{1:T})$ denote the model's prediction for a sequence of length T, and let $x_{1:T}^{(t_s,t_e)}$ denote the same sequence with the segment $[t_s, t_e]$ replaced by a neutral value. The temporal sensitivity score for that segment is:

$$S(t_s, t_e) = F(x_{1:T}) - F\left(x_{1:T}^{(t_s,t_e)}\right) \tag{1.16}$$

A high value of $S(t_s, t_e)$ indicates that the model's prediction depends strongly on the information contained in that temporal window. By sweeping the occlusion window across all positions in the sequence, a *temporal sensitivity profile* can be constructed, providing an interpretable map of which phases of the developmental sequence are most informative for the model's decision.

The primary strength of temporal occlusion is its model-agnostic nature: it requires no knowledge of or access to the model's internal parameters, operating solely through input-output queries. This makes it applicable to any sequential model regardless of its architecture, including recurrent networks, Transformers, and hybrid spatiotemporal encoders. It is also particularly well-suited to domains where the input has a natural temporal segmentation aligned with domain knowledge: in embryology, for instance, known developmental milestones such as fertilization, pronuclear formation, first cleavage, compaction, and blastulation define meaningful temporal landmarks against which the model's sensitivity

profile can be validated [40].

The main limitation of perturbation-based methods in general, and of temporal occlusion in particular, is their computational cost: a sequence of length T requires up to $\mathcal{O}(T^2)$ forward passes if all contiguous windows are evaluated, which can be prohibitive for long sequences or large models. In practice, this is addressed either by restricting the analysis to fixed-length non-overlapping windows, or by using a coarser temporal resolution informed by domain knowledge. A further limitation is that occlusion-based attributions can be misleading when input features are strongly correlated: masking a segment that is redundant with another part of the sequence will produce a low sensitivity score even if the masked content is genuinely informative, simply because the model can recover the information from the remaining context [29].

Despite these limitations, temporal occlusion remains one of the most practically useful XAI tools for sequential models in clinical settings, precisely because its output is directly interpretable by domain experts without requiring knowledge of machine learning. A sensitivity curve over time can be read and validated by an embryologist in the same way that a developmental timeline would be, providing a natural bridge between the model's learned representations and established biological knowledge.

Chapter 2

Realized Application

This chapter describes the design and implementation of the proposed system. It begins with the dataset and the preprocessing pipeline, covering temporal truncation, spatial stabilisation, visual enhancement, and optical flow extraction. It then details the clip generation strategy adopted for contrastive training, followed by a description of the four contrastive loss functions implemented, namely NT-Xent, InfoNCE, VICReg, and the joint NT-Xent + Triplet formulation, as well as the reconstruction and combined objectives used to supervise the decoder. The encoder and decoder architectures are then presented in full, including the 3D CNN backbone, the modular temporal aggregation modules, and the upsampling decoder variants. The chapter closes with the two explainability methods developed for spatial and temporal attribution: a modified Integrated Gradients formulation with a temporal median baseline and SmoothGrad averaging, and a temporal occlusion analysis with three occlusion strategies.

2.1 Loss Functions

A central design decision in this work concerns the choice of objective functions used to train the spatiotemporal autoencoder. Two families of losses are employed jointly: a **contrastive loss** that drives the encoder to learn discriminative latent representations, and a **reconstruction loss** that supervises the decoder. These are combined into a single **combined loss** that governs the end-to-end training procedure. The following sections describe each component in detail.

2.1.1 Contrastive Loss Functions

Four contrastive objectives were implemented and made available as interchangeable components of the training pipeline. Their mathematical formulations and theoretical properties are discussed in Section 1.3.5 and the surrounding background material; the focus here is on the implementation choices and the rationale for including each variant.

NT-Xent and InfoNCE are treated as the primary objectives. As established in the theoretical background, the two are numerically equivalent on two-view batches, but their explicit implementations differ in how the positive and negative sets are constructed internally, making InfoNCE somewhat easier to extend to asymmetric or cross-modal settings. Both use a fixed temperature $\tau = 0.07$ throughout all experiments.

VICReg was included specifically to investigate whether its independence from negative pairs could compensate for the limited batch sizes available under the hardware constraints of this work. The three weighting coefficients (λ, μ, ν) governing the invariance, variance, and covariance terms are exposed as configuration parameters, with the paper's recommended values $(25, 25, 1)$ used as the default.

The fourth objective combines NT-Xent with a Triplet Loss [11], motivated by the complementary properties of the two terms discussed in Section 2.1.1. NT-Xent provides dense batch-level discrimination at every update step, while the triplet term enforces an explicit geometric margin α that can sharpen cluster boundaries in the latent space. In the joint objective the two contributions are weighted independently:

$$\mathcal{L}_{\text{combined}} = \lambda_{\text{enc}} \cdot \mathcal{L}_{\text{NT-Xent}} \; + \; \lambda_{\text{triplet}} \cdot \mathcal{L}_{\text{triplet}} \tag{2.1}$$

where λ_{enc} and λ_{triplet} are scalar weights that control the relative contribution of each term. This formulation allows the triplet component to act as a regulariser on top of the contrastive objective rather than as a replacement for it, preserving the convergence speed of NT-Xent while benefiting from the margin-based geometric constraint.

All four objectives share a common interface: given two batches of embeddings $\mathbf{z}_1, \mathbf{z}_2 \in \mathbb{R}^{N \times D}$ and an optional tensor of class labels, any loss function can be

passed to the combined training objective described in Section 2.1.3, making it straightforward to switch between objectives without modifying the training pipeline.

2.1.2 Reconstruction Loss

The decoder is supervised by a reconstruction objective that encourages it to recover meaningful spatial content from the latent representation produced by the encoder. Two reconstruction losses were implemented, selected based on the decoder type and input modality.

Flow-Based Reconstruction Loss

For configurations where the input consists of optical flow fields or multimodal fusion data, the decoder is trained to predict a binary motion mask derived from the flow magnitude. Given the predicted mask $\hat{\mathbf{M}} \in [0,1]^{B\times 1\times T\times H\times W}$, a pseudo-label target is constructed at each training step by normalising the L2 magnitude of the flow channels and thresholding at a fixed value $\theta = 0.3$:

$$\mathbf{M}^* = \mathbf{1}\left[\frac{\| (d_x, d_y) \|_2 - \min}{\max - \min + \epsilon} > \theta\right] \tag{2.2}$$

The reconstruction loss is then the binary cross-entropy between the decoder output and this pseudo-label:

$$\mathcal{L}_{\text{recon}} = \text{BCE}\Big(\hat{\mathbf{M}},\, \mathbf{M}^*\Big) \tag{2.3}$$

This formulation is self-supervised: no manual annotation is required, as the target is derived directly from the input optical flow. The decoder is therefore encouraged to localise regions of significant cytoplasmic motion, which aligns naturally with the biological objective of detecting wave activity.

Grayscale Reconstruction Loss

For grayscale configurations, a reconstruction loss is employed. The decoder is trained to reproduce the luminance channel of the input, computed as the ITU-R BT.601 weighted combination of the RGB channels. To prevent the trivial solution of predicting the per-clip mean intensity, which is particularly likely

for low-contrast embryo frames, the target luminance is contrast-stretched per sample to the range [0.05, 0.95] before computing the loss. The objective combines a mean squared error term with an L1 regularisation term to balance a strong gradient signal early in training with resistance to over-smoothing at convergence:

$$\mathcal{L}_{\text{gray}} = 0.7 \cdot \text{MSE}(\hat{\mathbf{Y}},\, \mathbf{Y}^*) \;+\; 0.3 \cdot \text{MAE}(\hat{\mathbf{Y}},\, \mathbf{Y}^*) \tag{2.4}$$

where $\mathbf{Y}^*$ is the contrast-stretched luminance target and $\hat{\mathbf{Y}}$ is the decoder output.

2.1.3 Combined Loss

The encoder and decoder are trained jointly through a combined objective that weights the contrastive and reconstruction terms:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{enc}} \cdot \mathcal{L}_{\text{contrastive}} \;+\; \lambda_{\text{dec}} \cdot \mathcal{L}_{\text{recon}} \tag{2.5}$$

with default weights $\lambda_{\text{enc}} = 0.8$ and $\lambda_{\text{dec}} = 0.2$. This weighting deliberately prioritises the contrastive objective, which is responsible for the quality of the latent representations, while allowing the reconstruction term to act as a regulariser that anchors the decoder to spatially meaningful features and prevents the encoder from collapsing to degenerate solutions. The contrastive component accepts any of the four loss functions described above, making the combined loss fully modular: the objective function can be swapped at configuration time without modifying the training pipeline. As established by the training dynamics reported in Section 3.3.4, the combined loss is dominated by the contrastive term throughout training, with the decoder loss contributing a smaller but stable regularisation signal.

2.2 Contrastive Encoder Architecture

The contrastive encoder is the core component of the proposed system. Its purpose is to map a raw spatiotemporal clip (B, C, T, H, W) to a compact embedding vector that captures the biologically relevant dynamics of the input sequence. The architecture is composed of three sequential stages: a **3D convo-**

lutional backbone that extracts spatial features frame by frame, a **temporal aggregation module** that integrates these features across time, and a **projection head** that maps the resulting representation into the contrastive embedding space. Each stage is described in detail below.

2.2.1 3D Convolutional Backbone

Spatial feature extraction is performed by a lightweight 3D convolutional backbone (`CNN3DBackbone`), composed of four sequential convolutional blocks. Each block consists of a `Conv3d` layer followed by Batch Normalisation and a ReLU activation:

$$\mathbf{x}^{(l)} = \mathrm{ReLU}\Big(\mathrm{BN}\Big(\mathrm{Conv3D}\Big(\mathbf{x}^{(l-1)}\Big)\Big)\Big) \tag{2.6}$$

All convolutions use a $3 \times 3 \times 3$ kernel with unit temporal stride and spatial stride 2, so the temporal dimension T is preserved throughout while the spatial dimensions are progressively halved at each block. After four blocks, the spatial resolution is reduced by a factor of 16 in both H and W. The number of feature channels follows a doubling schedule starting from a configurable base channel width C_{base}:

$$C^{(1)} = C_{\text{base}}, \quad C^{(2)} = 2\,C_{\text{base}}, \quad C^{(3)} = 4\,C_{\text{base}}, \quad C^{(4)} = 8\,C_{\text{base}} \tag{2.7}$$

With $C_{\text{base}} = 32$, as used in all experiments, the backbone produces output feature maps of shape $(B,\ 256,\ T',\ H/16,\ W/16)$. The intermediate feature maps from each block are also stored and exposed for gradient-based explainability analyses.

Following the backbone, a `Dropout3d` layer with drop probability $p = 0.1$ is applied to the output feature maps. Unlike standard dropout, which zeroes individual activations, `Dropout3d` randomly silences entire feature channels during training, acting as a structured regulariser that prevents the model from relying on any single spatial feature map. Spatial pooling then collapses the H' and W' dimensions via adaptive average pooling, yielding a temporal sequence of feature vectors of shape $(B,\ T',\ C^{(4)})$, which is passed to the temporal module.

2.2.2 Temporal Aggregation Module

The temporal module processes the sequence of per-frame feature vectors produced by the backbone and aggregates them into a single fixed-size embedding that captures the temporal dynamics of the clip. Three alternative temporal modules were implemented and evaluated: GRU, LSTM, and a Transformer encoder.

GRU and LSTM

Both recurrent modules follow the same interface. The input sequence (B, T', $C^{(4)}$) is processed by a multi-layer recurrent network with hidden dimension H_{rnn} and two stacked layers. A dropout rate of 0.2 is applied between layers during training. The final embedding is taken as the last hidden state of the top layer, $\mathbf{h}_{T'}^{(\text{top})}$, which summarises the entire temporal context accumulated up to the last frame:

- **GRU:** each gate update is governed by the standard gated recurrent unit equations, with reset and update gates controlling the flow of information from previous hidden states. With fewer parameters than the LSTM, the GRU is less prone to overfitting on small datasets, which is consistent with its superior performance in the ablation experiments (Section 3.3.3).
- **LSTM:** extends the GRU with an explicit cell state that is updated via input, forget, and output gates, providing in principle a more expressive memory mechanism at the cost of additional parameters. The embedding is taken from the hidden state $\mathbf{h}_{T'}$, not the cell state.

In both cases, the full sequence of hidden states ($\mathbf{h}_1, \ldots, \mathbf{h}_{T'}$) is also stored and returned for use by the temporal occlusion explainability module.

Transformer Encoder

The Transformer-based temporal module projects the input sequence into a D_{hidden}-dimensional space via a learned linear projection, adds sinusoidal positional encodings to inject temporal order information, and prepends a learnable `[CLS]` token:

$$\mathbf{x}^{(0)} = \Big[\mathbf{e}_{\text{CLS}} \;\Big|\; \text{PE}(\mathbf{W}_{\text{in}}\,\mathbf{s}_1)\,,\;\ldots,\;\text{PE}(\mathbf{W}_{\text{in}}\,\mathbf{s}_{T'})\Big] \tag{2.8}$$

where $\text{PE}(\cdot)$ denotes sinusoidal positional encoding with maximum length 512 and $\mathbf{W}_{\text{in}}$ is the input projection matrix. The sequence is then processed by L standard Transformer encoder layers, each comprising multi-head self-attention with n_h heads and a position-wise feed-forward network with hidden dimension $4 \times D_{\text{hidden}}$. The final embedding is read from the `[CLS]` token position of the last layer's output:

$$\mathbf{e} = \text{Transformer}(\mathbf{x}^{(0)})[\,0\,] \tag{2.9}$$

To enable explainability, the forward pass is implemented layer by layer rather than through `nn.TransformerEncoder.forward`, which allows per-head attention weights to be extracted at each layer. The average attention map across layers and heads is stored and returned alongside the embedding for downstream analysis. This design choice has no effect on training dynamics but is essential for interpretability.

2.2.3 Projection Head

The raw embedding produced by the temporal module is passed through a two-layer *projection head* before being fed to the contrastive loss. This is a standard component in contrastive learning frameworks, introduced in SimCLR [14], which has been shown empirically to improve the quality of the representations retained in the pre-projection space. The projection head is a small multi-layer perceptron:

$$\mathbf{z} = \mathbf{W}_2 \cdot \text{Dropout}(\text{ReLU}(\mathbf{W}_1\mathbf{e})) \in \mathbb{R}^{D_{\text{emb}}} \tag{2.10}$$

where $\mathbf{e} \in \mathbb{R}^{D_{\text{hidden}}}$ is the temporal module output and $D_{\text{emb}} = 128$ is the contrastive embedding dimension. A dropout layer with $p = 0.2$ is inserted between the two linear layers to provide additional regularisation within the projection pathway. The contrastive loss is computed on $\mathbf{z}$; for downstream analysis and latent space visualisation, the pre-projection embedding $\mathbf{e}$ is used instead, as it retains richer structural information.

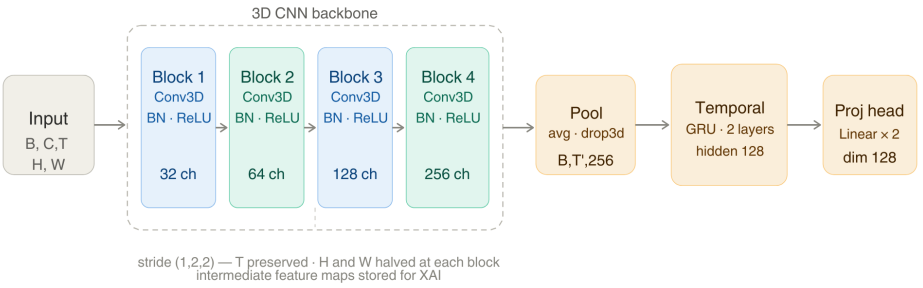


Figure 2.1: Architecture of the contrastive encoder. The 3D CNN backbone extracts spatial features with progressive channel doubling (32, 64, 128, 256). The GRU temporal module aggregates the per-frame feature sequence into a fixed-size embedding, which is then mapped to the contrastive space via a two-layer projection head. The pre-projection embedding $\mathbf{e}$ is used for latent space analysis and explainability; the projection $\mathbf{z}$ is fed to the contrastive loss.

2.3 Decoder Architecture

The decoder operates as the generative counterpart of the encoder: given the compact latent embedding $\mathbf{e} \in \mathbb{R}^{D_{\text{emb}}}$ produced by the projection head, it reconstructs a spatiotemporal volume of the same temporal and spatial extent as the input clip. Two decoder variants were implemented, differing in their reconstruction target but sharing the same overall upsampling architecture.

2.3.1 General Architecture

Both decoder variants follow a symmetric mirror of the CNN backbone: a fully connected layer first projects the latent embedding into a compact 3D feature volume, which is then progressively upsampled back to the original clip resolution through four transposed convolution blocks.

Latent projection. The embedding vector $\mathbf{e} \in \mathbb{R}^{128}$ is first mapped to a dense feature volume via a learned linear layer:

$$\mathbf{x}_0 = \text{reshape}(\mathbf{W}_{\text{fc}}\,\mathbf{e} + \mathbf{b}_{\text{fc}}) \in \mathbb{R}^{256\times4\times8\times8} \tag{2.11}$$

This initial volume of shape $(256, 4, 8, 8)$ represents a spatially compressed, channel-rich encoding of the clip's dynamics, from which the decoder will reconstruct the full spatiotemporal volume. This seed shape mirrors the output of the CNN backbone's fourth block, ensuring a consistent channel dimensionality at the interface between encoder and decoder.

Upsampling blocks. The feature volume is progressively expanded by four upsampling blocks, each doubling the spatial resolution and halving the number of channels:

$$\mathbf{x}^{(l)} = \text{ReLU}\Big(\text{BN}\Big(\text{ConvTranspose3D}_{2\times2\times2}\Big(\mathbf{x}^{(l-1)}\Big)\Big)\Big) \tag{2.12}$$

The channel schedule follows the inverse of the encoder backbone: $256 \to 128 \to 64 \to 32 \to 16$. After four blocks the spatial dimensions reach $(4 \times 16,\ 8 \times 16,\ 8 \times 16) = (64,\ 128,\ 128)$. A final trilinear interpolation step resizes the volume to the exact target shape $(T,\ H,\ W)$, accommodating variable clip lengths without requiring architectural changes:

$$\hat{\mathbf{X}} = \sigma\Big(\text{Conv3D}_{1\times1\times1}\Big(\text{Interp}_{(T,\,H,\,W)}\Big(\mathbf{x}^{(4)}\Big)\Big)\Big) \in [0,1]^{B\times1\times T\times H\times W} \tag{2.13}$$

where $\sigma(\cdot)$ is the sigmoid activation, which constrains the output to the unit interval regardless of the reconstruction target.

2.3.2 Wave Probability Decoder

The first decoder variant, `WaveDecoder`, is designed for use with optical flow or multimodal fusion inputs. Its reconstruction target is not the raw input itself but a *binary motion mask* $\mathbf{M}^* \in \{0,1\}^{B\times1\times T\times H\times W}$ derived from the L2 magnitude of the optical flow channels, as described in Section 2.1.2. The decoder is therefore trained to predict, for each spatial location and time step, the probability that significant cytoplasmic motion is occurring, effectively learning to localise the wave event within the clip.

The final $1 \times 1 \times 1$ convolution reduces the 16-channel feature map to a single-channel output, which after sigmoid activation represents the pixel-wise wave probability. The binary cross-entropy against the flow-derived pseudo-label provides a self-supervised training signal that requires no manual annotation.

2.3.3 Grayscale Reconstruction Decoder

The second variant, `GrayscaleDecoder`, shares an identical architecture with `WaveDecoder`, with the only differences being in the initialisation of the final convolutional layer and the reconstruction target.

Specifically, the weights of the final $1 \times 1 \times 1$ convolution are initialised from a zero-mean normal distribution with standard deviation 0.01, and its bias is set to zero. This initialisation places the pre-sigmoid output near zero at the start of training, so that $\sigma(0) = 0.5$ provides a neutral mid-range starting point for luminance prediction, avoiding early saturation in either direction and ensuring healthy gradient flow from the first epoch. The reconstruction target is the contrast-stretched luminance of the input, as defined in Section 2.1.2.

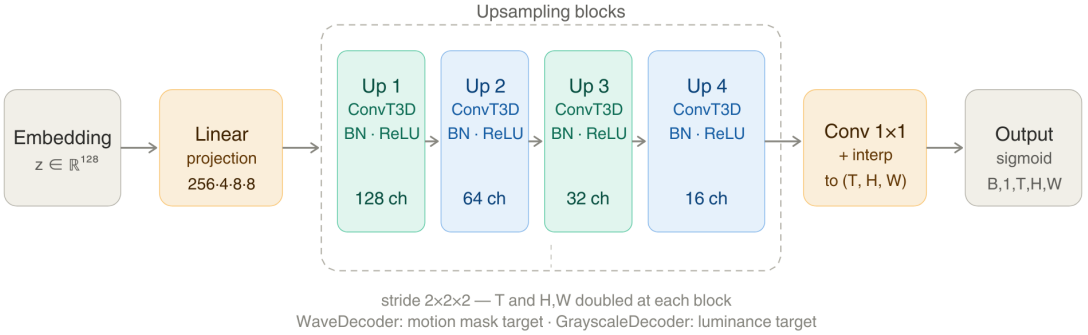


Figure 2.2: Architecture of the decoder. The latent embedding $\mathbf{z} \in \mathbb{R}^{128}$ is projected by a fully connected layer into a compact feature volume of shape (256, 4, 8, 8), which mirrors the output of the encoder backbone. Four transposed convolution blocks then progressively double the spatial resolution and halve the number of channels following the schedule $256 \to 128 \to 64 \to 32 \to 16$. A final $1 \times 1 \times 1$ convolution reduces the feature maps to a single channel, and trilinear interpolation resizes the volume to the target clip dimensions (T, H, W). A sigmoid activation constrains the output to $[0, 1]$. Two variants share this architecture: `WaveDecoder`, which reconstructs a binary motion mask derived from the optical flow magnitude, and `GrayscaleDecoder`, which reconstructs the luminance channel of the input clip.

2.4 Explainability Methods

A critical requirement for any system intended to support clinical decision-making is the ability to explain *why* the model produces a given output. Black-box representations, however rich, provide limited diagnostic value if they cannot be interrogated to reveal which spatial regions and temporal intervals drove the encoder's decision. To address this, two complementary explainability methods were implemented: a **spatial attribution** technique based on a modified formulation of Integrated Gradients, which produces per-pixel importance maps for each frame, and a **temporal occlusion** technique, which identifies which seg-

ments of the clip contribute most to the learned representation. Together, they provide a spatial and temporal account of the encoder's behaviour.

2.4.1 Integrated Gradients

Integrated Gradients (IG) [33] is a gradient-based attribution method that assigns an importance score to each input feature by integrating the gradient of the model's output with respect to the input along a straight path from a reference baseline $\mathbf{x}'$ to the actual input $\mathbf{x}$:

$$\mathrm{IG}_i(\mathbf{x}) = (\mathbf{x}_i - \mathbf{x}'_i) \cdot \int_0^1 \frac{\partial F(\mathbf{x}' + \alpha\,(\mathbf{x} - \mathbf{x}'))}{\partial \mathbf{x}_i}\,\mathrm{d}\alpha \tag{2.14}$$

where $F(\cdot)$ is the scalar score function and i indexes a single input feature, whether pixel, channel, or frame. The integral is approximated numerically using a Riemann sum over n_{steps} uniformly spaced interpolation steps.

Three adaptations to the standard formulation were required to make IG effective in the spatiotemporal, self-supervised setting of this work.

Prototype score function. Standard IG requires a scalar output to differentiate against, which is naturally available in supervised classifiers but not in a self-supervised encoder that produces an embedding vector with no associated class score. To define a meaningful scalar, a *prototype-based score function* was constructed. Two prototype embeddings are pre-computed: $\mathbf{z}_{\text{wave}}$, the mean normalised embedding of all wave videos in the dataset, and $\mathbf{z}_{\text{no-wave}}$, the corresponding prototype for the other class. The score function is then defined as the difference between the cosine similarity of the current embedding to the wave prototype and its similarity to the no-wave prototype:

$$F(\mathbf{x}) = \langle \hat{\mathbf{z}}(\mathbf{x}),\ \hat{\mathbf{z}}_{\text{wave}} \rangle - \langle \hat{\mathbf{z}}(\mathbf{x}),\ \hat{\mathbf{z}}_{\text{no-wave}} \rangle \tag{2.15}$$

where $\hat{\cdot}$ denotes L2 normalisation. This formulation is fully differentiable and provides a semantically meaningful scalar: a positive score indicates that the input is driving the embedding towards the wave class, and a large gradient at a given pixel indicates that this pixel is responsible for that alignment. Importantly, the score is defined *without* using any class labels at inference time.

Temporal median baseline. The choice of baseline $\mathbf{x}'$ is critical in IG: a poor baseline yields attributions that measure deviation from an uninformative reference rather than from a semantically grounded one. The standard all-zero baseline is particularly problematic for embryo video data, where a black frame is far from any natural input the encoder has seen during training and carries no biological meaning. Instead, a *temporal median baseline* is constructed by computing, for each pixel location (h, w) and channel c, the median value across the temporal axis:

$$\mathbf{x}'_{c,h,w} = \text{median}_{t \in [1,T]} \ \mathbf{x}_{c,t,h,w} \tag{2.16}$$

This baseline represents the *static appearance* of the oocyte, that is, the morphological structure that does not change over time. By integrating from this reference, IG attributes importance only to features that deviate from the static state, which are precisely the temporally dynamic events, including cytoplasmic wave activity, that the encoder is expected to attend to.

L2 channel norm and smoothing. For multimodal inputs comprising both grayscale frames and optical flow channels (d_x, d_y), the standard approach of summing attributions across channels risks cancellation: positive attributions on d_x may be partially negated by negative attributions on d_y at the same spatial location, even when both channels carry relevant motion information. To avoid this, the L2 norm across channels is computed at each spatial location, yielding a single non-negative saliency map per frame:

$$\text{saliency}(t, h, w) = \left\| \max(\mathbf{IG}_{:,t,h,w}, \ \mathbf{0}) \right\|_2 \tag{2.17}$$

where positive clamping is applied first to retain only the features that actively push the score in the wave direction. The resulting maps are then smoothed along the temporal axis with a one-dimensional Gaussian filter (σ_t) and spatially with a two-dimensional Gaussian filter per frame (σ_s), suppressing high-frequency pixel-level noise and producing stable, interpretable attribution maps.

SmoothGrad. To further reduce variance in the gradient estimates, a known issue with standard IG on deep networks, a SmoothGrad [41] averaging strategy

is applied. For each video, n_{smooth} independent noisy copies of the input are generated by adding zero-mean Gaussian noise with standard deviation σ_{noise}, and the IG attributions are computed and averaged across all copies:

$$\widetilde{\text{IG}}(\mathbf{x}) = \frac{1}{n_{\text{smooth}}} \sum_{k=1}^{n_{\text{smooth}}} \text{IG}(\mathbf{x} + \boldsymbol{\epsilon}_k), \quad \boldsymbol{\epsilon}_k \sim \mathcal{N}(\mathbf{0}, \sigma^2_{\text{noise}}\mathbf{I}) \tag{2.18}$$

2.4.2 Temporal Occlusion

While Integrated Gradients reveals *where* the encoder attends within each frame, it does not directly answer the question of *when* the most informative events occur along the temporal axis. Temporal Occlusion addresses this by systematically masking a sliding window of consecutive frames and measuring the resulting perturbation in the encoder's output embedding:

$$\text{importance}(t) = \max\Big(0,\ \|\mathbf{z}(\mathbf{x})\| - \Big\|\mathbf{z}(\mathbf{x}_{\setminus W_t})\Big\|\Big) \tag{2.19}$$

where $\mathbf{x}_{\setminus W_t}$ denotes the clip with the temporal window $W_t = [t - \lfloor w/2 \rfloor,\ t + \lfloor w/2 \rfloor]$ occluded, and $\|\mathbf{z}(\cdot)\|$ is the L2 norm of the encoder embedding. Frames whose removal causes a large drop in embedding norm are interpreted as temporally critical: the encoder relies on the content of those frames to produce its representation. The positive clamping ensures that frames whose occlusion accidentally improves the embedding norm, a degenerate case that may occur near the boundaries of the clip, are assigned zero importance rather than a negative value. The resulting profile is normalised to $[0, 1]$ before visualisation.

Window size. A window size of $w = 1$ occludes a single frame at each position, providing fine-grained temporal attribution. However, for architectures with a large temporal receptive field, such as the 3D CNN backbone which spans three frames per convolution, occluding a single frame may be partially compensated by its immediate neighbours, underestimating its true importance. Larger windows ($w = 33$ or $w = 40$ in the experiments of Section 3.10.2) mitigate this effect by masking a temporally extended neighbourhood simultaneously, at the cost of reduced temporal resolution.

Occlusion strategies. Three occlusion strategies were implemented and evaluated:

- **Zero occlusion:** the masked frames are replaced with all-zero pixels. This is the most disruptive strategy and serves as a baseline, but it introduces an obvious out-of-distribution input that may cause the encoder to respond to the artefact rather than to the absence of the original content.
- **Mean occlusion:** the masked frames are replaced with the temporal mean frame of the clip, computed once before the occlusion loop. This strategy preserves global brightness and spatial structure while removing the temporal variation at position t, providing a more naturalistic perturbation than zero-filling.
- **Blur occlusion:** a strong separable Gaussian blur ($k = 31$, $\sigma = 10$) is applied to the masked frames, destroying fine spatial detail while preserving global intensity gradients. This strategy is particularly suited to distinguishing whether the encoder is exploiting fine-grained texture at a given time step or merely responding to the coarse spatial structure of the frame.

The mean and blur strategies are preferred over zero occlusion for biological video data, where black frames are strongly out-of-distribution and likely to confound the attribution signal.

Attention map (Transformer only). For completeness, an additional attention-based attribution method was implemented for the Transformer encoder variant. The per-head self-attention weights extracted from each layer are averaged across heads and layers, and the resulting column sums of the attention matrix are interpreted as a measure of how much each temporal position is attended to by the rest of the sequence. An attention rollout correction [42] is optionally applied to propagate attention through layers. This method is not available for the GRU and LSTM encoders, which do not expose attention weights, and was therefore not used in the primary experimental analysis.

Chapter 3

Experimental Results

This chapter presents the full experimental evaluation of the proposed framework. It is organised to follow the natural progression from model configuration to representation quality. The chapter opens with a series of ablation studies that isolate the contribution of individual design choices, namely the contrastive loss function, input modality, temporal architecture, and batch size, using the original fifteen-video dataset as the common reference. A validation analysis on the selected GRU configuration confirms the absence of overfitting, and a summary table collects the key training metrics across all runs. The chapter then turns to VICReg as a batch-size-independent alternative, comparing its training dynamics and generalisation behaviour against NT-Xent across all three temporal architectures. A data augmentation study follows, in which two balanced datasets of 24 and 60 videos are constructed and used to evaluate the joint NT-Xent + Triplet objective under more favourable data conditions. The second half of the chapter assesses the quality of the learned representations directly: latent space projections using PCA, t-SNE, UMAP, and Isomap are presented for both the original and augmented configurations, followed by a quantitative clustering evaluation using Gaussian Mixture Models. The chapter closes with the explainability analysis, combining Integrated Gradients spatial attribution maps and temporal occlusion profiles to verify that the encoder's attention is anchored to the biologically relevant phases of embryo development.

3.1 Dataset and Data Preprocessing

In the domain of Assisted Reproductive Technology, data collection is notoriously challenging due to strict privacy regulations and the highly sensitive nature of medical records. Acquiring large-scale, annotated datasets is therefore a major hurdle. For this study, a proprietary dataset called *EXPREMIA_DATASET* was provided by the *Tortorella Clinic*. It consists of 15 high-quality time-lapse embryo videos. Despite the limited size of the dataset, each video was meticulously analysed and manually annotated by an expert embryologist, providing a highly reliable qualitative ground truth regarding the presence or absence of cytoplasmic waves.

3.1.1 Preprocessing Pipeline

To facilitate accurate computational analysis and remove irrelevant background artefacts, each frame undergoes a multi-step preprocessing pipeline before being fed to the network.

1. **Temporal Truncation:** The primary objective of this study is the spatiotemporal analysis of cytoplasmic waves, a biological event that occurs prior to the first cell divisions. Once the embryo reaches the cleavage phase and begins to divide into multiple distinct cells, such as the 2-cell stage *t2*, the 3-cell stage *t3*, and so forth, the global cytoplasmic wave can no longer be visually or physically observed.

   To prevent the neural network from learning irrelevant features from post-cleavage stages, the temporal scope of the videos was strictly truncated. Only the first 125 frames of each time-lapse video were extracted for analysis, capturing the continuous developmental phase where the wave is expected to occur. This yields a standardised dataset of 125×15 total frames.

2. **Segmentation and Spatial Stabilization:** A dedicated segmentation model processes the first 125 frames to extract a precise mask of the embryo. This mask is then used to stabilise the embryo's spatial position across the temporal axis. By aligning the embryo and removing rigid global

translations such as drifting in the culture medium, the subsequent motion analysis can focus exclusively on internal cellular dynamics.

3. **Visual Enhancement:** To accentuate the subtle visual characteristics of the cytoplasmic fluid and suppress background noise, the stabilised and masked frames are subjected to:

    - *Temporal frame normalisation*, to standardise intensity values across the sequence.
    - *Contrast Limited Adaptive Histogram Equalization (CLAHE)*, to improve local contrast without amplifying noise.
    - *Temporal pixel smoothing*, to reduce high-frequency artefacts across consecutive frames.

The complete preprocessing workflow is summarised in Figure 3.1.

3.1.2 Optical Flow Extraction

Following spatial and visual enhancement, motion features are explicitly extracted. The *Dual TV-L1 (Total Variation L1)* optical flow algorithm is applied to the preprocessed video sequence, computing dense motion fields that yield horizontal (d_x) and vertical (d_y) displacement vectors for every pixel between consecutive frames. TV-L1 was selected for its robustness to illumination changes and its capacity to preserve motion discontinuities, both critical properties when analysing the subtle internal dynamics of a developing embryo. This optical flow extraction stage is illustrated in Figure 3.1.

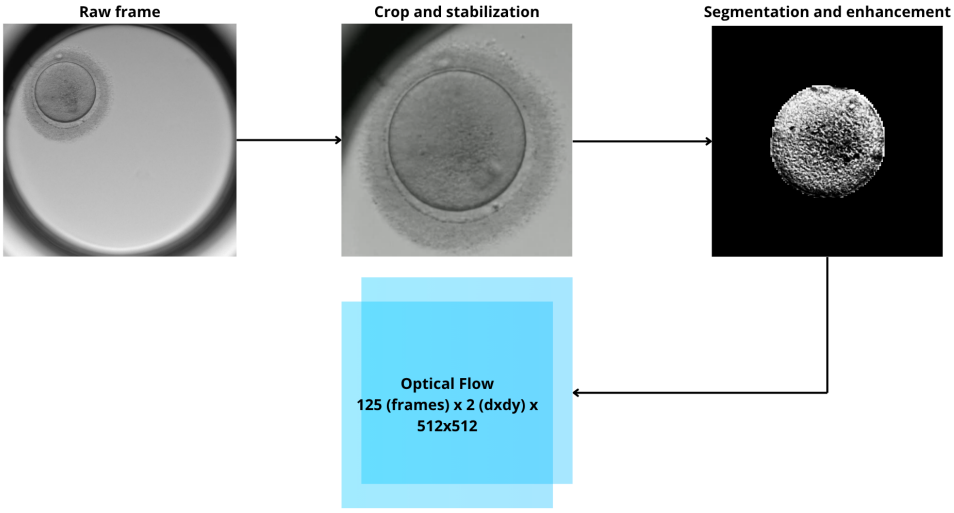


Figure 3.1: Preprocessing pipeline used before model training, including segmentation, spatial stabilization, visual enhancement, and optical flow extraction.

3.1.3 Clip Generation for Contrastive Learning

To prepare the dataset for the self-supervised contrastive learning phase, an additional temporal partitioning step was required. Feeding the entire 125-frame sequence simultaneously is memory-intensive and sub-optimal for contrastive objectives. The complete spatiotemporal volume, comprising the 125 preprocessed frames together with the corresponding optical flow fields (d_x, d_y), was therefore partitioned into N shorter, overlapping clips via a sliding-window strategy. Given a fixed `clip_len` and a temporal `stride`, each pass of the window extracts one temporally coherent sub-sequence. This approach serves a dual purpose: it keeps memory consumption tractable and acts as implicit temporal data augmentation, substantially increasing the number of distinct views available for the contrastive objective.

3.2 Baseline Model Configuration

Prior to conducting the ablation studies, a baseline model configuration was established to serve as the common reference point for all comparative experiments. Unless otherwise stated, every ablation isolates a single variable while

keeping this configuration fixed.

The baseline architecture consists of a **CNN–GRU encoder** paired with a convolutional decoder. Spatial features are first extracted from each frame by a convolutional backbone with a base channel width of 32, operating on 128×128 input frames. The resulting per-frame feature maps are then processed along the temporal axis by a two-layer *Gated Recurrent Unit* (GRU) with a hidden size of 128, producing a compact latent representation of dimension 128 that captures the spatiotemporal dynamics of the clip. The decoder mirrors this structure to reconstruct the input from the latent space.

Clips of length 32 frames are extracted from each preprocessed video using a sliding window with a temporal stride of 4, yielding a substantial number of overlapping views per video while maintaining temporal coherence within each clip. Training is performed for 100 epochs with a batch size of 16.

The complete set of baseline hyperparameters is summarised in Table 3.1.

Table 3.1: Baseline model hyperparameters used across all ablation studies.

Hyperparameter	**Value**
Spatial input size	128×128
Clip length	32 frames
Temporal stride	4 frames
CNN base channels	32
Temporal model	GRU
GRU hidden size	128
GRU layers	2
Embedding dimension	128
Batch size	16
Training epochs	100

3.3 Ablation Studies

To determine the optimal configuration of the spatiotemporal autoencoder, a systematic series of ablation experiments was conducted. Each study isolates a single design choice—objective function, input modality, encoder architecture, decoder architecture—while keeping all other hyperparameters fixed, so that the contribution of each factor can be assessed independently.

3.3.1 Contrastive Loss Function

The first ablation compared four self-supervised objective functions: NT-Xent, InfoNCE and VICReg. All other architectural hyperparameters were held constant throughout this comparison.

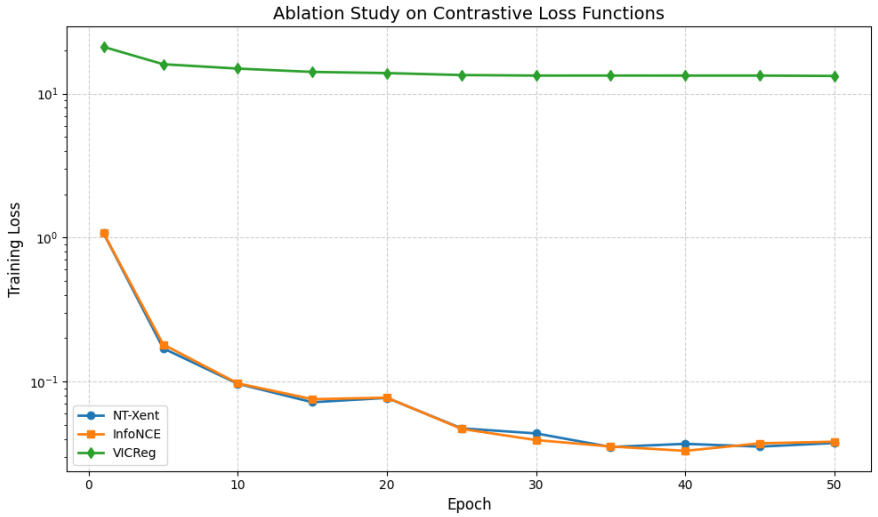


Figure 3.2: Training curves for three contrastive loss functions. NT-Xent and InfoNCE exhibit nearly identical convergence behaviour, while VICReg operates at substantially different loss scales due to differences in their formulation.

As shown in Figure 3.2, NT-Xent and InfoNCE converge along virtually identical trajectories and reach the lowest numerical loss values. This result is expected: NT-Xent is in essence a temperature-scaled instantiation of the InfoNCE objective, and the two are theoretically equivalent under standard settings. VICReg, on the other hand, optimizes for structurally different objectives, which explains why its loss curves stabilize at significantly higher numerical values. This difference in scale does not necessarily imply lower representation quality, but it complicates direct comparisons between methods based solely on loss magnitude.

On the basis of convergence speed and numerical stability, **NT-Xent** was selected as the objective function for all subsequent experiments.

3.3.2 Input Modality

Having fixed the loss function, the second ablation investigated how the choice of input modality affects the quality of the learned representations. Three con-

figurations were evaluated:

- **Spatial-only:** the network receives only the preprocessed grayscale (or RGB) frames, providing appearance information without explicit motion cues.
- **Motion-only:** the network receives only the 2-channel optical flow fields (d_x, d_y), encoding temporal dynamics while discarding raw appearance.
- **Multimodal fusion:** both modalities are concatenated along the channel dimension, allowing the model to exploit visual morphology and explicit motion simultaneously.

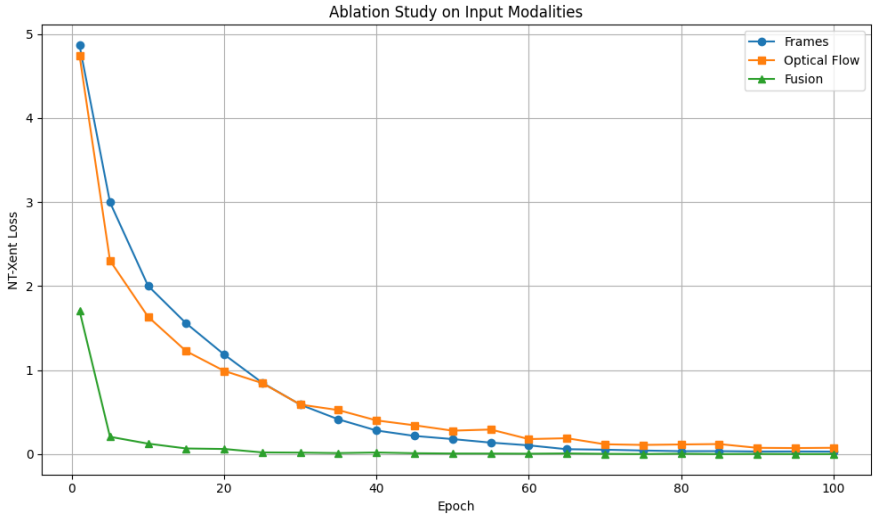


Figure 3.3: NT-Xent training curves for the three input configurations. The multimodal fusion setup converges markedly faster and reaches a substantially lower final loss than either unimodal baseline.

As illustrated in Figure 3.3, all three configurations progressively reduce the NT-Xent loss; however, the multimodal fusion configuration achieves notably faster convergence and a lower final loss value. This result suggests that appearance and motion information are complementary: the optical flow fields provide explicit spatiotemporal cues that the RGB frames alone cannot encode, while the frames supply rich morphological context that the flow fields lack. Their combination therefore yields more discriminative latent representations for the contrastive objective.

3.3.3 Temporal Model Architecture and Batch Size

Having established the optimal loss function and input modality, the next ablation jointly investigates the choice of temporal model architecture and the effect of batch size on the contrastive learning objective. Three recurrent and attention-based architectures were compared: LSTM, GRU, and Transformer. Each was trained for approximately 200 epochs under four batch size conditions: 16, 32, 64, and 128. Larger batch sizes were not explored due to hardware memory constraints.

Role of batch size in contrastive learning. Batch size plays a structurally different role in contrastive learning compared to standard supervised training. In the NT-Xent framework, all samples within a batch that are not augmented views of the same clip are treated as negatives. Consequently, a larger batch size directly increases the number of negative pairs presented to the encoder at each optimisation step. This harder, more informative training signal raises the absolute value of the NT-Xent loss, as visible across all three panels of Figure 3.4, but in principle drives the encoder to produce more discriminative and geometrically structured embeddings. The trade-off is therefore between training stability, which is favoured by smaller batches, and representation richness, which is favoured by larger batches, subject to the available computational budget.

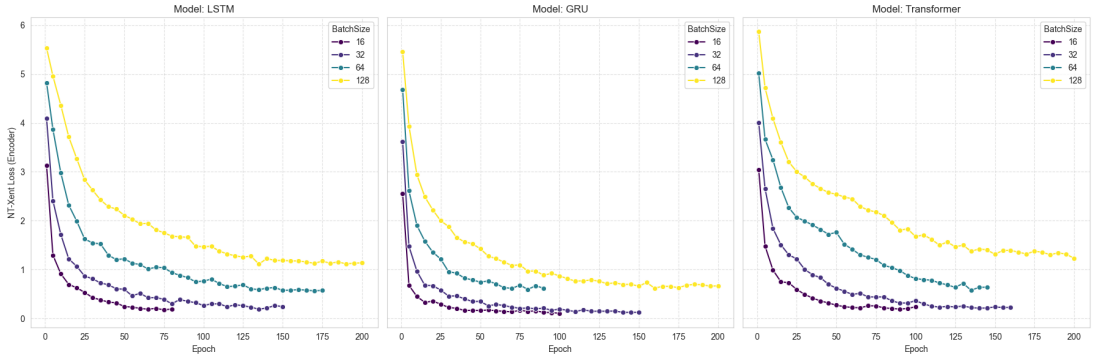


Figure 3.4: NT-Xent encoder loss curves over 200 epochs for LSTM, GRU, and Transformer under four batch size conditions (16, 32, 64, 128). In all three architectures, larger batch sizes yield higher absolute loss values throughout training, consistent with the increased number of negative pairs in the NT-Xent objective.

Comparison across architectures. Figure 3.5 summarises the final NT-Xent loss achieved by each model-batch size combination. The GRU consistently attains the lowest final loss across all batch sizes, with the exception of batch size 64, where the LSTM reaches a marginally lower value. The Transformer, despite its greater representational capacity, does not outperform the recurrent models in this regime, a result that is likely attributable to the limited dataset size, which prevents the self-attention mechanism from fully exploiting long-range dependencies.

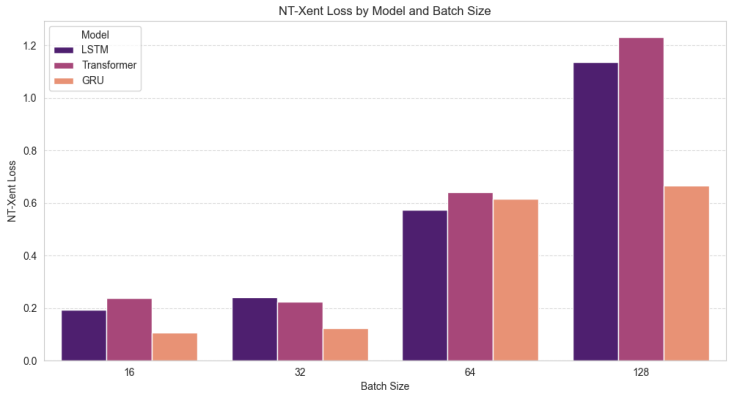
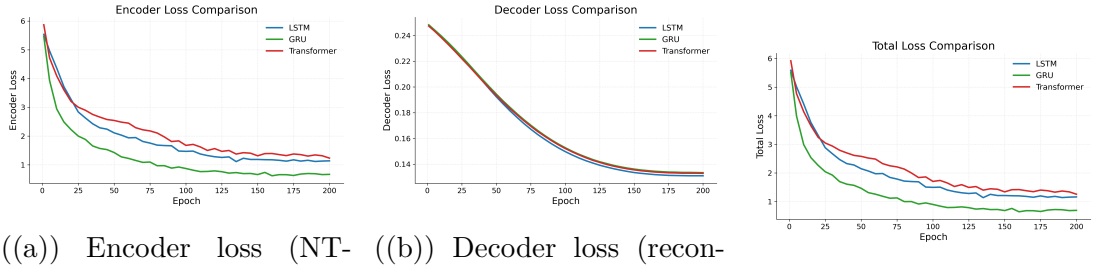


Figure 3.5: Final NT-Xent loss at epoch 200 for each combination of temporal architecture and batch size. The GRU achieves the lowest loss in three out of four batch size conditions; at batch size 64 the LSTM yields a marginally lower value.

On the basis of these results, the **GRU** architecture with a **batch size of 128** was selected for the subsequent experiments. Although larger batch sizes expose the model to more negatives per step, batch size 16 consistently yields the lowest absolute loss across all three architectures, indicating that the available dataset is too small to benefit from the denser negative sampling of larger batches without incurring overfitting or instability. The GRU's efficiency advantage over the LSTM, namely fewer parameters with comparable or superior convergence, further motivates this choice given the constrained computational setting.

3.3.4 Training Dynamics

To gain a more granular understanding of how each architectural component contributes to the overall training dynamics, the encoder loss, decoder reconstruction loss, and combined loss were monitored independently throughout training. This analysis was conducted at batch size 128, the most demanding negative-sampling regime, across all three temporal architectures.



((a)) Encoder loss (NT-Xent).

((b)) Decoder loss (reconstruction).

((c)) Combined loss.

Figure 3.6: Training loss curves at batch size 128 for LSTM, GRU, and Transformer. Left: encoder NT-Xent loss. Centre: decoder reconstruction loss. Right: combined loss. The GRU converges faster and reaches a lower final encoder loss, while decoder dynamics remain comparable across architectures. The combined loss closely mirrors the encoder loss, indicating that the contrastive term is the dominant component of the joint objective.

The encoder loss (Figure 3.6(a)) confirms the trend observed in the previous ablation: the GRU achieves faster convergence and a lower final loss compared to LSTM and Transformer. This behaviour is consistent with both the architectural properties of the GRU, which with fewer parameters than the LSTM and a simpler gating mechanism is less prone to overfitting on a small dataset, and with the limited number of training examples available, which penalises more complex models.

The decoder loss (Figure 3.6(b)), by contrast, evolves similarly across all three architectures. This suggests that the reconstruction capability of the decoder is not strongly sensitive to the choice of temporal model in the encoder, and that all three architectures learn a latent representation from which the decoder can reconstruct the input with comparable fidelity.

Finally, the combined loss (Figure 3.6(c)) closely mirrors the encoder loss in both shape and relative ordering of the three curves. This indicates that the con-

trastive term dominates the joint objective, and that the overall training dynamics are primarily driven by the encoder's ability to learn discriminative spatiotemporal representations rather than by the decoder's reconstruction performance.

Validation Analysis

To assess whether the model generalises beyond the training set, a validation study was conducted on the GRU architecture at batch size 128. The dataset was partitioned at the video level using an 80/20 split, yielding a training set of 12 videos and a validation set of 3 videos. The split was stratified to preserve class balance: the validation set contains one video without cytoplasmic waves and two videos with waves, reflecting the approximate class distribution of the full dataset. All clips extracted from a given video were assigned exclusively to either the training or the validation set, preventing any temporal leakage between the two splits.



((a)) Encoder loss. ((b)) Decoder loss. ((c)) Combined loss.

Figure 3.7: Training and validation loss curves for the GRU model at batch size 128. Left: encoder NT-Xent loss. Centre: decoder reconstruction loss. Right: combined loss. In all three cases the validation loss tracks the training loss closely, with no divergence indicative of overfitting.

As shown in Figure 3.7, the validation loss closely tracks the training loss throughout the entire training procedure for all three components of the objective. Neither the encoder nor the decoder exhibits the characteristic divergence between training and validation curves that would signal overfitting. The combined loss, dominated by the encoder term as established in the previous section, confirms this behaviour at the level of the joint objective.

This result suggests that, despite the small size of the dataset, the model is learning spatiotemporal representations that generalise to unseen embryo videos

rather than memorising the specific dynamics of the training sequences. The use of overlapping clip generation as implicit data augmentation, combined with the self-supervised contrastive objective that does not rely on explicit labels, likely contributes to this generalisation capacity.

Ablation Summary

Tables 3.2 and 3.3 summarise the quantitative outcomes of all ablation experiments conducted in this study. Table 3.2 reports the final loss value achieved by each configuration, while Table 3.3 distils the key design decisions into the optimal configuration adopted for the subsequent experiments.

Table 3.2: Complete ablation results. For each study, all tested configurations are reported with their final NT-Xent loss value. The selected configuration for each study is highlighted in bold.

Ablation	**Configuration**	**Final Loss**	**Epoch**
Loss Function	**NT-Xent**	**0.0374**	**50**
	InfoNCE	0.0382	50
	VICReg	13.2923	50
Input Modality	Spatial-only (RGB frames)	0.0309	100
	Motion-only (Optical flow)	0.0761	100
	Multimodal fusion	**0.0015**	**100**
Architecture × Batch Size	LSTM, bs=16	0.1926	80
	LSTM, bs=32	0.2415	150
	LSTM, bs=64	0.5747	175
	LSTM, bs=128	1.1374	200
	GRU, bs=16	**0.1050**	**100**
	GRU, bs=32	0.1234	150
	GRU, bs=64	0.6164	90
	GRU, bs=128	**0.6669**	**200**
	Transformer, bs=16	0.2376	100
	Transformer, bs=32	0.2247	160
	Transformer, bs=64	0.6404	145
	Transformer, bs=128	1.2316	200
Validation (GRU, bs=128)	Encoder loss (train / val)	0.6669 / 0.5466	200
	Decoder loss (train / val)	0.1334 / 0.1328	200
	Combined loss (train / val)	0.6936 / 0.4639	200

Table 3.3: Final selected configuration resulting from the ablation studies.

Design Choice	**Selected Option**	**Motivation**
Loss function	NT-Xent	Fastest convergence, lowest final loss
Input modality	Multimodal fusion (RGB + Optical flow)	Markedly lower loss; complementary cues
Temporal model	GRU	Best performance on small dataset
Batch size	128	Most reliable and stable training behaviour
Generalisation	No overfitting detected	Val loss tracks train loss consistently

3.4 VICReg as Batch-Size-Free Alternative

A fundamental limitation of the NT-Xent objective is its structural dependence on batch size: the number of negative pairs available at each optimisation step grows linearly with the batch size, meaning that small batches imposed by hardware constraints necessarily reduce the informativeness of the contrastive signal. To investigate whether this bottleneck could be circumvented, an additional series of experiments was conducted using **VICReg** (Variance-Invariance-Covariance Regularisation) [17] as an alternative self-supervised objective.

Unlike contrastive losses, VICReg does not rely on negative pairs at all. Instead, it jointly optimises three complementary regularisation terms applied directly to the distribution of embeddings within a batch: an *invariance* term that encourages the representations of two views of the same clip to be similar; a *variance* term that penalises representational collapse by enforcing a minimum standard deviation along each embedding dimension; and a *covariance* term that decorrelates the components of the embedding vector to prevent redundancy. The three terms are weighted by scalar coefficients λ, μ, and ν respectively, whose recommended values in the original paper are $\lambda = 25$, $\mu = 25$, $\nu = 1$ [17].

This formulation makes VICReg theoretically attractive under computational constraints: because the objective does not require negatives, its quality does not degrade as batch size decreases, potentially offering a more stable learning signal than NT-Xent in resource-limited settings.

3.4.1 Architecture Comparison under VICReg

The three temporal architectures, LSTM, GRU, and Transformer, were trained under VICReg using the recommended coefficients $(\lambda, \mu, \nu) = (25, 25, 1)$

and the same baseline hyperparameters established in Section 3.4.

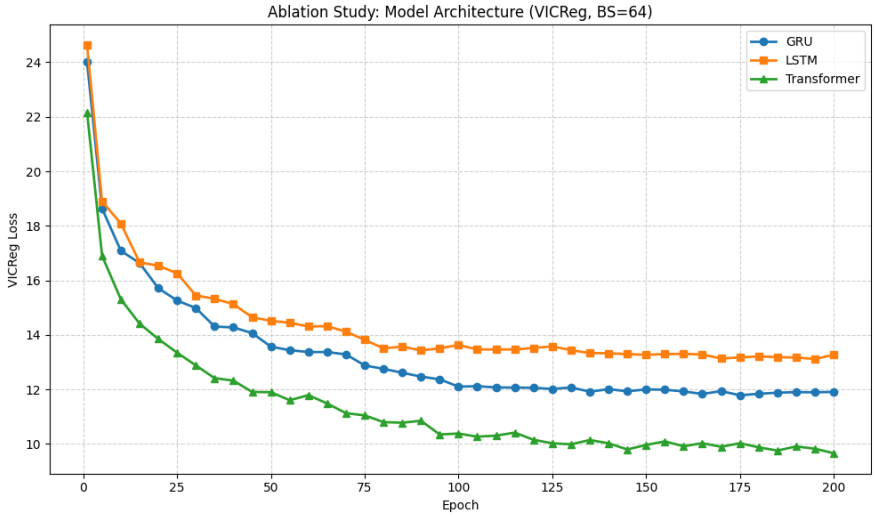
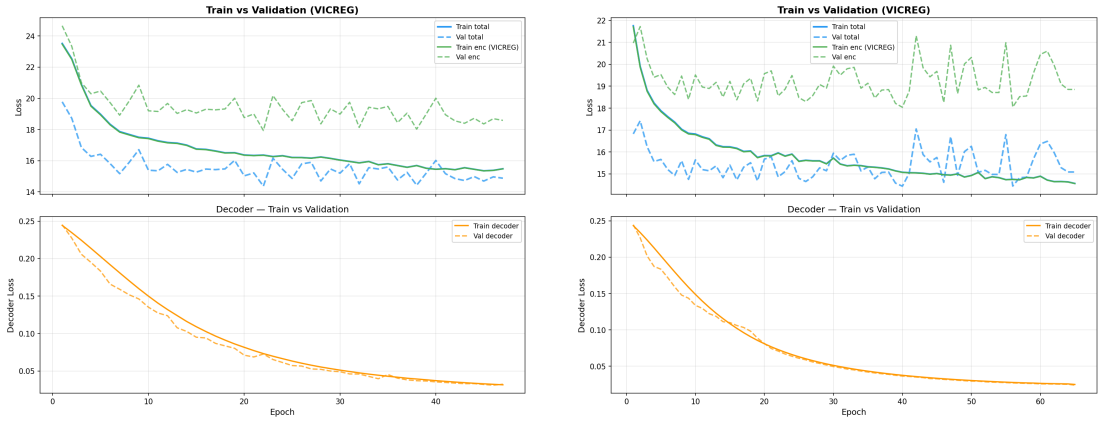


Figure 3.8: VICReg training loss curves for LSTM, GRU, and Transformer. All three architectures converge, but the absolute loss values are substantially higher than those observed under NT-Xent, consistent with the different scale of the VICReg objective. The Transformer achieves the lowest final training loss among the three models.

As shown in Figure 3.8, all three architectures converge under VICReg, though the absolute loss values are considerably higher than those observed with NT-Xent, which is an expected consequence of the different mathematical formulation of the objective. Among the three models, the Transformer attains the lowest final training loss. However, as discussed in the following subsection, training loss alone is not a sufficient indicator of generalisation quality under VICReg.

3.4.2 Validation Stability: GRU vs. Transformer

To assess generalisation, training and validation loss curves were monitored separately for the GRU and the Transformer under VICReg. The two models are compared side by side in Figure 3.9.



((a)) GRU: train vs. validation.

((b)) Transformer: train vs. validation.

Figure 3.9: Train and validation loss curves (encoder and decoder components shown separately) for GRU and Transformer under VICReg with $(\lambda, \mu, \nu) = (25, 25, 1)$. The GRU validation loss tracks the training loss more consistently, while the Transformer exhibits pronounced oscillations in the encoder validation curve, indicating instability and a tendency towards overfitting.

The validation analysis reveals an important qualitative difference between the two architectures. For the GRU (Figure 3.9(a)), the encoder validation loss, while oscillating, broadly follows the downward trend of the training loss, and the decoder validation curve tracks its training counterpart very closely throughout. This suggests that the GRU is learning representations that generalise reasonably well to unseen clips, despite a degree of instability in the contrastive component. For the Transformer (Figure 3.9(b)), the encoder validation loss oscillates markedly and diverges from the training curve, indicating a clear tendency towards overfitting in the representational component of the objective. The decoder, by contrast, behaves stably in both architectures, a pattern consistent with what was observed under NT-Xent, further confirming that decoder reconstruction dynamics are relatively insensitive to the choice of temporal model and objective function.

Overall, although the Transformer achieves a marginally lower training loss, the GRU offers substantially better generalisation stability under VICReg, making it the more reliable choice in this low-data regime.

3.4.3 Sensitivity Analysis: VICReg Coefficients on GRU

The VICReg objective is governed by three weighting coefficients (λ, μ, ν) that balance the invariance, variance, and covariance terms respectively. To assess the sensitivity of the GRU model to these hyperparameters and to verify the recommendations of the original paper, four coefficient configurations were evaluated: $(\lambda = 25, \mu = 25, \nu = 1)$, $(\lambda = 25, \mu = 25, \nu = 10)$, $(\lambda = 50, \mu = 25, \nu = 1)$, and $(\lambda = 50, \mu = 25, \nu = 5)$.

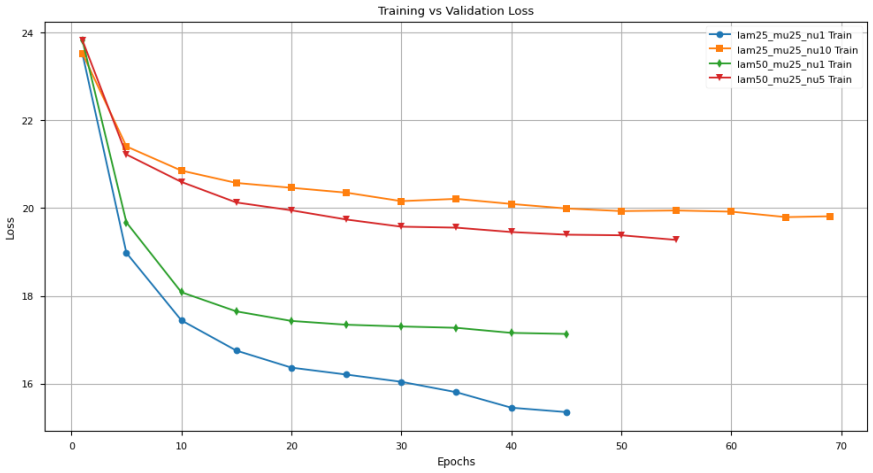


Figure 3.10: GRU training loss under four VICReg coefficient configurations. The configuration $(\lambda = 25, \mu = 25, \nu = 1)$ achieves the fastest convergence and the lowest final loss, consistent with the recommendations of the original VICReg [17].

As illustrated in Figure 3.10, the configuration $(\lambda = 25, \mu = 25, \nu = 1)$ achieves the fastest convergence and the lowest final training loss among those tested. Increasing ν to 10 or raising λ to 50 leads to slower convergence and higher stabilisation values, suggesting that over-weighting the covariance or invariance terms impairs the balance between the three regularisation objectives. These results are consistent with the empirical recommendations provided in the original VICReg paper, lending confidence to the choice of default coefficients for this study.

3.4.4 NT-Xent vs. VICReg

Table 3.4 summarises the final training loss, best validation loss, and stopping epoch for each architecture under both objective functions. All VICReg runs used coefficients ($\lambda = 25, \mu = 25, \nu = 1$) and early stopping with patience 20. NT-Xent results are reported at the epoch of lowest validation loss for consistency.

Table 3.4: Comparative results for NT-Xent and VICReg across all three temporal architectures. VICReg losses are not directly comparable in magnitude to NT-Xent losses due to the different scale of each objective.

Loss	Model	Final Train Loss	Best Val Loss	Stop Epoch
NT-Xent BS=128	LSTM	1.1990	0.6966	175
	GRU	**0.6823**	**0.4572**	**170**
	Transformer	1.3404	0.8005	195
VICReg	LSTM	16.3415	14.9864	65
	GRU	**15.3506**	**14.3629**	**47**
	Transformer	15.0188	14.4397	65

Despite the Transformer achieving a marginally lower best validation loss than the GRU under VICReg (14.44 vs. 14.36), the GRU converges in fewer epochs and exhibits substantially more stable validation dynamics, as shown in Figure 3.9. Combined with its superior performance under NT-Xent, where it achieves the lowest training loss across all architectures and batch sizes, the GRU is confirmed as the most robust temporal model for this dataset and task. The analysis also highlights a key practical distinction between the two objectives: under NT-Xent, the lowest numerical error is obtained with batch size 16, but the batch-size-128 configuration shows more reliable behaviour across training and validation dynamics. This confirms that loss magnitude alone is not sufficient for model selection. VICReg removes the structural dependence on batch size, but in these experiments training proved less stable and showed signs of overfitting.

3.4.5 Data Augmentation

One of the most immediate limitations of the available dataset is its size. Fifteen videos, twelve with cytoplasmic wave and three without, represent a solid

clinical ground truth but a very constrained training set for any data-driven method. To investigate how the model's behaviour changes as more examples become available, an offline augmentation pipeline was developed to generate physically saved copies of the original videos, effectively expanding the dataset in a controlled and reproducible way.

The augmentation strategy was designed with one key constraint in mind: transformations should remain biologically plausible, meaning the augmented videos should still look like realistic embryo recordings rather than artefacts.

The pipeline supports two families of transformations. Geometric augmentations are applied by default and include random cropping with bilinear resize, small translations, rotations, and horizontal and vertical flips. Intensity augmentations, namely brightness jitter, Gaussian blur, and additive noise, are disabled by default and operate on the frame channels only, since applying brightness shifts or blur to optical flow vectors has no physically meaningful interpretation. When noise is enabled, a substantially smaller standard deviation is used for the flow channels, specifically $0.1\times$ the frame noise level, to avoid corrupting the motion signal.

Using this pipeline, two augmented datasets were generated from the original fifteen videos:

- A **24-video dataset** comprising 12 wave and 12 no-wave videos, obtained by generating augmented copies of the three no-wave examples to bring the minority class to parity with a representative subset of the wave class.
- A **60-video dataset** comprising 30 wave and 30 no-wave videos, obtained by augmenting both classes to a balanced target of 30 examples each.

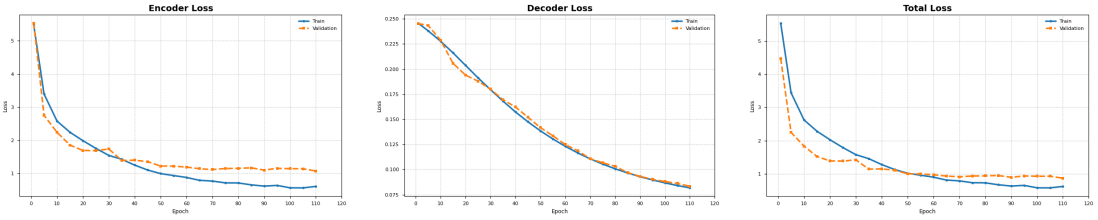
Both datasets preserve the original 80/20 video-level train/validation split: augmented copies of a video are always assigned to the same partition as their source, so no information from the validation set leaks into the training set through the augmentation process.

3.5 Results on Augmented Datasets

The experiments described in this section extend the analysis to the two augmented datasets introduced in Section 3.4.5, with the dual objective of assessing whether a larger training set improves generalisation and of evaluating the joint NT-Xent + Triplet objective under more favourable data conditions.

3.5.1 NT-Xent + Triplet 24-video dataset

The first experiment combines NT-Xent with a triplet term weighted at $\lambda_{\text{triplet}} = 0.1$, trained on the 24-video balanced dataset. At this weighting, the triplet component acts as a mild geometric regulariser on top of the dominant contrastive objective, contributing a margin-enforcing signal without substantially altering the overall training dynamics.



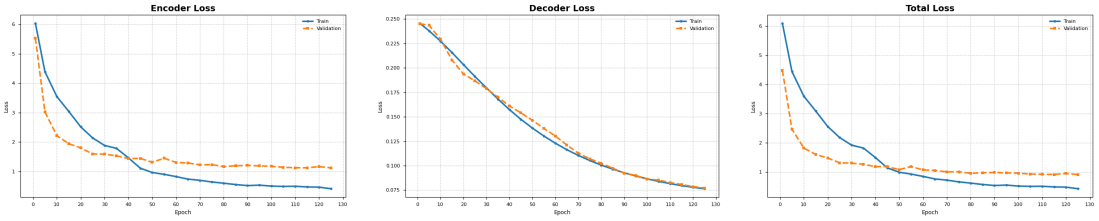
((a)) Encoder loss. ((b)) Decoder loss. ((c)) Combined loss.

Figure 3.11: Training and validation loss curves for NT-Xent + Triplet ($\lambda_{\text{triplet}} = 0.1$) on the 24-video dataset. The encoder exhibits a mild divergence between training and validation curves in the later epochs, indicating a light degree of overfitting. Decoder and combined dynamics remain stable throughout.

The training curves in Figure 3.11 show overall stable convergence. The encoder loss reveals a modest gap between the training and validation curves that becomes apparent in the later stages of training, a light overfitting signal that, given the limited size of the dataset, is not unexpected. Importantly, the divergence remains contained and does not accelerate, suggesting that the regularising effect of the triplet term is partially compensating for the data constraints. The decoder behaves consistently, with training and validation losses tracking each other closely throughout, and the combined loss mirrors the encoder dynamics as expected from the weighting scheme.

NT-Xent + Triplet, $\lambda_{\text{triplet}} = 0.5$

Increasing the triplet weight to $\lambda_{\text{triplet}} = 0.5$ substantially amplifies the margin-based component of the joint objective, forcing the encoder to devote a larger fraction of its learning capacity to explicitly separating the two classes in the embedding space.



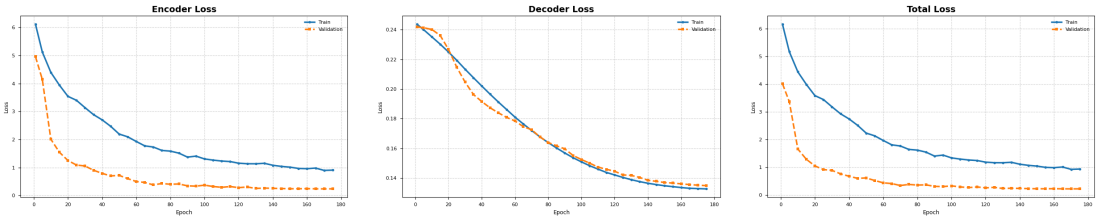
((a)) Encoder loss. ((b)) Decoder loss. ((c)) Combined loss.

Figure 3.12: Training and validation loss curves for NT-Xent + Triplet ($\lambda_{\text{triplet}} = 0.5$) on the 24-video dataset. The overall dynamics are comparable to the $\lambda = 0.1$ configuration, with a similarly light encoder overfitting signal and stable decoder behaviour.

Despite the stronger triplet contribution, the training dynamics remain qualitatively similar to the previous configuration. A light encoder overfitting signal is again visible, of comparable magnitude to the $\lambda_{\text{triplet}} = 0.1$ case, suggesting that the degree of generalisation difficulty is primarily driven by the dataset size rather than by the objective weighting. The decoder loss remains well-behaved, and the combined loss converges smoothly. These results indicate that, at 24 videos, both triplet weightings produce similarly regularised solutions and that further improvements in generalisation are unlikely to come from tuning λ_{triplet} alone.

3.5.2 NT-Xent + Triplet 60-video dataset

Expanding the training set to 60 videos and using $\lambda_{\text{triplet}} = 0.5$ produces a noticeably different picture on the encoder side.



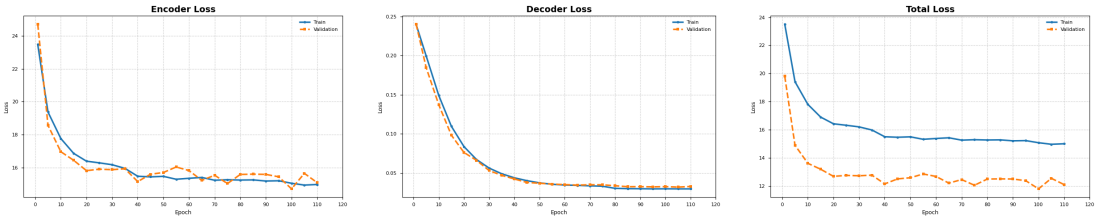
((a)) Encoder loss. ((b)) Decoder loss. ((c)) Combined loss.

Figure 3.13: Training and validation loss curves for NT-Xent + Triplet ($\lambda_{\text{triplet}} = 0.5$) on the 60-video dataset. The encoder no longer exhibits overfitting, with training and validation curves tracking each other consistently. The decoder shows a modest train-validation gap, which remains stable and does not deteriorate over training.

The encoder training and validation losses track each other closely throughout the entire training run, with no sign of the divergence observed on the smaller datasets. This improvement is attributable to the larger and more balanced training set, which provides the model with a broader variety of augmented views and reduces the risk of the encoder latching onto video-specific features rather than class-level ones. The decoder, by contrast, shows a slightly more pronounced gap between training and validation loss compared to the encoder, a mild but consistent overfitting signal that nonetheless remains stable and does not worsen over epochs. This asymmetry between encoder and decoder behaviour is consistent with what was observed on the original dataset: the reconstruction objective is more sensitive to individual video characteristics than the contrastive one, and a fixed held-out split necessarily limits how representative the validation set can be at this scale. Addressing this properly would require a cross-validation scheme, ideally a leave-one-video-out or stratified k-fold procedure, which given the computational cost of training is left as a direction for future work.

3.5.3 VICReg with GRU 60-video dataset

The final experiment revisits VICReg on the 60-video dataset, motivated by the instability that characterised its training dynamics on the original fifteen videos.



((a)) Encoder loss. ((b)) Decoder loss. ((c)) Combined loss.

Figure 3.14: Training and validation loss curves for VICReg on the 60-video dataset. In contrast to the oscillatory behaviour observed on the original dataset, both encoder and combined loss curves are smooth and stable, with training and validation tracking each other closely throughout.

The results on the 60-video dataset mark a clear improvement over the original VICReg experiments. Where previously the encoder validation loss exhibited pronounced oscillations and persistent divergence from the training curve, here both curves converge smoothly and remain close to each other throughout training. This strongly suggests that the instability observed earlier was primarily a consequence of data scarcity: with only fifteen videos and a fixed 80/20 split, the three validation examples provided too weak and too variable a signal to produce smooth validation curves, and the variance-covariance regularisation of VICReg, which depends on batch-level statistics, was operating in too noisy a regime to converge reliably. The larger dataset mitigates both problems simultaneously, providing more stable batch statistics during training and a more representative validation partition. These results are encouraging and suggest that VICReg, given sufficient data, may be a competitive alternative to NT-Xent in this setting, particularly given its independence from batch size, which remains a practical constraint in the clinical data regime.

3.6 Summary of Training Results

Table 3.5 summarises the main training results for the configurations retained in the final comparison, organised by experimental group. For each configuration, the final training loss, the best validation loss, and the stopping epoch are reported. VICReg values are included only as convergence references and are not directly comparable in magnitude to NT-Xent or NT-Xent + Triplet because the

underlying objectives operate on different scales.

Table 3.5: Summary of the main training results for the configurations retained in the final comparison. †VICReg losses are reported only as convergence references and are not directly comparable in magnitude to NT-Xent-based losses. The best result within each group is shown in bold.

Group	Configuration	Train loss	Best val. loss	Epoch
NT-Xent BS 15-video set	GRU, bs=16	**0.1050**	**0.3866**	100
	GRU, bs=128	0.6669	0.4639	200
VICReg 15-video set	GRU	15.3506	14.3629	47
	Transformer	15.0188	14.4397	65
N+T augmented sets	$\lambda_T = 0.1$, Aug(12,12)	0.6287	0.8765	114
	$\lambda_T = 0.5$, **Aug(12,12)**	**0.4255**	**0.9094**	**126**
	$\lambda_T = 0.5$**, Aug(30,30)**	**0.9325**	**0.2134**	**177**
	VICReg†, Aug(30,30)	15.0003	11.7750	111

3.7 Latent Space Separability Analysis

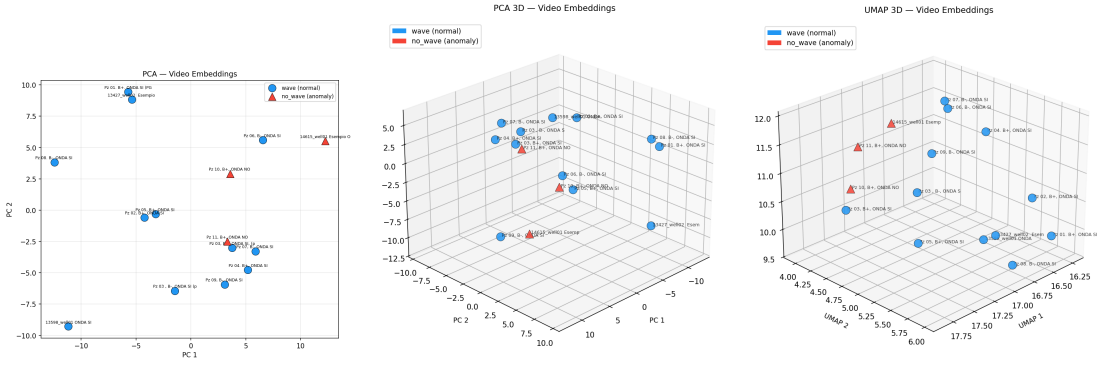
Beyond the quantitative evaluation of training and validation losses, a fundamental question concerns whether the learned latent representations encode semantically meaningful structure. Specifically, one wants to know whether embeddings of videos belonging to the same class tend to occupy similar regions of the latent space, and whether the two classes (wave and no_wave) are geometrically separable. To investigate this, the high-dimensional embeddings produced by each trained model were projected into two- and three-dimensional spaces using multiple dimensionality reduction techniques: Principal Component Analysis (PCA), t-Distributed Stochastic Neighbour Embedding (t-SNE), Uniform Manifold Approximation and Projection (UMAP), and Isomap. Each technique captures different geometric properties of the embedding manifold: PCA preserves global linear variance, t-SNE and UMAP emphasise local neighbourhood structure, and Isomap approximates geodesic distances on the data manifold,

providing complementary views of the latent space geometry.

It is important to note that the analysis operates on a total of 15 videos, each represented by a single aggregated embedding. This imposes an intrinsic limit on the statistical confidence of any observed structure; nonetheless, the qualitative patterns described below are consistent across multiple reduction methods and provide meaningful diagnostic insight into what each model configuration has learned.

3.7.1 NT-Xent with GRU and Batch Size 16

The first configuration analysed is the GRU encoder trained with NT-Xent at batch size 16, which achieved the lowest absolute training loss across all experiments. Despite this numerical advantage, the latent space visualisations reveal limited geometric structure.



((a)) PCA 2D. ((b)) PCA 3D. ((c)) UMAP 3D.

Figure 3.15: Latent space projections for NT-Xent GRU trained at batch size 16. Blue circles denote wave videos; red triangles denote no_wave videos. No clear class-discriminative clustering is apparent in any of the three projections.

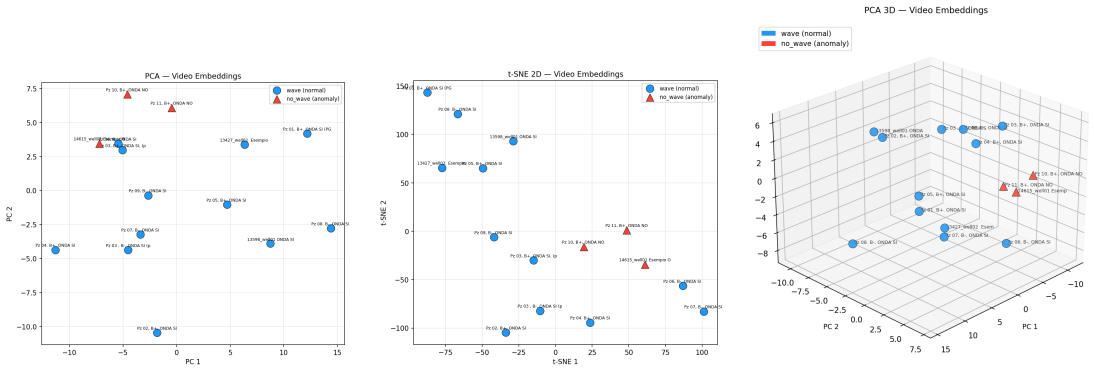
As shown in Figure 3.15, the embeddings do not exhibit clear class-discriminative organisation under any of the three projection methods. In the PCA 2D plot, the no_wave samples (red triangles) are scattered across the space and interleave with wave samples, with no discernible linear boundary between the two classes. The PCA 3D projection confirms this picture: the three no_wave points are distributed across different regions of the volume alongside the wave embeddings, with no coherent grouping. The UMAP 3D projection,

which is more sensitive to local neighbourhood structure, similarly shows the no_wave samples dispersed at different locations without forming a distinct cluster, although a loose spatial tendency can be observed in the left portion of the manifold.

This result is consistent with the known limitation of contrastive learning at small batch sizes: with only 16 samples per batch, the NT-Xent objective is exposed to very few negative pairs at each step, producing a weak discriminative signal that is insufficient to impose a strongly structured geometry on the latent space, even when the training loss is numerically low.

3.7.2 NT-Xent with GRU and Batch Size 128

Increasing the batch size to 128 substantially augments the number of negative pairs available at each optimisation step, providing a richer contrastive signal regardless of the higher absolute loss values. Furthermore, as established in Section 3.7, this configuration does not exhibit overfitting, lending additional reliability to the learned representations.



((a)) PCA 2D. ((b)) t-SNE 2D. ((c)) PCA 3D.

Figure 3.16: Latent space projections for NT-Xent GRU trained at batch size 128. An emergent class-discriminative structure is visible, particularly in the PCA 3D projection where the three no_wave embeddings form a compact and spatially distinct cluster.

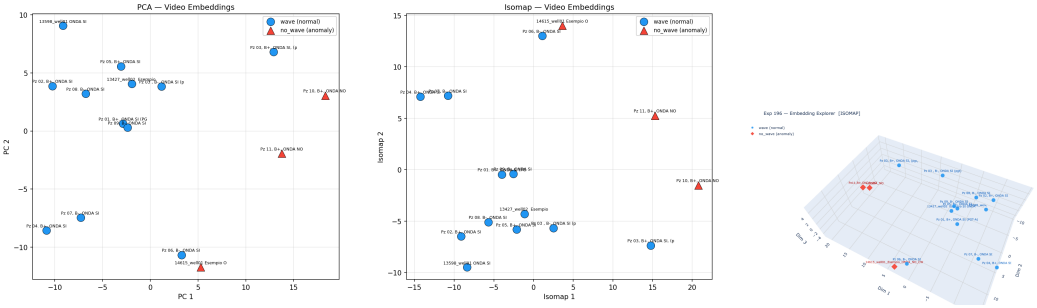
The visualisations in Figure 3.16 reveal a markedly different picture compared to batch size 16. In the PCA 2D projection, the no_wave samples begin to separate from the wave samples: two of the three no_wave embeddings (Pz 10

and Pz 11) cluster together in the upper portion of the PC1–PC2 plane, while the third (14615_well01) is displaced towards the right, co-located near a wave sample at the boundary of the distribution. The t-SNE 2D projection further highlights this tendency, placing the three no_wave points in a spatially proximate region near the centre-right of the manifold, clearly separated from the majority of wave embeddings which occupy the upper-left and lower quadrants.

The most compelling evidence of emergent structure appears in the PCA 3D projection, where three distinct groupings are identifiable: a compact cluster of no_wave embeddings (Pz 10, Pz 11, and 14615_well01) occupying the positive PC1 / mid-range PC3 region, and two broader wave clusters distributed across the remaining volume. This tri-clustered organisation, comprising two wave groups and one no_wave group, suggests that the encoder, exposed to a richer contrastive signal at batch size 128, has begun to capture class-relevant spatiotemporal structure in its latent representations, even in the absence of any explicit supervision.

3.7.3 VICReg with GRU

The final configuration evaluated is the GRU encoder trained with VICReg under the recommended coefficients. As discussed in Section 1.3.5, VICReg circumvents the batch-size dependency of contrastive objectives by directly regularising the embedding distribution, making it theoretically better suited to the computational constraints of this study.



((a)) PCA 2D. ((b)) Isomap 2D. ((c)) Isomap 3D.

Figure 3.17: Latent space projections for VICReg GRU with ($\lambda = 25, \mu = 25, \nu = 1$). The class separation is more pronounced than in either NT-Xent configuration, with no_wave embeddings forming a spatially cohesive group clearly distinct from the wave cluster. In the Isomap 3D projection, a single wave video (Pz 06) appears proximate to the no_wave region.

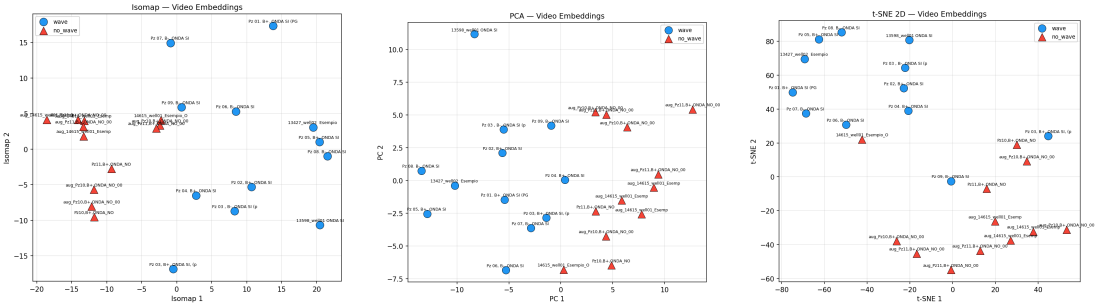
The VICReg embeddings display the strongest class-discriminative structure among all configurations tested. In the PCA 2D projection, the two no_wave samples (Pz 10 and Pz 11) are clearly displaced towards the positive PC1 axis, forming a well-separated group from the wave embeddings, which occupy the negative-to-central PC1 region. The third no_wave sample (14615_well01) is positioned in the lower portion of the plot, adjacent to a single wave video (Pz 06), representing the only ambiguous point in this projection.

The Isomap 2D projection reinforces this picture from the perspective of geodesic manifold structure: the no_wave samples form two spatially separated groups, with Pz 10 isolated at the far right and Pz 11 together with 14615_well01 in the upper-right quadrant, while the wave embeddings occupy a broad but coherent region on the left side of the manifold. The Isomap 3D projection provides the clearest separation of all: the no_wave embeddings (red diamonds) are tightly grouped in a distinct corner of the three-dimensional manifold, while virtually all wave videos (blue circles) occupy the opposite region. The sole exception is Pz 06, a wave video whose embedding lies at the boundary of the no_wave cluster, suggesting that its cytoplasmic dynamics share some spatiotemporal characteristics with the anomalous class, a finding that warrants further clinical investigation.

3.8 Latent Space Separability on Augmented Datasets

The latent space analyses presented in Section 3.7 were conducted on the original fifteen videos. Extending the evaluation to the augmented datasets allows a more direct assessment of how the increased number of training examples affects the geometric structure of the learned representations, independently of the training dynamics already discussed.

3.8.1 NT-Xent + Triplet, $\lambda_{\text{triplet}} = 0.1$, 24-video dataset



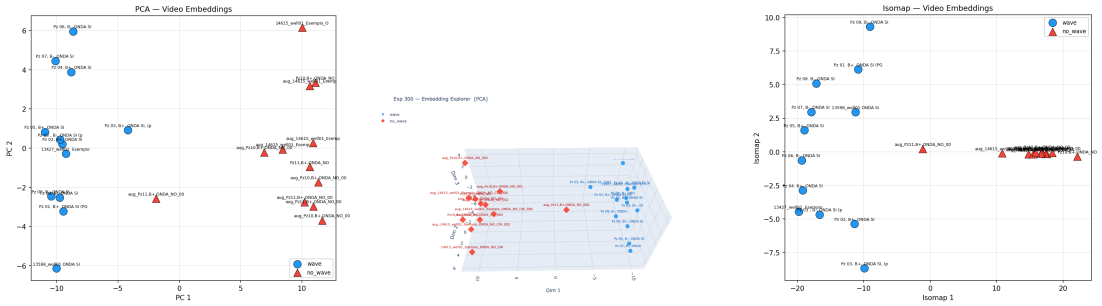
((a)) Isomap 2D. ((b)) PCA 2D. ((c)) t-SNE 2D.

Figure 3.18: Latent space projections for NT-Xent + Triplet ($\lambda_{\text{triplet}} = 0.1$) trained on the 24-video dataset. Blue circles denote wave videos; red triangles denote no-wave videos, including their augmented copies. A degree of class separation is already visible across all three projections, with the no-wave embeddings beginning to form a coherent group distinct from the wave cluster.

Even with a mild triplet weight, the 24-video dataset produces noticeably more structured latent representations than those observed on the original fifteen videos. The Isomap 2D projection is the most informative here: the no-wave embeddings, comprising both the three original no-wave videos and their augmented copies, form a discrete cluster on the left side of the manifold, while the wave embeddings occupy a broadly separate region. The PCA 2D and t-SNE 2D projections confirm this tendency, showing the two classes distributed in different areas of the projected space with limited overlap. It is worth noting that the wave class was not augmented in this configuration, as only the no-wave examples were expanded to reach parity, so the increased separability is attributable both to the

joint NT-Xent + Triplet objective and to the more representative sampling of the no-wave class provided by the augmented copies.

3.8.2 NT-Xent + Triplet, $\lambda_{\text{triplet}} = 0.5$, 24-video dataset

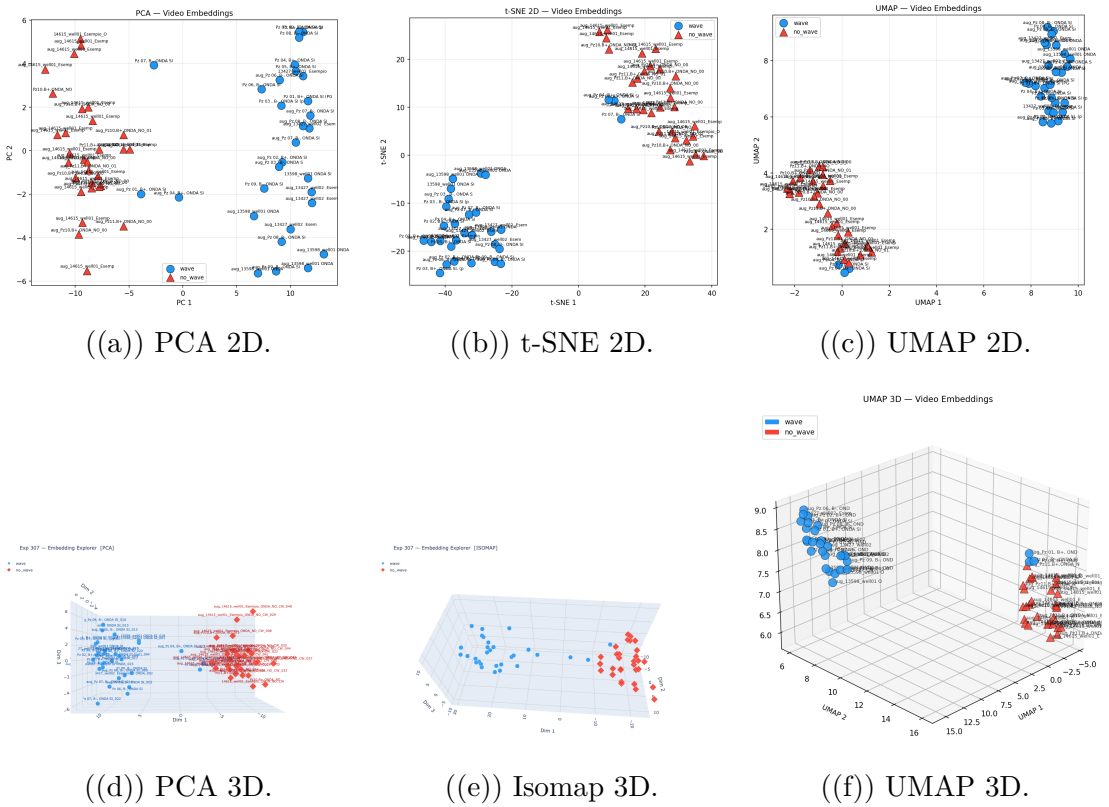


((a)) PCA 2D. ((b)) PCA 3D. ((c)) Isomap 2D.

Figure 3.19: Latent space projections for NT-Xent + Triplet ($\lambda_{\text{triplet}} = 0.5$) trained on the 24-video dataset. Increasing the triplet weight produces more compact and better separated clusters. The PCA 3D projection in particular reveals two well-defined groups with clear spatial separation between the wave and no-wave classes.

Raising the triplet weight to $\lambda_{\text{triplet}} = 0.5$ has a clear and consistent effect on the latent space geometry. In the PCA 2D projection, the no-wave embeddings have migrated to the right portion of the PC1–PC2 plane and are no longer interleaved with wave samples. The Isomap 2D projection reinforces this picture, with the two classes occupying largely disjoint regions of the geodesic manifold. The most compelling result is the PCA 3D projection: two compact and spatially separated clusters are clearly identifiable, one for each class, with very limited boundary overlap. This is a meaningful improvement over the $\lambda_{\text{triplet}} = 0.1$ configuration and confirms that the explicit margin-based constraint of the triplet term is playing a substantive role in shaping the embedding geometry, beyond what the NT-Xent objective alone would produce.

3.8.3 NT-Xent + Triplet, $\lambda_{\text{triplet}} = 0.5$, 60-video dataset



((a)) PCA 2D. ((b)) t-SNE 2D. ((c)) UMAP 2D.

((d)) PCA 3D. ((e)) Isomap 3D. ((f)) UMAP 3D.

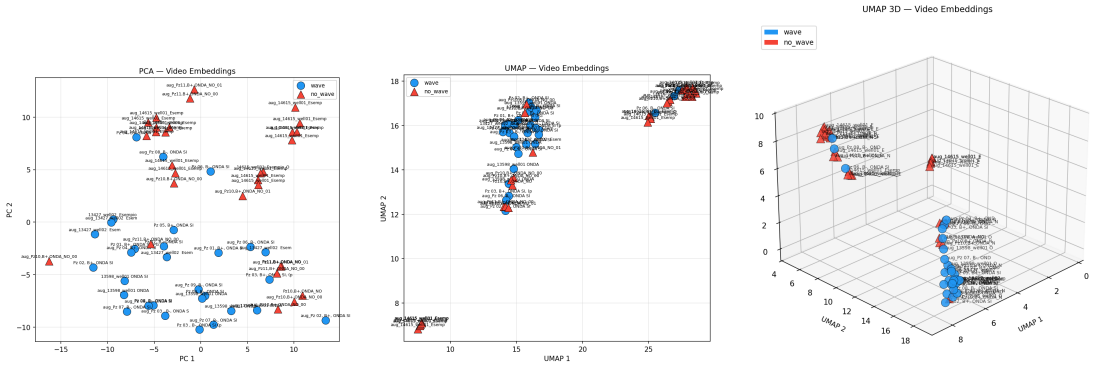
Figure 3.20: Latent space projections for NT-Xent + Triplet ($\lambda_{\text{triplet}} = 0.5$) trained on the 60-video dataset. Top row: 2D projections. Bottom row: 3D projections. The class separation improves markedly compared to the 24-video configurations, with t-SNE and UMAP producing compact, well-defined clusters in both two and three dimensions. A small number of boundary cases remain, consistent with the inherent difficulty of the task.

Expanding the dataset to 60 videos produces the most structured latent space observed across all configurations. The PCA 2D projection already shows a clean separation along the first principal component, with the wave and no-wave embeddings occupying opposite halves of the PC1 axis and only moderate scatter along PC2. The improvement becomes considerably more pronounced with t-SNE and UMAP, both of which construct tight, cohesive clusters for each class with a clearly defined gap between them. The wave cluster in the UMAP 2D projection is particularly compact, suggesting that the additional training examples have allowed the encoder to learn a more consistent representation of normal

cytoplasmic dynamics across the biological variability present in the dataset.

The three-dimensional projections confirm and extend this picture. PCA 3D and Isomap 3D both show well-separated class regions, and the UMAP 3D projection is the sharpest of all: the two classes occupy completely distinct portions of the three-dimensional manifold, with the no-wave cluster concentrated in the upper-left region and the wave cluster forming a dense, elongated group in the lower-right. A small number of borderline cases appear near the class boundary across all projections, which is consistent with the inherent variability of biological data and the approximate nature of the augmentation process.

3.8.4 VICReg, 60-video dataset



((a)) PCA 2D. ((b)) UMAP 2D. ((c)) UMAP 3D.

Figure 3.21: Latent space projections for VICReg trained on the 60-video dataset. Class structure emerges without any form of label or negative pair supervision. The PCA 2D projection shows partial separation with some overlap at the boundary; UMAP in both 2D and 3D reveals three distinct groupings, two wave subclusters and one no-wave cluster, with a small number of misplaced examples.

The VICReg results on the 60-video dataset are noteworthy precisely because the objective uses no negative pairs and no class labels of any kind: the observed structure emerges entirely from the variance, invariance, and covariance regularisation terms acting on the batch embedding distribution. The PCA 2D projection shows a partial but visible separation, with the no-wave embeddings gravitating towards the positive PC1 region while the wave embeddings spread across the remainder of the space; some boundary overlap is present, making the separation less crisp than in the NT-Xent + Triplet configurations.

The UMAP projections are considerably more revealing. In the 2D case, three spatially distinct groupings are clearly identifiable: two sub-clusters of wave embeddings and one compact no-wave cluster, separated by empty regions of the manifold. The UMAP 3D projection confirms this tri-clustered organisation, with the no-wave group occupying a well-defined corner of the three-dimensional space while the wave embeddings form two adjacent but distinct concentrations. A small number of examples appear in the wrong cluster region, reflecting the higher boundary ambiguity of VICReg compared to the margin-enforcing triplet configurations. This result is nonetheless encouraging: it demonstrates that distributional regularisation alone, given a sufficiently large and balanced dataset, can produce a latent space with meaningful class structure, a finding with practical implications for settings where even weak class labels are unavailable.

3.9 Clustering Evaluation

Beyond the qualitative assessment of latent space geometry provided by the dimensionality reduction visualisations, a quantitative evaluation of class separability was conducted by fitting unsupervised clustering algorithms directly in the embedding space. The objective is to measure how well the learned representations support the automatic recovery of the two ground-truth classes, wave and no-wave, without any label information at inference time.

Three clustering algorithms were evaluated: k-Means, Agglomerative Clustering, and Gaussian Mixture Models (GMM). For each configuration the number of components k and, in the case of GMM, the covariance structure (*full*, *diag*, or *spherical*) were selected by grid search over a small set of candidates, retaining the combination that maximised the Adjusted Rand Index on the embedding set. Across the large majority of configurations, GMM consistently outperformed the other algorithms; k-Means achieved competitive results only in one case (NT-Xent with batch size 128), likely because the corresponding embedding distribution is approximately spherical and does not require the additional flexibility of a full covariance model.

Three complementary metrics were used to evaluate the quality of each clustering solution. The **Adjusted Rand Index** (ARI) measures the agreement

between the predicted cluster assignments and the ground-truth class labels, corrected for chance, with a score of 1 indicating perfect recovery. The **Normalised Mutual Information** (NMI) quantifies the mutual information between the two partitions normalised to the $[0, 1]$ interval, providing a symmetric measure of cluster purity. The **Silhouette score** assesses the geometric compactness and separation of the clusters directly in embedding space, independently of the ground-truth labels, with higher values indicating denser and more separated groups.

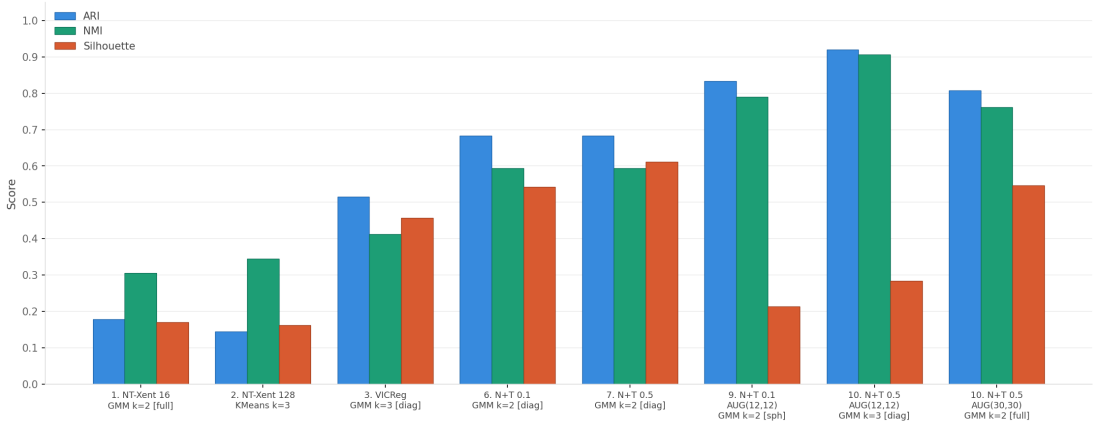


Figure 3.22: ARI, NMI, and Silhouette scores for all evaluated configurations. Results are ordered by increasing dataset size and objective complexity. GMM was selected in all cases except NT-Xent with batch size 128, where k-Means achieved the best ARI.

The results follow a clear progression. The two NT-Xent baseline configurations (batch sizes 16 and 128, on the original 15 videos) achieve the lowest scores across all three metrics, with ARI values below 0.2 and Silhouette scores around 0.17. This is consistent with the latent space visualisations: when the contrastive signal is limited by small batch sizes and few training examples, the resulting embeddings do not carry enough discriminative structure for an unsupervised clustering algorithm to recover the class partition reliably. VICReg on the original dataset already shows a substantial improvement, with ARI rising to 0.52 and Silhouette to 0.46, confirming that the distributional regularisation it enforces produces more compact and geometrically structured representations even without negative pairs.

The joint NT-Xent + Triplet objective on the original 15 videos (configurations 6 and 7) achieves ARI and NMI values of approximately 0.68 and 0.59 respectively for both triplet weights, with the higher weight ($\lambda_{\text{triplet}} = 0.5$) producing a notably better Silhouette score (0.61 vs. 0.54), suggesting that stronger margin enforcement yields more geometrically compact clusters even when the assignment quality remains comparable.

The most significant improvements come from the augmented datasets. NT-Xent + Triplet with $\lambda_{\text{triplet}} = 0.1$ on the balanced 24-video set already pushes ARI to 0.83 and NMI to 0.79, a jump of roughly fifteen percentage points over the non-augmented equivalent. Raising the triplet weight to 0.5 on the same 24-video set brings ARI to 0.92 and NMI to 0.91, the highest values recorded across the entire evaluation, confirming that the combination of data augmentation and a stronger margin-based objective is particularly effective at shaping a linearly separable embedding space. The Silhouette score for these two augmented configurations is comparatively lower (0.21 and 0.28), which reflects the fact that the augmented copies of the same video are geometrically close to each other in embedding space, reducing the apparent inter-cluster distance as measured by the Silhouette coefficient. This is an artefact of the augmentation strategy rather than a genuine indicator of poor separation. The 60-video configuration with $\lambda_{\text{triplet}} = 0.5$ achieves ARI of 0.81 and NMI of 0.76 with a Silhouette of 0.55, a well-balanced result that combines strong assignment quality with good geometric compactness, consistent with the richer and more varied training set.

Table 3.6: Clustering evaluation results for all configurations. ARI: Adjusted Rand Index. NMI: Normalised Mutual Information. Silhouette: geometric cluster compactness and separation. All scores in $[0, 1]$; higher is better.

Configuration	**Algorithm**	**ARI**	**NMI**	**Silhouette**
NT-Xent, bs=16	GMM $k = 2$ [full]	0.179	0.305	0.170
NT-Xent, bs=128	KMeans $k = 3$	0.144	0.345	0.162
VICReg	GMM $k = 3$ [diag]	0.515	0.412	0.457
N+T $\lambda = 0.1$	GMM $k = 2$ [diag]	0.683	0.594	0.542
N+T $\lambda = 0.5$	GMM $k = 2$ [diag]	0.683	0.594	0.612
N+T $\lambda = 0.1$, Aug(12,12)	GMM $k = 2$ [sph]	0.833	0.790	0.213
N+T $\lambda = 0.5$, Aug(12,12)	**GMM $k = 3$ [diag]**	**0.920**	**0.906**	0.284
N+T $\lambda = 0.5$, Aug(30,30)	GMM $k = 2$ [full]	0.807	0.761	0.547

Taken together, these results provide quantitative confirmation of the qualitative trends observed in the latent space visualisations: the NT-Xent + Triplet objective, combined with even modest data augmentation, substantially improves the unsupervised recoverability of the class structure in the embedding space. The best overall configuration by ARI and NMI is NT-Xent + Triplet with $\lambda_{\text{triplet}} = 0.5$ on the 24-video balanced dataset, while the 60-video configuration offers the best trade-off between assignment quality and geometric compactness.

3.10 Explainability Analysis

To move beyond aggregate loss metrics and latent space geometry, and to gain a more direct understanding of what the encoder has learned to attend to, an explainability analysis was conducted using two complementary approaches: **Integrated Gradients** and **Temporal Occlusion**.

3.10.1 Integrated Gradients

The Integrated Gradients method attributes the model's output to each input feature by integrating the gradients of the output with respect to the input along a straight path from a reference baseline to the actual input. The resulting attribution maps highlight the spatial regions that most influence the learned representation, providing a biologically interpretable view of the encoder's internal focus.

NT-Xent with GRU and Batch Size 16

The attribution maps produced for the NT-Xent GRU model trained at batch size 16 show a general concentration of high-attribution regions towards the cellular mass and the morphological boundary of the oocyte, indicating that the encoder is attending to biologically plausible structures. However, residual activations are visible in the background region outside the embryo, manifesting as scattered pixel-level artefacts. This background noise, while not dominant, reflects the limited discriminative precision of the representations learned under a weak contrastive signal, consistent with the sparse latent space structure observed in Section 3.15.

Figure 3.23 illustrates the IG attribution maps for frames 70 through 84 of a representative wave video, covering a temporally continuous window within the pre-cleavage developmental phase.

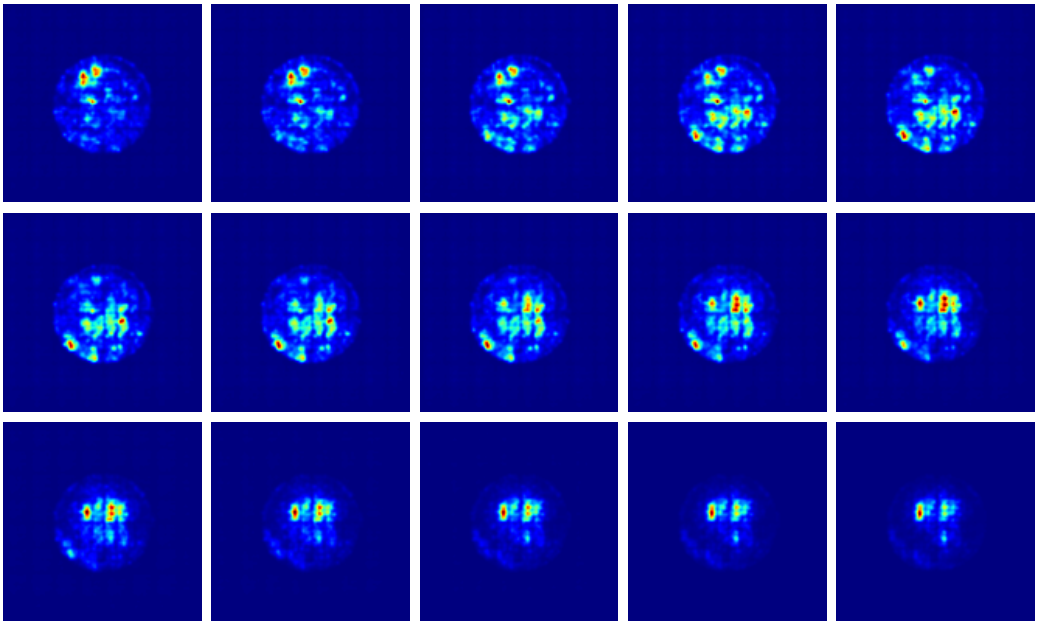


Figure 3.23: Integrated Gradients attribution maps for NT-Xent GRU (batch size 16), frames 70–84 of a representative wave video. High-attribution regions (warm colours) concentrate on the cellular mass and oocyte boundary, with residual background activations visible outside the embryo mask.

NT-Xent with GRU and Batch Size 128

The attribution maps for the NT-Xent GRU model trained at batch size 128 present a markedly cleaner and more focused activation pattern. The high-attribution regions are tightly localised within the oocyte boundary, with virtually no spurious activations in the background. The spatial distribution of warm-coloured attributions traces the internal structure of the cytoplasm with sufficient precision to convey a sense of volumetric depth, as a quasi-three-dimensional rendering of the cellular interior emerges from the overlay of attribution intensities across the frame sequence. This spatial precision reflects the richer discriminative representations learned from the denser negative sampling at batch size 128, and is directly coherent with the stronger class separation observed in the corresponding latent space projections (Section 3.16).

Figure 3.24 reports the attribution maps for the same temporal window (frames 70–84) of the same representative video.

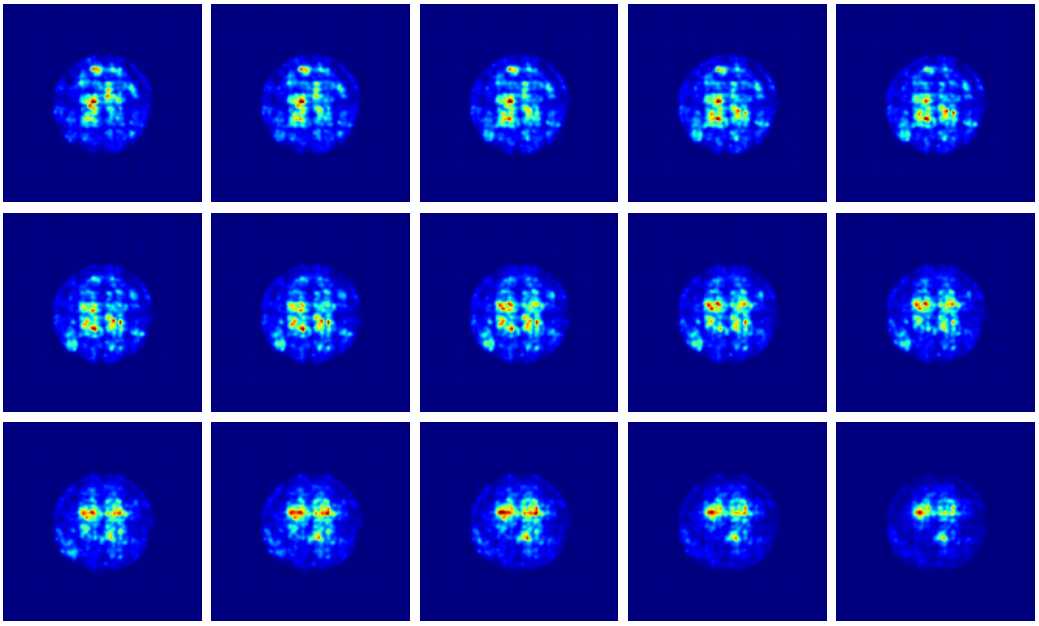


Figure 3.24: Integrated Gradients attribution maps for NT-Xent GRU (batch size 128), frames 70–84. Attribution is tightly confined within the oocyte, with no background noise. The spatial intensity pattern suggests sensitivity to the internal cytoplasmic dynamics rather than to surface morphology alone.

VICReg with GRU

The attribution maps produced by the VICReg-trained GRU encoder show a qualitatively intermediate behaviour. The primary activation focus remains correctly localised on the cytoplasmic wave and the oocyte morphology, demonstrating that VICReg has also guided the encoder towards biologically meaningful features. However, the maps are appreciably noisier than those of the NT-Xent batch size 128 model: residual activations of moderate intensity are visible across a broader spatial extent, including regions proximate to, but outside, the embryo boundary. This increased noise level is consistent with the less stable training dynamics and the higher degree of validation oscillation observed for the VICReg configuration in Section 3.9, and aligns with the expectation that a less precisely converged encoder produces less focused attribution patterns.

Figure 3.25 shows the corresponding attribution maps for frames 70–84.

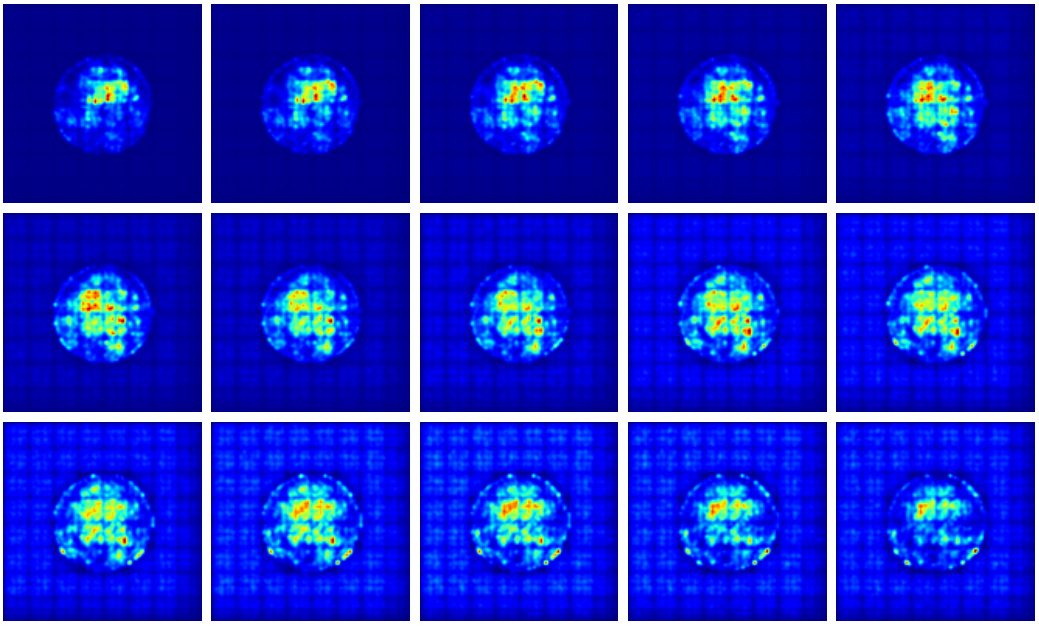


Figure 3.25: Integrated Gradients attribution maps for VICReg GRU ($\lambda = 25, \mu = 25, \nu = 1$), frames 70–84. Attributions are centred on the oocyte and cytoplasmic dynamics, but exhibit greater spatial noise compared to the NT-Xent batch size 128 configuration, consistently with the less stable training dynamics of VICReg.

NT-Xent + Triplet, $\lambda_{\text{triplet}} = 0.5$, 60-video dataset

The attribution maps produced by the NT-Xent + Triplet model trained on the 60-video augmented dataset represent the cleanest result across all configurations evaluated. The high-attribution regions are tightly and consistently confined within the oocyte boundary across all fifteen frames of the temporal window, with no residual activations in the background and no scattered artefacts outside the embryo mask. The warm colour gradients trace the internal cytoplasmic structure with notable spatial coherence from frame to frame, suggesting that the encoder has developed a temporally stable and geometrically precise sensitivity to the dynamics of the wave. The combination of a larger and more balanced training set with the explicit margin enforcement of the triplet term appears to produce representations that are not only more separable in the latent space, as demonstrated by the clustering results in Section 3.9, but also more spatially focused in their attribution behaviour.

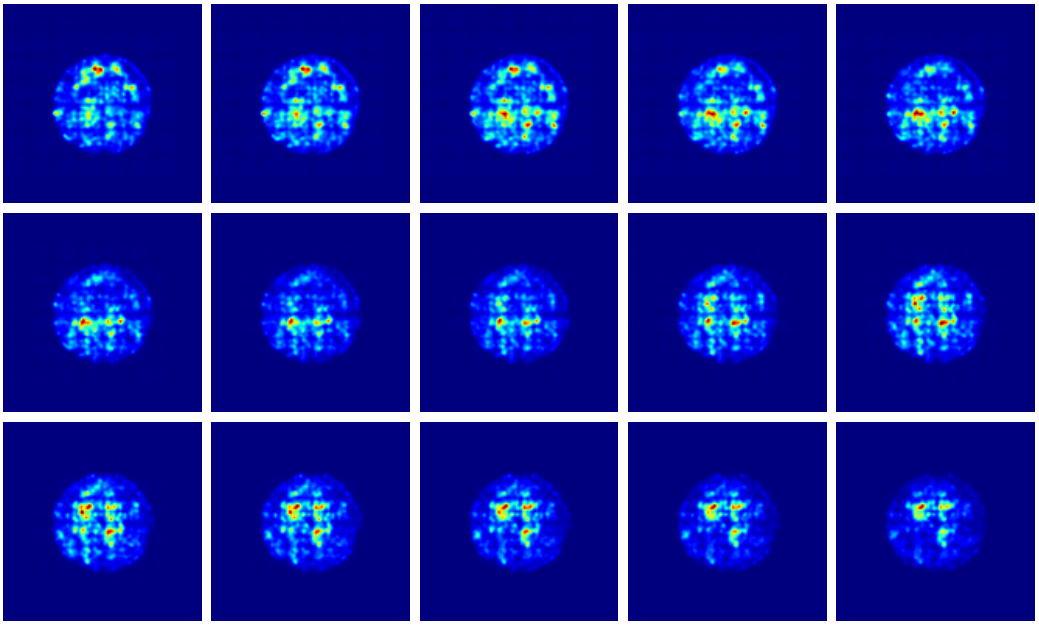


Figure 3.26: Integrated Gradients attribution maps for NT-Xent + Triplet ($\lambda_{\text{triplet}} = 0.5$, 60-video dataset), frames 70–84 of a representative wave video. Attribution is uniformly confined within the oocyte across all frames, with no background noise. The spatial intensity pattern traces the internal cytoplasmic structure with temporal consistency throughout the window.

3.10.2 Temporal Occlusion

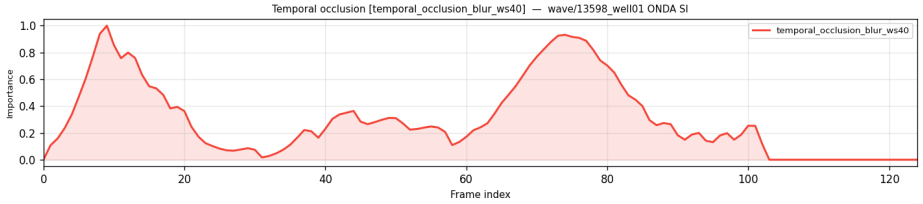
To complement the spatial attribution maps produced by Integrated Gradients, a **Temporal Occlusion** analysis was conducted to identify which temporal segments of the input sequence contribute most to the encoder's learned representation. The method proceeds by systematically occluding a sliding window of consecutive frames, replacing them with either a blurred version of the original content (*blur* strategy) or the temporal mean of the sequence (*mean* strategy), and measuring the resulting change in the encoder's output embedding. Frames whose occlusion induces the largest deviation in the latent representation are assigned the highest importance score. The result is a one-dimensional importance profile over the 125-frame temporal axis, providing a direct temporal analogue to the spatial attribution maps of the previous section.

Two window sizes were explored across configurations: $ws = 33$ and $ws = 40$ frames. Larger windows produce smoother profiles at the cost of temporal

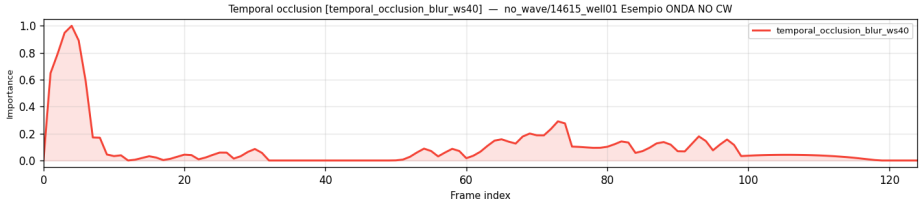
resolution; smaller windows are more sensitive to localised events. Both strategies and window sizes are reported below to assess the robustness of the temporal attribution across models.

NT-Xent with GRU and Batch Size 16

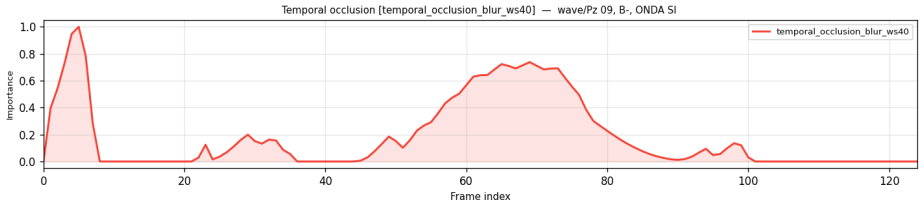
Three representative videos are analysed for this configuration, using the blur occlusion strategy with window size $ws = 40$.



((a)) Wave video (13598_well01, ONDA SI). Two distinct importance peaks are visible: a primary peak around frames 5–15 and a prominent secondary peak spanning frames 65–80, corresponding to the main cytoplasmic wave event.



((b)) No-wave video (14615_well01, ONDA NO). The importance profile is dominated by a single sharp peak at the very beginning of the sequence (frames 1–8), after which it decays rapidly and remains near zero throughout the remainder of the video, reflecting the absence of any temporally localised dynamic event.

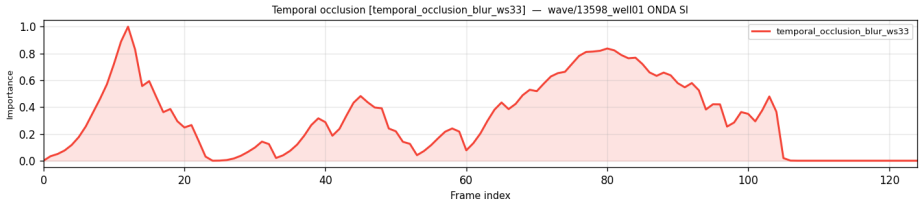


((c)) Wave video (Pz 09, B−, ONDA SI). A clear importance peak is localised in the frame range 60–80, coinciding with the period of maximal cytoplasmic movement in this video, with a smaller secondary activation near the beginning of the sequence.

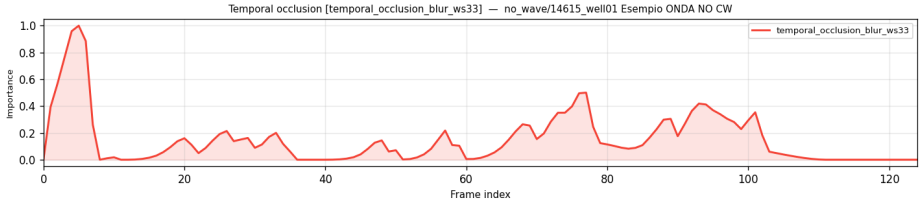
Figure 3.27: Temporal occlusion profiles (blur, $ws = 40$) for NT-Xent GRU at batch size 16. Importance on the y-axis is normalised to $[0, 1]$.

The NT-Xent GRU model at batch size 16 demonstrates a meaningful temporal sensitivity in the wave videos: the highest-importance windows correspond, in both cases, to the temporal intervals where cytoplasmic wave activity is expected to be most pronounced. For the no-wave video, the profile collapses to a near-flat signal after the initial frames, which is consistent with the absence of any biologically distinctive dynamic event beyond the early morphological settling of the oocyte. However, the profiles for the wave videos exhibit a somewhat fragmented structure, with multiple secondary activations distributed across the sequence, suggesting that the model trained with a limited contrastive signal has not fully learned to suppress temporally irrelevant content.

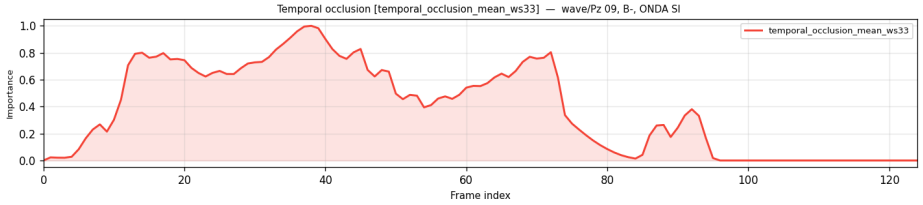
NT-Xent with GRU and Batch Size 128



((a)) Wave video (13598_well01, ONDA SI). The importance profile shows a well-defined primary peak centred around frames 10–20, and a broad, sustained high-importance region spanning frames 65–105, directly corresponding to the cytoplasmic wave propagation phase.



((b)) No-wave video (14615_well01, ONDA NO). The profile is dominated by a sharp peak in the first few frames, followed by a low-amplitude, irregular signal. Compared to batch size 16, the post-peak importance is more uniformly low, indicating a cleaner discrimination of the absence of dynamic events.



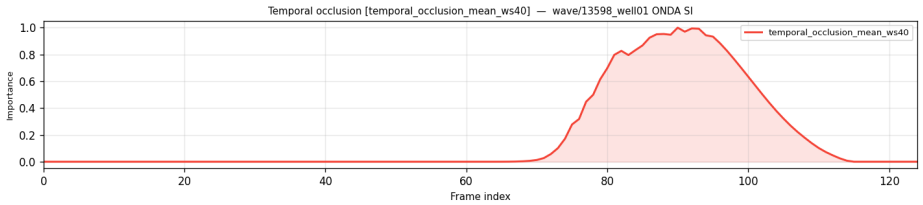
((c)) Wave video (Pz 09, B−, ONDA SI). Using the mean occlusion strategy, the profile reveals sustained high importance throughout the pre-cleavage phase (frames 5–80), with a noticeable dip around frames 50–55 before recovering, and a sharp drop after frame 80 as the wave activity subsides.

Figure 3.28: Temporal occlusion profiles (blur, $ws = 33$ for well01 videos; mean, $ws = 33$ for Pz 09) for NT-Xent GRU at batch size 128.

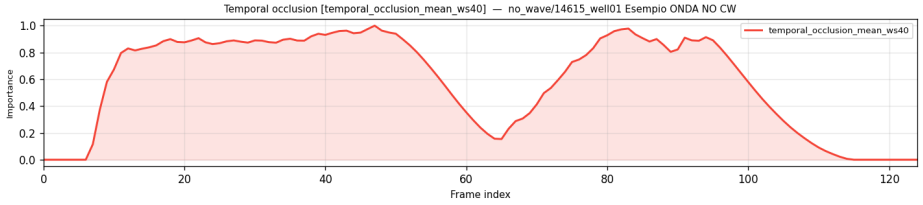
The temporal occlusion profiles for the batch size 128 model are noticeably more structured and temporally coherent than those at batch size 16. For the wave videos, the high-importance regions are broader and more continuously elevated across the frames where wave activity is expected, with sharper transitions at the onset and conclusion of the dynamic event. The no-wave profile exhibits a cleaner

baseline, with the post-initial-peak signal remaining at very low values across virtually the entire sequence. This improved temporal precision is consistent with the richer negative sampling at batch size 128, which encourages the encoder to build representations that are more selectively sensitive to the dynamic content that discriminates between clips.

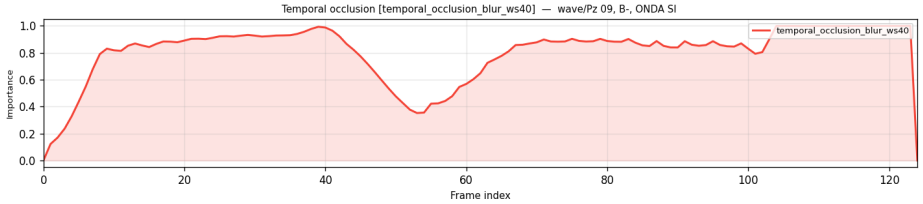
VICReg with GRU



((a)) Wave video (13598_well01, ONDA SI). The profile is highly focused: importance remains near zero until approximately frame 70, then rises sharply to a sustained plateau centred around frames 80–100, before dropping back to zero. This corresponds precisely to the wave event window and represents the most temporally localised attribution among all configurations tested.



((b)) No-wave video (14615_well01, ONDA NO). Unlike the near-flat profile obtained under NT-Xent, this model assigns high importance across a broad range of frames, approximately frames 10–60 and again 75–100, without any clear temporal focus. This diffuse activation pattern is difficult to interpret biologically and may reflect the less stable convergence of VICReg on this dataset.



((c)) Wave video (Pz 09, B−, ONDA SI). The importance profile is nearly uniformly high across the entire sequence (frames 5–105), with only a moderate dip around frames 50–55. While this indicates that the model is sensitive to the video content, it lacks the temporal selectivity that would make the attribution diagnostically useful.

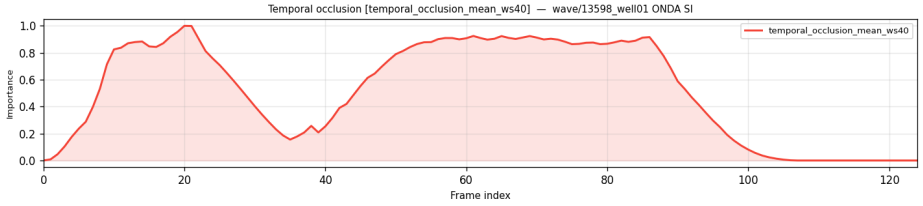
Figure 3.29: Temporal occlusion profiles (mean, $ws = 40$ for well01 videos; blur, $ws = 40$ for Pz 09) for VICReg GRU ($\lambda = 25, \mu = 25, \nu = 1$).

The VICReg temporal occlusion results present a mixed picture. On one hand, the wave profile for 13598_well01 is the most temporally precise of all configurations: the importance is essentially zero outside the wave event window

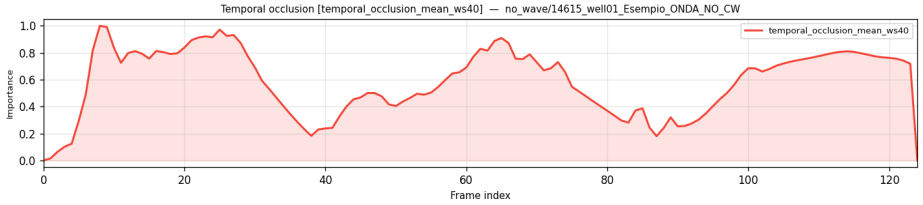
and peaks sharply within it, suggesting that in this specific case VICReg has learned a highly focused temporal sensitivity. On the other hand, the remaining two profiles, the no-wave video and Pz 09, display a diffuse, broad activation that occupies most of the temporal axis without clear localisation. This inconsistency can be understood in light of the training dynamics discussed in Section 3.4: VICReg's less stable convergence and higher validation oscillation may result in representations that are sensitive to global video-level statistics rather than to the specific temporal intervals where biologically relevant dynamics occur, producing temporally non-selective occlusion profiles for videos where the wave event is less prominent or absent.

NT-Xent + Triplet, $\lambda_{\text{triplet}} = 0.5$, 60-video dataset

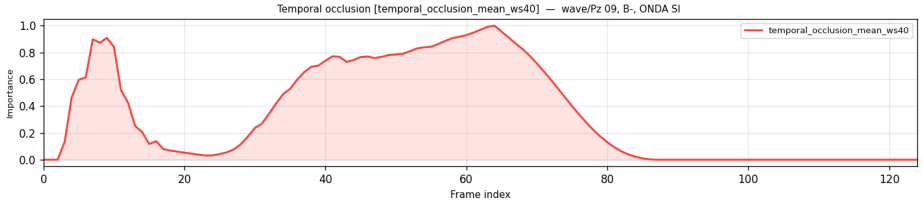
The temporal occlusion analysis on the same model confirms and extends the picture suggested by the Integrated Gradients maps.



((a)) Wave video (13598_well01, ONDA SI). The importance profile rises steeply in the early frames, reaches its peak during the wave propagation window (approximately frames 10–90), and drops sharply to near zero thereafter, cleanly delineating the onset and conclusion of the cytoplasmic event.



((b)) No-wave video (14615_well01, ONDA NO). In the absence of a cytoplasmic wave, the importance is distributed broadly and irregularly across the entire temporal axis, with no concentrated peak in the interval where wave activity would be expected. This diffuse profile is the expected signature of a video lacking the specific dynamic pattern the encoder has learned to respond to.



((c)) Wave video (Pz 09, B−, ONDA SI). An early importance peak is visible in the first fifteen frames, followed by a broad sustained region of high importance covering approximately frames 35–85, which corresponds to the cytoplasmic wave activity in this video. The profile drops cleanly to zero after frame 90, confirming that the encoder is not responding to post-wave content.

Figure 3.30: Temporal occlusion profiles (mean, $ws = 40$) for NT-Xent + Triplet ($\lambda_{\text{triplet}} = 0.5$, 60-video dataset). The wave videos show focused importance in the biologically relevant temporal window followed by a clean return to zero. The no-wave profile is broadly distributed, consistent with the absence of a localised dynamic event.

The three profiles collectively provide the most coherent temporal attribution observed across all configurations. For both wave videos, the importance rises precisely when the cytoplasmic wave is expected to occur and returns to zero

once it concludes, a behaviour that closely mirrors what an expert embryologist would attend to when reviewing the same footage. The contrast with the no-wave profile is equally informative: the absence of any concentrated peak, replaced by a broadly distributed and irregular importance signal, is consistent with a video that lacks the specific spatiotemporal pattern the encoder has been trained to distinguish. Taken together with the Integrated Gradients results, these findings suggest that the NT-Xent + Triplet objective on the 60-video dataset produces representations that are not only geometrically well-structured in the latent space but also temporally and spatially grounded in the biological event of interest, a property that is essential for clinical interpretability.

Chapter 4

Discussion and Conclusions

This chapter brings together the findings of the thesis and situates them in a broader context. The discussion revisits the key experimental results, including the comparison between NT-Xent and VICReg, the role of optical flow and data augmentation, the effect of the joint NT-Xent + Triplet objective, and the evidence provided by the explainability analyses, and reflects on what they collectively say about the viability of self-supervised contrastive learning for cytoplasmic wave detection. The conclusions summarise the main contributions and their clinical implications. The chapter closes with a set of concrete directions for future work, covering dataset expansion, generative modelling in the latent space, and multi-task extensions of the architecture.

4.1 Discussion

The results presented in this thesis offer several points of reflection, both on the methodological choices made and on what the experimental evidence actually supports.

The comparison between NT-Xent and VICReg turned out to be more nuanced than a simple ranking of final loss values would suggest. NT-Xent with batch size 128 proved to be the most balanced configuration overall: training dynamics were stable, no overfitting was observed in the validation curves, and the latent space projections showed a coherent grouping of wave and no-wave videos that became progressively more pronounced as the batch size grew. This last point is not incidental. In contrastive learning, the batch size is not merely a

computational parameter: it determines how many negative examples the model sees at each step, and therefore how rich the discriminative signal is. With only fifteen videos available, squeezing meaningful structure out of such a small dataset requires precisely this kind of dense negative sampling. The fact that the model generalises without overfitting under these conditions is an encouraging result.

VICReg presents a different picture. Its independence from negative pairs is theoretically attractive in low-data regimes, and the latent space visualisations, particularly the Isomap 3D projections, show arguably the sharpest class separation of any configuration tested. At the same time, the training dynamics are noticeably less stable, the validation encoder loss oscillates throughout training, and the temporal occlusion profiles for some videos lack the focused localisation seen with NT-Xent. This inconsistency suggests that VICReg, in its standard parameterisation, does not converge as reliably as NT-Xent on a dataset of this size, even though when it does converge it produces well-structured representations. The two objectives are therefore not straightforwardly comparable: NT-Xent offers more predictable training behaviour and cleaner explainability maps, while VICReg occasionally achieves stronger geometric separation but at the cost of stability.

One aspect that consistently emerges across all configurations is the role of optical flow. The ablation on input modality showed unambiguously that multimodal fusion, combining preprocessed grayscale frames with the horizontal and vertical flow fields, leads to substantially faster convergence and a lower final contrastive loss than either modality alone. This is interpretable: the raw frames capture the morphological appearance of the oocyte, which changes slowly and subtly, while the optical flow fields make cytoplasmic motion explicit in a form that the encoder can exploit directly. Providing both gives the model a cleaner description of the phenomenon it is asked to characterise.

The augmented dataset experiments introduced an important additional dimension to the analysis. Extending the training set through biologically plausible geometric transformations proved to be one of the most effective levers available in this low-data regime. Even a modest expansion to 24 videos, obtained by bringing the no-wave class to parity through augmentation, produced noticeably more structured latent representations and improved clustering performance. Expand-

ing further to 60 videos yielded the most stable training dynamics observed across the entire study, eliminating the encoder overfitting signal that had characterised the smaller configurations and producing latent space projections in which the two classes occupy clearly distinct and compact regions of the manifold.

The joint NT-Xent + Triplet objective played a complementary role in this improvement. Adding a margin-based triplet term on top of the contrastive loss had a measurable effect on latent space geometry: raising the triplet weight from 0.1 to 0.5 produced more compact and better separated clusters, as confirmed both by the dimensionality reduction visualisations and by the quantitative clustering evaluation. The best overall result by Adjusted Rand Index and Normalised Mutual Information was achieved by the NT-Xent + Triplet configuration with $\lambda_{\text{triplet}} = 0.5$ on the 24-video balanced dataset, reaching ARI of 0.92 and NMI of 0.91. The 60-video configuration offered a complementary advantage: slightly lower assignment scores but a substantially better Silhouette coefficient (0.55), indicating a healthier trade-off between cluster compactness and generalisation across a more varied set of examples. Across all augmented configurations, GMM consistently outperformed the other clustering algorithms, a result that reflects the non-spherical and multi-modal structure of the embedding distributions learned by these models.

The fact that VICReg also stabilised considerably on the 60-video dataset is worth noting. The oscillatory validation behaviour that had characterised its training on the original fifteen videos largely disappeared when more data was available, and the resulting latent space showed meaningful class structure even without any negative pairs or class labels. This suggests that VICReg's instability on the original dataset was primarily a data scarcity problem rather than a fundamental limitation of the objective, and that it may become a competitive alternative to NT-Xent as dataset size grows.

The explainability analysis confirmed and extended this picture. The spatial attribution maps produced by the modified Integrated Gradients show that the encoder consistently directs its attention to the interior of the oocyte, essentially ignoring the background. This is a direct validation of the preprocessing pipeline: the segmentation, spatial stabilisation, and visual enhancement steps have succeeded in presenting the model with clean, centred, artefact-free input.

What is particularly striking in the maps for the NT-Xent batch size 128 configuration is that the warm-coloured attributions do not merely highlight the cell boundary but trace what appears to be the internal volumetric structure of the cytoplasm. This suggests that the temporal component of the architecture, the GRU operating on the sequence of CNN features, is genuinely capturing the dynamics of cytoplasmic movement rather than static morphological cues. The handcrafted optical flow representation, fed as an explicit input channel, appears to have guided this process by providing a frame-by-frame description of motion that the encoder could anchor its temporal reasoning to. The NT-Xent + Triplet model trained on the 60-video dataset produced the cleanest attribution maps of all: high-attribution regions uniformly confined within the oocyte, with no background noise and a spatially coherent warm gradient tracing the cytoplasmic structure across consecutive frames.

The temporal occlusion profiles are broadly consistent with this interpretation. For wave videos, the highest-importance windows correspond to the temporal intervals where cytoplasmic activity is expected to peak, and the no-wave profiles remain comparatively flat throughout. The profiles for the 60-video NT-Xent + Triplet model were the most temporally precise observed: importance rose sharply at the onset of the wave event and returned cleanly to zero once it concluded, with a clear contrast against the diffuse and irregular profile of the no-wave video. There is, however, a recurring feature worth noting: an early importance peak appearing in roughly the first ten frames across several videos and configurations. This spike does not correspond to any obvious wave event visible to the human eye. One possibility is that the model has identified a subtle dynamic characteristic of the oocyte in the immediate post-fertilisation phase, a transient structural feature that precedes the wave itself and is not accessible to standard visual inspection. Whether this represents a genuine biological signal or a model-specific artefact is an open question that warrants further investigation with a larger dataset and domain expert input.

4.2 Conclusions

This thesis has explored the extent to which self-supervised contrastive learning can be applied to the detection of cytoplasmic waves in IVF time-lapse videos, under the realistic constraint of having very few annotated examples available. The results suggest that the answer is cautiously positive. A GRU encoder trained with NT-Xent at batch size 128, combined with multimodal optical flow input, learns representations that are geometrically structured, generalisable, and interpretable, all without relying on a single manual label. Extending the framework with a triplet regularisation term and a biologically grounded augmentation strategy further strengthened these properties: the best configurations achieved ARI values above 0.90 on unsupervised clustering tasks, and the corresponding explainability maps showed spatial and temporal attribution patterns that are directly aligned with the biological event of interest.

More broadly, the self-supervised paradigm appears well suited to the challenges of the IVF domain and, by extension, to other problems in clinical embryology where data is scarce and annotation is expensive. The framework developed here is not specific to cytoplasmic waves: with appropriate preprocessing and input representations, the same architecture could in principle be directed towards other morphokinetic events such as pronuclear fading, cleavage timing, and blastulation dynamics, which are equally difficult to characterise systematically at scale. The key insight is that self-supervised learning shifts the bottleneck from annotation to data collection, which, while still challenging in the clinical setting, is a more tractable problem to solve over time.

4.3 Future Work

The most immediate limitation of this study is the dataset size. Fifteen videos is sufficient to demonstrate proof of concept and to draw meaningful qualitative conclusions, but it is far too small to draw statistically robust ones. Expanding the dataset through collaboration with multiple IVF clinics is therefore the most important next step, and also the one that would unlock the most significant improvements in model performance.

A larger dataset would open several directions that are currently out of reach. First, the latent space geometry, which already shows encouraging class separation, could be refined considerably, either by training with larger batch sizes under NT-Xent to provide a denser and more informative negative sampling, or by exploring supervised contrastive fine-tuning once sufficient annotated examples are available. Second, once a well-structured latent space has been established on a larger collection, it becomes natural to consider generative models operating in that space. A latent diffusion model or a conditional variational autoencoder trained on the learned representations could be used to synthesise realistic embryo clips with controlled wave characteristics, simultaneously augmenting the training data and providing a tool for biological hypothesis testing. Third, the temporal occlusion analysis hinted at the possible existence of early-phase dynamic features that precede the visible wave and are not currently understood. A targeted study of this phenomenon, combining computational attribution with expert embryological review, could be scientifically valuable in its own right.

Finally, the modular design of the architecture makes it straightforward to extend to multi-task settings, where the encoder is shared across several morphokinetic prediction objectives simultaneously. Training a single encoder to represent cytoplasmic waves, cleavage events, and blastocyst formation jointly would likely produce richer and more generalisable latent representations than any single-task model, and would bring the framework closer to the kind of comprehensive embryo characterisation tool that clinical practice ultimately requires.

Bibliography

[1] M. H. Johnson, "Robert edwards: Nobel laureate in physiology or medicine," *Nobel Lectures In Physiology Or Medicine (2006-2010)*, p. 379, 2015.

[2] A. G. Sutcliffe and M. Ludwig, "Outcome of assisted reproduction," *The Lancet*, vol. 370, no. 9584, pp. 351–359, 2007.

[3] S. Armstrong, A. Vail, S. Mastenbroek, V. Jordan, and C. Farquhar, "Time-lapse in the ivf-lab: how should we assess potential benefit?," *Human Reproduction*, vol. 30, no. 1, pp. 3–8, 2015.

[4] J.-P. Ozil and K. Swann, "Stimulation of repetitive calcium transients in mouse eggs.," *The Journal of physiology*, vol. 483, no. 2, pp. 331–346, 1995.

[5] D. Fortun, P. Bouthemy, and C. Kervrann, "Optical flow modeling and computation: A survey," *Computer Vision and Image Understanding*, vol. 134, pp. 1–21, 2015.

[6] R. Szeliski, *Computer vision: algorithms and applications*. Springer Nature, 2022.

[7] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *IJCAI'81: 7th international joint conference on Artificial intelligence*, vol. 2, pp. 674–679, 1981.

[8] A. Dosovitskiy, P. Fischer, E. Ilg, P. Hausser, C. Hazirbas, V. Golkov, P. Van Der Smagt, D. Cremers, and T. Brox, "Flownet: Learning optical flow with convolutional networks," in *Proceedings of the IEEE international conference on computer vision*, pp. 2758–2766, 2015.

[9] E. Ilg, N. Mayer, T. Saikia, M. Keuper, A. Dosovitskiy, and T. Brox, "Flownet 2.0: Evolution of optical flow estimation with deep networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2462–2470, 2017.

[10] A. Jaiswal, A. R. Babu, M. Z. Zadeh, D. Banerjee, and F. Makedon, "A survey on contrastive self-supervised learning," *Technologies*, vol. 9, no. 1, p. 2, 2020.

[11] F. Schroff, D. Kalenichenko, and J. Philbin, "Facenet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 815–823, 2015.

[12] Y. Jiang, L. Zhang, and W. Wang, "Improved universal sentence embeddings with prompt-based contrastive learning and energy-based learning," in *Findings of the association for computational linguistics: EMNLP 2022*, pp. 3021–3035, 2022.

[13] K. Chaitanya, E. Erdil, N. Karani, and E. Konukoglu, "Local contrastive loss with pseudo-label based self-training for semi-supervised medical image segmentation," *Medical image analysis*, vol. 87, p. 102792, 2023.

[14] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *International conference on machine learning*, pp. 1597–1607, PmLR, 2020.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

[16] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[17] A. Bardes, J. Ponce, and Y. LeCun, "Vicreg: Variance-invariance-covariance regularization for self-supervised learning," *arXiv preprint arXiv:2105.04906*, 2021.

[18] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *science*, vol. 313, no. 5786, pp. 504–507, 2006.

[19] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 2002.

[20] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE transactions on neural networks*, vol. 5, no. 2, pp. 157–166, 1994.

[21] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[22] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," *arXiv preprint arXiv:1406.1078*, 2014.

[23] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.

[24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[25] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2020.

[26] G. Bertasius, H. Wang, and L. Torresani, "Is space-time attention all you need for video understanding?," in *Icml*, vol. 2, p. 4, 2021.

[27] B. Goodman and S. Flaxman, "European union regulations on algorithmic decision-making and a "right to explanation"," *AI magazine*, vol. 38, no. 3, pp. 50–57, 2017.

[28] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. García, S. Gil-López, D. Molina, R. Benjamins, *et al.*, "Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai," *Information fusion*, vol. 58, pp. 82–115, 2020.

[29] C. Molnar, *Interpretable machine learning*. Lulu. com, 2020.

[30] M. T. Ribeiro, S. Singh, and C. Guestrin, "" why should i trust you?" explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pp. 1135–1144, 2016.

[31] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.

[32] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE international conference on computer vision*, pp. 618–626, 2017.

[33] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *International conference on machine learning*, pp. 3319–3328, PMLR, 2017.

[34] K. Devireddy, "A comparative study of explainable ai methods: Model-agnostic vs. model-specific approaches," *arXiv preprint arXiv:2504.04276*, 2025.

[35] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[36] W. Samek and K.-R. Müller, "Towards explainable artificial intelligence," in *Explainable AI: interpreting, explaining and visualizing deep learning*, pp. 5–22, Springer, 2019.

[37] E. Tjoa and C. Guan, "A survey on explainable artificial intelligence (xai): Toward medical xai," *IEEE transactions on neural networks and learning systems*, vol. 32, no. 11, pp. 4793–4813, 2020.

[38] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part I 13*, pp. 818–833, Springer, 2014.

[39] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 15619–15629, 2023.

[40] S. Medenica, D. Zivanovic, L. Batkoska, S. Marinelli, G. Basile, A. Perino, G. Cucinella, G. Gullo, and S. Zaami, "The future is coming: artificial intelligence in the treatment of infertility could improve assisted reproduction outcomes—the value of regulatory frameworks," *Diagnostics*, vol. 12, no. 12, p. 2979, 2022.

[41] D. Smilkov, N. Thorat, B. Kim, F. Viégas, and M. Wattenberg, "Smoothgrad: removing noise by adding noise," *arXiv preprint arXiv:1706.03825*, 2017.

[42] S. Abnar and W. Zuidema, "Quantifying attention flow in transformers," in *Proceedings of the 58th annual meeting of the association for computational linguistics*, pp. 4190–4197, 2020.

Appendix A

This appendix provides selected portions of the implementation developed for this thesis. All code listings are reported in their original form to ensure clarity, transparency, and reproducibility. The listings cover the contrastive and reconstruction loss functions, the encoder and decoder architectures, and the two explainability modules.

A.1 Contrastive Loss Functions

```python
from __future__ import annotations

import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import Optional


class NTXentLoss(nn.Module):
    """
    Normalized Temperature-scaled Cross-Entropy (SimCLR).

    For a batch of N clips, 2N views total.
    The positive pair for each view is its augmented counterpart.

    z1, z2 : (N, D) - raw (un-normalized) embeddings
    """

    def __init__(self, temperature: float = 0.07):
        super().__init__()
        self.temperature = temperature

    def forward(
        self,
        z1: torch.Tensor,
        z2: torch.Tensor,
```

```
        labels: Optional[torch.Tensor] = None, # unused, kept for API compat
    ) -> torch.Tensor:
        N = z1.shape[0]
        z = F.normalize(torch.cat([z1, z2], dim=0), dim=1) # (2N, D)
        sim = torch.mm(z, z.T) / self.temperature # (2N, 2N)

        mask = torch.eye(2 * N, dtype=torch.bool, device=z.device)
        sim.masked_fill_(mask, float("-inf"))

        labels_ce = torch.cat([
            torch.arange(N, 2 * N, device=z.device),
            torch.arange(0, N, device=z.device),
        ])
        return F.cross_entropy(sim, labels_ce)


class VICRegLoss(nn.Module):
    """
    VICReg: Variance-Invariance-Covariance Regularization
    (Bardes, Ponce, LeCun, 2022).

    Three components operating on the full-batch embeddings:

    Invariance : MSE between the two views (push positives together)
    Variance : hinge on per-dim std >= V (prevent collapse)
    Covariance : off-diagonal covariance (decorrelate features)

    Total loss: I * invariance + V * variance + C * covariance

    Typical values (from paper): I=25, V=25, C=1.

    z1, z2 : (N, D) - raw embeddings (do NOT need to be normalized)
    """

    def __init__(
        self,
        lam: float = 25.0, # invariance weight
        mu: float = 25.0, # variance weight
        nu: float = 1.0, # covariance weight
        gamma: float = 1.0, # target std for variance hinge
        eps: float = 1e-4, # numerical stability for std
    ):
        super().__init__()
        self.lam = lam
        self.mu = mu
        self.nu = nu
        self.gamma = gamma
        self.eps = eps

    def forward(
        self,
```

```
        z1: torch.Tensor,
        z2: torch.Tensor,
        labels: Optional[torch.Tensor] = None, # unused
    ) -> torch.Tensor:
        N, D = z1.shape

        # Invariance: MSE between paired views
        inv_loss = F.mse_loss(z1, z2)

        # Variance: hinge on per-dimension std
        # Operates on each view independently, then averages
        def _var_loss(z: torch.Tensor) -> torch.Tensor:
            z_centered = z - z.mean(dim=0, keepdim=True)
            std = z_centered.pow(2).mean(dim=0).add(self.eps).sqrt() # (D,)
            return F.relu(self.gamma - std).mean()

        var_loss = (_var_loss(z1) + _var_loss(z2)) / 2.0

        # Covariance: off-diagonal cov matrix terms
        def _cov_loss(z: torch.Tensor) -> torch.Tensor:
            z_c = z - z.mean(dim=0, keepdim=True)
            cov = (z_c.T @ z_c) / (N - 1) # (D, D)
            # Zero out the diagonal - only penalize off-diagonal
            diag = torch.eye(D, device=z.device, dtype=z.dtype)
            off = cov.pow(2) * (1.0 - diag)
            return off.sum() / D

        cov_loss = (_cov_loss(z1) + _cov_loss(z2)) / 2.0

        return self.lam * inv_loss + self.mu * var_loss + self.nu * cov_loss
```

Listing A.1: Contrastive loss functions (NT-Xent, VICReg).

A.2 Reconstruction and Combined Loss

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import Optional

from training.contrastive import NTXentLoss # re-export for existing imports


class ReconstructionLoss(nn.Module):
    """
    Binary cross-entropy on the decoder output (wave probability mask).
    Used with decoder_type = "simple" | "unet".
    Target is a pseudo-label from optical flow magnitude.
    """

    def __init__(self, flow_threshold: float = 0.3):
```

```
        super().__init__()
        self.flow_threshold = flow_threshold
        self.bce = nn.BCELoss()

    def forward(self, pred_mask: torch.Tensor,
                input_clip: torch.Tensor) -> torch.Tensor:
        with torch.no_grad():
            if input_clip.shape[1] == 2:
                # Pure flow: L2 norm of (vx, vy)
                magnitude = input_clip.norm(dim=1, keepdim=True)
            elif input_clip.shape[1] == 5:
                # Fusion: use flow channels (4,5) for motion magnitude
                magnitude = input_clip[:, 3:].norm(dim=1, keepdim=True)
            else:
                # RGB only: temporal difference as proxy for motion
                mean_frame = input_clip.mean(dim=2, keepdim=True)
                magnitude = (input_clip - mean_frame).abs().mean(dim=1,
                                                                 keepdim=True)
            mag_flat = magnitude.view(magnitude.shape[0], -1)
            mag_min = mag_flat.min(dim=1).values.view(-1,1,1,1,1)
            mag_max = mag_flat.max(dim=1).values.view(-1,1,1,1,1)
            mag_norm = (magnitude - mag_min) / (mag_max - mag_min + 1e-6)
            # target always (B,1,T,H,W) to match pred_mask regardless of input
                channels
            target = (mag_norm > self.flow_threshold).float()
        return self.bce(pred_mask, target)


class GrayscaleLoss(nn.Module):
    _LUMA = torch.tensor([0.299, 0.587, 0.114])

    def forward(
        self,
        pred: torch.Tensor, # (B, 1, T, H, W) sigmoid [0, 1]
        target: torch.Tensor, # (B, C, T, H, W) normalised input clip
    ) -> torch.Tensor:
        B, C, T, H, W = target.shape

        with torch.no_grad():
            # Luminance extraction
            if C >= 3:
                # ITU-R BT.601 weighted sum of RGB channels
                w = self._LUMA.to(target.device)
                luma = (target[:, :3] * w[None, :, None, None, None]).sum(
                    dim=1, keepdim=True) # (B,1,T,H,W) in [0,1]
            elif C == 2:
                # Flow-only: L2 magnitude as luminance proxy
                luma = target.norm(dim=1, keepdim=True)
                luma = luma / (luma.amax((1,2,3,4), keepdim=True) + 1e-6)
            else:
                luma = target[:, :1]
```

```
67
68          # Per-sample contrast stretch -> [0.05, 0.95]
69          # Prevents trivial solution (predict clip mean) when frame is
70          # low-contrast. Clamp range to avoid division by near-zero.
71          lo = luma.view(B, -1).min(dim=1).values.view(B, 1, 1, 1, 1)
72          hi = luma.view(B, -1).max(dim=1).values.view(B, 1, 1, 1, 1)
73          rng = (hi - lo).clamp(min=0.05)
74          luma = (luma - lo) / rng # [0, 1]
75          luma = luma * 0.90 + 0.05 # [0.05, 0.95]
76
77          # Resize to decoder output resolution if needed
78          if luma.shape[2:] != pred.shape[2:]:
79              luma = F.interpolate(luma, size=pred.shape[2:],
80                                   mode='trilinear', align_corners=False)
81
82      # 0.7*MSE gives strong gradient for large errors early on
83      # 0.3*L1 prevents over-smoothing once the decoder converges
84      return 0.7 * F.mse_loss(pred, luma) + 0.3 * F.l1_loss(pred, luma)
85
86
87  class CombinedLoss(nn.Module):
88      """
89      Combined loss: contrastive (pluggable) + optional reconstruction.
90
91      Supports both ReconstructionLoss (binary mask) and RGBReconLoss (RGB).
92      The correct reconstruction loss is selected based on decoder_type.
93
94      Parameters
95      temperature : for NT-Xent / InfoNCE / SupCon
96      recon_weight : weight of the reconstruction term
97      use_decoder : whether a decoder is present
98      contrastive_loss_fn : nn.Module with forward(z1, z2, labels=None)
99      """
100
101     def __init__(self, temperature: float = 0.07, recon_weight: float = 0.2,
102                  use_decoder: bool = True,
103                  contrastive_loss_fn: Optional[nn.Module] = None,
104                  decoder_type: str = "simple",
105                  input_mode: str = "fusion",
106                  rgb_l1_weight: float = 1.0,
107                  rgb_perceptual_weight: float = 0.1,
108                  rgb_ssim_weight: float = 0.1,
109                  contrastive_weight: float = 0.8):
110         super().__init__()
111         self.contrastive = contrastive_loss_fn or NTXentLoss(temperature)
112         self.contrastive_weight = contrastive_weight # lambda_enc
113         self.recon_weight = recon_weight # lambda_dec
114         self.use_decoder = use_decoder
115         self.decoder_type = decoder_type
116
117         if use_decoder:
```

```
            if decoder_type == "grayscale":
                self.recon = GrayscaleLoss()
            else:
                self.recon = ReconstructionLoss()

    def forward(self, z1: torch.Tensor, z2: torch.Tensor,
                pred_mask: Optional[torch.Tensor] = None,
                input_clip: Optional[torch.Tensor] = None,
                labels: Optional[torch.Tensor] = None) -> torch.Tensor:
        """
        loss = lambda_enc * contrastive(z1,z2) + lambda_dec * recon(pred, clip)
        """
        enc_loss = self.contrastive(z1, z2, labels=labels)
        loss = self.contrastive_weight * enc_loss
        if self.use_decoder and pred_mask is not None and input_clip is not None:
            loss = loss + self.recon_weight * self.recon(pred_mask, input_clip)
        return loss
```

Listing A.2: Reconstruction loss and combined training objective.

A.3 Encoder

```
from __future__ import annotations

import math
import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import List, Optional, Tuple

class Conv3dBlock(nn.Module):
    """Single 3D conv + BN + ReLU block."""

    def __init__(
        self,
        in_ch: int,
        out_ch: int,
        kernel: tuple = (3, 3, 3),
        stride: tuple = (1, 2, 2),
        padding: tuple = (1, 1, 1),
    ):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv3d(
                in_ch, out_ch,
                kernel_size=kernel, stride=stride,
                padding=padding, bias=False,
            ),
            nn.BatchNorm3d(out_ch),
            nn.ReLU(inplace=True),
        )
```

```
30
31      def forward(self, x: torch.Tensor) -> torch.Tensor:
32          return self.block(x)
33
34
35  class CNN3DBackbone(nn.Module):
36      """
37      Lightweight 3D CNN.
38
39      Input : (B, C, T, H, W)
40      Output: (B, out_channels, T', H', W')
41              T' == T (temporal stride = 1 throughout)
42              H' == H/16, W' == W/16
43
44      Intermediate feature maps [block1..block4] stored for GradCAM.
45      Block channels: base_ch, 2x, 4x, 8x.
46      """
47
48      def __init__(self, in_channels: int, base_ch: int = 32):
49          super().__init__()
50          self.block1 = Conv3dBlock(in_channels, base_ch, stride=(1, 2, 2))
51          self.block2 = Conv3dBlock(base_ch, base_ch * 2, stride=(1, 2, 2))
52          self.block3 = Conv3dBlock(base_ch * 2, base_ch * 4, stride=(1, 2, 2))
53          self.block4 = Conv3dBlock(base_ch * 4, base_ch * 8, stride=(1, 2, 2))
54          self.out_channels = base_ch * 8
55          self.base_ch = base_ch
56
57      def forward(
58          self, x: torch.Tensor
59      ) -> Tuple[torch.Tensor, List[torch.Tensor]]:
60          feats: List[torch.Tensor] = []
61          x = self.block1(x); feats.append(x) # (B, ch, T', H/2, W/2)
62          x = self.block2(x); feats.append(x) # (B, 2ch, T', H/4, W/4)
63          x = self.block3(x); feats.append(x) # (B, 4ch, T', H/8, W/8)
64          x = self.block4(x); feats.append(x) # (B, 8ch, T', H/16, W/16)
65          return x, feats
66
67
68  class GRUTemporal(nn.Module):
69      """GRU temporal aggregation. Returns (embedding, all_hidden, None)."""
70
71      def __init__(self, feat_dim: int, hidden_dim: int, num_layers: int):
72          super().__init__()
73          self.rnn = nn.GRU(
74              input_size=feat_dim, hidden_size=hidden_dim,
75              num_layers=num_layers, batch_first=True,
76              dropout=0.2 if num_layers > 1 else 0.0,
77          )
78          self.hidden_dim = hidden_dim
79
80      def forward(
```

```
        self, seq: torch.Tensor
    ) -> Tuple[torch.Tensor, torch.Tensor, None]:
        all_hidden, h_n = self.rnn(seq) # (B,T',H), (layers,B,H)
        return h_n[-1], all_hidden, None


class LSTMTemporal(nn.Module):
    """LSTM temporal aggregation. Returns (embedding, all_hidden, None)."""

    def __init__(self, feat_dim: int, hidden_dim: int, num_layers: int):
        super().__init__()
        self.rnn = nn.LSTM(
            input_size=feat_dim, hidden_size=hidden_dim,
            num_layers=num_layers, batch_first=True,
            dropout=0.2 if num_layers > 1 else 0.0,
        )
        self.hidden_dim = hidden_dim

    def forward(
        self, seq: torch.Tensor
    ) -> Tuple[torch.Tensor, torch.Tensor, None]:
        all_hidden, (h_n, _) = self.rnn(seq)
        return h_n[-1], all_hidden, None


class PositionalEncoding(nn.Module):
    """Sinusoidal positional encoding. max_len=512 covers 256+ frame clips."""

    def __init__(self, d_model: int, dropout: float = 0.1, max_len: int = 512):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div = torch.exp(
            torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
        )
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer("pe", pe.unsqueeze(0)) # (1, max_len, d_model)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.dropout(x + self.pe[:, : x.size(1)])


class TransformerTemporal(nn.Module):
    """Transformer encoder for temporal aggregation."""

    def __init__(
        self,
        feat_dim: int,
        hidden_dim: int,
```

```
        num_layers: int,
        n_heads: int = 4,
        dropout: float = 0.1,
    ):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.n_heads = n_heads
        self.num_layers = num_layers

        self.input_proj = nn.Linear(feat_dim, hidden_dim)
        self.pos_enc = PositionalEncoding(hidden_dim, dropout, max_len=512)

        # CLS token -- one vector broadcast over batch at forward time
        self.cls_token = nn.Parameter(torch.zeros(1, 1, hidden_dim))
        nn.init.trunc_normal_(self.cls_token, std=0.02)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=hidden_dim,
            nhead=n_heads,
            dim_feedforward=hidden_dim * 4,
            dropout=dropout,
            batch_first=True,
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=
            num_layers)

    def forward(
        self, seq: torch.Tensor
    ) -> Tuple[torch.Tensor, torch.Tensor, Optional[torch.Tensor]]:
        """
        Manual layer-by-layer forward: the only reliable way to extract
        per-head attention weights in PyTorch (nn.TransformerEncoder.forward
        silently ignores need_weights).
        """
        B, T, _ = seq.shape

        x = self.pos_enc(self.input_proj(seq)) # (B, T', hidden_dim)
        cls = self.cls_token.expand(B, 1, -1)
        x = torch.cat([cls, x], dim=1) # (B, T'+1, hidden_dim)

        layer_attns: List[torch.Tensor] = []

        for layer in self.transformer.layers:
            # Self-attention
            x_norm = layer.norm1(x)
            attn_out, w = layer.self_attn(
                x_norm, x_norm, x_norm,
                need_weights=True,
                average_attn_weights=False, # (B, n_heads, T'+1, T'+1)
            )
            x = x + layer.dropout1(attn_out)
```

```
182
183             # Feed-forward
184             x_norm2 = layer.norm2(x)
185             ff = layer.linear2(
186                 layer.dropout(layer.activation(layer.linear1(x_norm2)))
187             )
188             x = x + layer.dropout2(ff)
189
190             # Drop CLS row/col so shape = (B, n_heads, T', T')
191             layer_attns.append(w[:, :, 1:, 1:].detach())
192
193         embedding = x[:, 0] # (B, hidden_dim) CLS
194         all_hidden = x[:, 1:] # (B, T', hidden_dim)
195
196         attn = torch.stack(layer_attns, dim=0).mean(dim=0) \
197               if layer_attns else None
198
199         return embedding, all_hidden, attn
200
201     def remove_hooks(self) -> None:
202         pass # API compatibility -- no hooks used
203
204
205 class VideoEncoder(nn.Module):
206     """Full video encoder: CNN3D backbone + modular temporal module."""
207
208     def __init__(
209         self,
210         in_channels: int,
211         base_ch: int = 32,
212         hidden_dim: int = 256,
213         num_layers: int = 2,
214         embedding_dim: int = 256,
215         model_type: str = "gru",
216         n_heads: int = 4,
217         temporal_pooling: str = "last",
218         spatial_pooling: str = "mean",
219         cnn_dropout: float = 0.1, # Dropout3d after CNN backbone output
220         proj_dropout: float = 0.2, # Dropout before projection head
221     ):
222         super().__init__()
223
224         self.model_type = model_type
225         self.hidden_dim = hidden_dim
226         self.embedding_dim = embedding_dim
227         self.temporal_pooling = temporal_pooling
228         self.spatial_pooling = spatial_pooling
229
230         # CNN backbone (UNCHANGED)
231         self.backbone = CNN3DBackbone(in_channels=in_channels, base_ch=base_ch)
232         cnn_out_ch = self.backbone.out_channels
```

```
233
234         # Dropout3d after CNN - drops entire feature channels randomly.
235         # Applied AFTER backbone, BEFORE spatial pooling.
236         # NOT placed inside attention pools (would corrupt attention weights).
237         # Set cnn_dropout=0 to disable.
238         self.cnn_dropout = (nn.Dropout3d(p=cnn_dropout)
239                             if cnn_dropout > 0.0 else None)
240
241         self.sp_pool = None
242         self._avg_pool = nn.AdaptiveAvgPool3d((None, 1, 1))
243
244         # Temporal module
245         if model_type == "gru":
246             self.temporal = GRUTemporal(cnn_out_ch, hidden_dim, num_layers)
247         elif model_type == "lstm":
248             self.temporal = LSTMTemporal(cnn_out_ch, hidden_dim, num_layers)
249         else:
250             self.temporal = TransformerTemporal(
251                 cnn_out_ch, hidden_dim, num_layers, n_heads=n_heads,
252             )
253
254         # Projection head with dropout before final linear.
255         # NOT inside attention pools - only here in the projection path.
256         # Set proj_dropout=0 to disable.
257         self.proj_head = nn.Sequential(
258             nn.Linear(hidden_dim, hidden_dim),
259             nn.ReLU(inplace=True),
260             nn.Dropout(p=proj_dropout) if proj_dropout > 0.0
261                 else nn.Identity(),
262             nn.Linear(hidden_dim, embedding_dim),
263         )
264
265     def _spatial_pool(self, cnn_out: torch.Tensor) -> torch.Tensor:
266         """(B, C, T', H', W') -> (B, T', C)"""
267         if self.sp_pool is not None:
268             return self.sp_pool(cnn_out)
269         pooled = self._avg_pool(cnn_out) # (B, C, T', 1, 1)
270         B, C, T, _, _ = pooled.shape
271         return pooled.view(B, C, T).permute(0, 2, 1) # (B, T', C)
272
273     def forward(
274         self, x: torch.Tensor,
275     ) -> Tuple[torch.Tensor, torch.Tensor, List[torch.Tensor], Optional[torch.
        Tensor]]:
276         """x : (B, C, T, H, W)"""
277         cnn_out, cnn_feats = self.backbone(x) # (B, ch, T', H', W')
278         # Dropout3d: zero entire channels randomly during training.
279         # Skipped in eval mode automatically by PyTorch.
280         if self.cnn_dropout is not None:
281             cnn_out = self.cnn_dropout(cnn_out)
282         seq = self._spatial_pool(cnn_out) # (B, T', ch)
```

```
283
284         raw_embed, all_hidden, attn_weights = self.temporal(seq)
285         # raw_embed : (B, D) -- CLS or last hidden
286         # all_hidden: (B, T', D)
287         # attn_weights: (B, n_heads, T', T') | None
288
289         if self.attn_pool is not None:
290             embedding = self.attn_pool(all_hidden) # (B, D)
291         else:
292             embedding = raw_embed
293
294         projection = self.proj_head(embedding) # (B, embedding_dim)
295         return embedding, projection, cnn_feats, attn_weights
296
297     def get_all_hidden(self, x: torch.Tensor) -> torch.Tensor:
298         """
299         (B, T', hidden_dim) -- per-frame hidden states.
300         Used by temporal occlusion XAI.
301         """
302         cnn_out, _ = self.backbone(x)
303         seq = self._spatial_pool(cnn_out)
304         _, all_hidden, _ = self.temporal(seq)
305         return all_hidden
```

Listing A.3: Spatiotemporal encoder: 3D CNN backbone, GRU/LSTM/Transformer temporal module, and projection head.

A.4 Decoder

```
1  from __future__ import annotations
2
3  import torch
4  import torch.nn as nn
5  import torch.nn.functional as F
6  from typing import List, Optional
7
8
9  def _upsample_block(in_ch: int, out_ch: int) -> nn.Sequential:
10     """Trilinear upsample (x2 spatial) + Conv3d + BN + ReLU."""
11     return nn.Sequential(
12         nn.Upsample(scale_factor=(1, 2, 2), mode="trilinear", align_corners=False),
13         nn.Conv3d(in_ch, out_ch, kernel_size=3, padding=1, bias=False),
14         nn.BatchNorm3d(out_ch),
15         nn.ReLU(inplace=True),
16     )
17
18 class WaveDecoder(nn.Module):
19     """
20     Original decoder: latent vector -> wave probability mask [0,1].
```

```
    Output: (B, 1, T, H, W) sigmoid.
    Kept unchanged for backward compatibility.
    """

    def __init__(self, latent_dim: int, target_clip_len: int = 32,
                 target_spatial: int = 128, base_ch: int = 32):
        super().__init__()
        self.target_clip_len = target_clip_len
        self.target_spatial = target_spatial

        self.fc = nn.Linear(latent_dim, base_ch * 8 * 4 * 8 * 8)
        self.init_shape = (base_ch * 8, 4, 8, 8)

        self.up1 = self._up(base_ch * 8, base_ch * 4)
        self.up2 = self._up(base_ch * 4, base_ch * 2)
        self.up3 = self._up(base_ch * 2, base_ch)
        self.up4 = self._up(base_ch, base_ch // 2)
        self.final = nn.Conv3d(base_ch // 2, 1, kernel_size=1)

    @staticmethod
    def _up(ic, oc):
        return nn.Sequential(
            nn.ConvTranspose3d(ic, oc, kernel_size=2, stride=2),
            nn.BatchNorm3d(oc), nn.ReLU(inplace=True),
        )

    def forward(self, embedding: torch.Tensor,
                target_t: Optional[int] = None,
                cnn_feats: Optional[List[torch.Tensor]] = None) -> torch.Tensor:
        B = embedding.shape[0]
        T = target_t if target_t is not None else self.target_clip_len
        x = self.fc(embedding).view(B, *self.init_shape)
        x = self.up1(x); x = self.up2(x); x = self.up3(x); x = self.up4(x)
        x = self.final(x)
        x = F.interpolate(x, size=(T, self.target_spatial, self.target_spatial),
                          mode="trilinear", align_corners=False)
        return torch.sigmoid(x)


class GrayscaleDecoder(nn.Module):
    """Grayscale decoder: latent vector -> single-channel luminance video."""

    def __init__(
        self,
        latent_dim: int,
        target_clip_len: int = 32,
        target_spatial: int = 128,
        base_ch: int = 32,
    ):
        super().__init__()
        self.target_clip_len = target_clip_len
```

```
        self.target_spatial = target_spatial
        self.out_channels = 1

        self.fc = nn.Linear(latent_dim, base_ch * 8 * 4 * 8 * 8)
        self.init_shape = (base_ch * 8, 4, 8, 8)
        nn.init.kaiming_uniform_(self.fc.weight, nonlinearity="relu")
        nn.init.zeros_(self.fc.bias)

        self.up1 = self._up_block(base_ch * 8, base_ch * 4)
        self.up2 = self._up_block(base_ch * 4, base_ch * 2)
        self.up3 = self._up_block(base_ch * 2, base_ch)
        self.up4 = self._up_block(base_ch, base_ch // 2)
        self.final = nn.Conv3d(base_ch // 2, 1, kernel_size=1, bias=True)
        # Small normal init: non-zero weights -> healthy gradient flow.
        # Bias=0 -> sigmoid(0)=0.5 which is mid-range - good starting point.
        nn.init.normal_(self.final.weight, mean=0.0, std=0.01)
        nn.init.constant_(self.final.bias, 0.0)

    @staticmethod
    def _up_block(in_ch: int, out_ch: int) -> nn.Sequential:
        return nn.Sequential(
            nn.ConvTranspose3d(in_ch, out_ch, kernel_size=2, stride=2),
            nn.BatchNorm3d(out_ch),
            nn.ReLU(inplace=True),
        )

    def forward(
        self,
        embedding: torch.Tensor,
        target_t: Optional[int] = None,
        cnn_feats: Optional[List[torch.Tensor]] = None, # ignored
    ) -> torch.Tensor:
        """
        embedding : (B, latent_dim)
        Returns : (B, 1, T, H, W) in [0, 1]
        """
        B = embedding.shape[0]
        T = target_t if target_t is not None else self.target_clip_len
        x = self.fc(embedding).view(B, *self.init_shape)
        x = self.up1(x)
        x = self.up2(x)
        x = self.up3(x)
        x = self.up4(x)
        x = self.final(x)
        x = F.interpolate(
            x,
            size=(T, self.target_spatial, self.target_spatial),
            mode="trilinear",
            align_corners=False,
        )
```

```
122         return torch.sigmoid(x) # (B, 1, T, H, W) in [0, 1]
```

Listing A.4: Decoder: WaveDecoder and GrayscaleDecoder variants.

A.5 Explainability

A.5.1 Spatial Attribution

```
1  from __future__ import annotations
2
3  import logging
4
5  import numpy as np
6  import torch
7  import torch.nn.functional as F
8
9  log = logging.getLogger(__name__)
10
11 def _get_embedding_safe(encoder, x: torch.Tensor) -> torch.Tensor:
12     """
13     Encoder-agnostic forward pass that returns only the embedding tensor.
14
15     Handles all return shapes our encoder can produce:
16       - tuple / list -> first element is the embedding (VideoEncoder default)
17       - dict -> value at key "embedding"
18       - bare Tensor -> returned as-is
19     """
20     out = encoder(x)
21     if isinstance(out, (tuple, list)):
22         return out[0]
23     elif isinstance(out, dict):
24         return out["embedding"]
25     return out
26
27
28 def _prototype_score(z: torch.Tensor,
29                      z_wave: torch.Tensor,
30                      z_nowave: torch.Tensor) -> torch.Tensor:
31
32     z = F.normalize(z.float(), dim=-1)
33
34     if z.dim() == 1:
35         z = z.unsqueeze(0)
36
37     z_w = F.normalize(z_wave.float(), dim=-1)
38     if z_w.dim() == 1:
39         z_w = z_w.unsqueeze(0)
40
41     z_nw = F.normalize(z_nowave.float(), dim=-1)
42     if z_nw.dim() == 1:
43         z_nw = z_nw.unsqueeze(0)
```

```
44
45      return ((z * z_w).sum(dim=-1) - (z * z_nw).sum(dim=-1)).sum()
46
47
48  def _build_prototype_score(z_wave_proto: torch.Tensor,
49                             z_nowave_proto: torch.Tensor):
50      """
51      Return a differentiable score closure for use inside the IG loops.
52
53          score(encoder, tensor_leaf) = _prototype_score(z, z_wave_proto,
                z_nowave_proto)
54
55      Uses _get_embedding_safe so it works with any encoder return shape.
56      Both prototype tensors are normalised inside _prototype_score - no
57      pre-normalisation required from the caller.
58      """
59      def _score(enc, tensor_leaf: torch.Tensor) -> torch.Tensor:
60          z = _get_embedding_safe(enc, tensor_leaf) # (1, D) or (D,)
61          return _prototype_score(z, z_wave_proto, z_nowave_proto)
62      return _score
63
64  def ig_prototype_ultimate(
65      encoder,
66      tensor: torch.Tensor, # L'input (Batch, C, T, H, W)
67      z_wave_proto: torch.Tensor,
68      z_nowave_proto: torch.Tensor,
69      n_steps: int = 20,
70      n_smooth: int = 5,
71      noise_sigma: float = 0.05,
72      temporal_sigma: float = 1.0,
73      spatial_sigma: float = 1.0,
74  ) -> np.ndarray:
75      """
76      IG Prototype Ultimate + Custom Baseline.
77
78      Perfect for Optical Flow on fixed backgrounds.
79      Combines:
80      1. Custom Baseline: Isolates motion anomalies by suppressing the background.
81      2. Numerical Precision: Float64 and secure handling of Leaf Tensors.
82      3. L2 Channel Norm: Avoids sign suppression for data like (dx, dy).
83      4. Spatial and Temporal Smoothing: Smooth and blobby maps, without salt-and-
            pepper noise.
84      """
85      try:
86          from scipy.ndimage import gaussian_filter1d, gaussian_filter
87      except ImportError:
88          raise ImportError("ig_prototype_ultimate requires scipy: pip install scipy"
                )
89
90      # 1. Calculate the median along the Time axis (dim=2)
91      # Use keepdim=True so the shape remains (1, C, 1, H, W)
```

```
    baseline_tensor = torch.median(tensor, dim=2, keepdim=True).values
    baseline_tensor = baseline_tensor.detach()

    if baseline_tensor.shape != tensor.shape:
        # 1. If the baseline is 4-dimensional (e.g. (1, C, H, W))
        if len(baseline_tensor.shape) == 4 and len(tensor.shape) == 5:
            baseline_tensor = baseline_tensor.unsqueeze(2).expand_as(tensor)
        # 2. If the baseline is 5-dimensional but has Time = 1 (e.g. (1, C, 1, H, W))
        elif len(baseline_tensor.shape) == 5 and baseline_tensor.shape[2] == 1:
            baseline_tensor = baseline_tensor.expand_as(tensor)
        else:
            raise ValueError(f"The shape of the baseline {baseline_tensor.shape} must be compatible with the input {tensor.shape}")

    encoder.eval()
    score_fn = _build_prototype_score(z_wave_proto, z_nowave_proto)
    _, C, T, H, W = tensor.shape

    # Apply custom baseline (instead of absolute zero)
    base = baseline_tensor.to(tensor.device)
    total_ig = torch.zeros_like(tensor, dtype=torch.float64)

    for _ in range(n_smooth):
        noise = torch.randn_like(tensor) * noise_sigma
        inp = tensor + noise
        step_grads = torch.zeros_like(tensor, dtype=torch.float64)

        for step in range(1, n_steps + 1):
            alpha = step / n_steps
            # Interpolation FROM baseline TO noisy input
            interp = (base + alpha * (inp - base)).detach().requires_grad_(True)

            score_fn(encoder, interp).backward()
            encoder.zero_grad()

            if interp.grad is not None:
                step_grads += interp.grad.detach().double()

        avg_grad = step_grads / n_steps
        total_ig += ((inp - base).double() * avg_grad)

    final_ig = (total_ig / n_smooth).float() # (1, C, T, H, W)
    # 1. Positive clamp and L2 norm on the channels (instead of simple sum)
    pos_ig = torch.clamp(final_ig[0], min=0.0) # (C, T, H, W)
    saliency = torch.norm(pos_ig, p=2, dim=0).cpu().numpy() # (T, H, W)
    # 2. Temporal smoothing
    if temporal_sigma > 0.0:
        saliency = gaussian_filter1d(saliency, sigma=temporal_sigma, axis=0)
    # 3. Spatial smoothing (Per-frame) to remove noise on a single pixel
    if spatial_sigma > 0.0:
```

```
        for t in range(T):
            saliency[t] = gaussian_filter(saliency[t], sigma=spatial_sigma)
    # Global Normalization
    lo, hi = float(saliency.min()), float(saliency.max())
    saliency = (saliency - lo) / (hi - lo + 1e-6)
    return saliency.astype(np.float32)
```

Listing A.5: Modified Integrated Gradients with temporal median baseline and SmoothGrad.

A.5.2 Temporal Occlusion

```
from __future__ import annotations

import logging
from typing import Optional

import numpy as np
import torch
import torch.nn.functional as F

from xai.utils import norm01, embedding_norm

log = logging.getLogger(__name__)


def _occlude_frames(tensor: torch.Tensor,
                    indices: list[int],
                    mode: str,
                    mean_frame: torch.Tensor) -> torch.Tensor:
    """
    Return a clone of *tensor* with the specified frame indices occluded.

    Parameters
    tensor : (1, C, T, H, W) input clip
    indices : list of temporal indices to occlude (already clipped to [0, T))
    mode : "zero" | "mean" | "blur"
    mean_frame : (1, C, 1, H, W) pre-computed clip mean, used when mode="mean"
    """
    x = tensor.clone()

    if mode == "zero":
        for t in indices:
            x[:, :, t] = 0.0

    elif mode == "mean":
        for t in indices:
            x[:, :, t] = mean_frame[:, :, 0]

    elif mode == "blur":
        # Strong Gaussian blur to destroy spatial detail while preserving
```

```
        # global brightness. We apply separable 1-D convolutions in H and W.
        k, sigma = 31, 10.0
        # Build 1-D Gaussian kernel on the same device as tensor
        coords = torch.arange(k, dtype=torch.float32, device=tensor.device) - k //
            2
        g1d = torch.exp(-0.5 * (coords / sigma) ** 2)
        g1d = g1d / g1d.sum()
        pad = k // 2
        C = tensor.shape[1]

        for t in indices:
            frame = x[:, :, t] # (1, C, H, W)

            # Separable convolution: blur H then W, per channel group
            kH = g1d.view(1, 1, k, 1).expand(C, 1, k, 1)
            kW = g1d.view(1, 1, 1, k).expand(C, 1, 1, k)

            f = F.conv2d(frame, kH, padding=(pad, 0), groups=C)
            f = F.conv2d(f, kW, padding=(0, pad), groups=C)

            x[:, :, t] = f

    else:
        raise ValueError(
            f"Unknown occlusion mode '{mode}'. "
            "Supported: 'zero', 'mean', 'blur'."
        )

    return x


@torch.no_grad()
def temporal_occlusion(
    encoder,
    tensor: torch.Tensor,
    mode: str = "zero",
    window_size: int = 1,
) -> np.ndarray:
    """
    Temporal occlusion importance.

    For each centre frame t in [0, T) a window of frames is occluded, the
    encoder is run on the perturbed clip, and the drop in embedding norm is
    recorded as the importance of that temporal position.

        importance[t] = max(0, ||emb(original)|| - ||emb(occluded_window_t)||)

    Frames whose removal causes a large drop were truly relied on by the model.

    Frame occlusion vs. window occlusion
    window_size=1
```

```
        Only frame t is occluded. Fine-grained - can pinpoint the exact
        frame the model relies on, but sensitive to model smoothing.

    window_size > 1
        A symmetric temporal window [t - w//2, ..., t + w//2] is occluded
        simultaneously. Better suited for models with temporal receptive
        fields > 1 (e.g. 3-D CNNs or Transformers) because removing a
        single frame may be compensated by neighbouring frames.
        Boundary indices are clipped to [0, T-1].

    Occlusion modes
    "zero"
        Replace the occluded frame(s) with all-zero pixels.
        Maximally disruptive; suitable as a first experiment.
    "mean"
        Replace with the temporal mean frame of the clip.
        Less disruptive than zero: global brightness is preserved, only
        the temporal variation at t is removed. Preferred for biological
        videos where a black frame is an obvious out-of-distribution input.

    "blur"
        Apply a strong Gaussian blur (kernel~31, sigma~10) instead of removing
        the frame. Destroys fine spatial structure while preserving global
        intensity gradients. Useful to distinguish "is the model using
        fine spatial texture at this moment?" from "does the frame exist?".
    """
    encoder.eval()
    _, C, T, H, W = tensor.shape

    base = embedding_norm(encoder, tensor).item()
    imp = np.zeros(T, dtype=np.float32)

    # Pre-compute mean frame once (used only for mode="mean")
    mean_frame = tensor.mean(dim=2, keepdim=True) # (1, C, 1, H, W)

    half = window_size // 2

    for t in range(T):
        # Build window indices, clipped to valid range
        indices = list(range(max(0, t - half), min(T, t + half + 1)))
        x_occ = _occlude_frames(tensor, indices, mode, mean_frame)
        imp[t] = max(0.0, base - embedding_norm(encoder, x_occ).item())

    if imp.max() > 0:
        imp /= imp.max()
    return imp


def attention_map(encoder, tensor: torch.Tensor,
                  mode: str = "rollout") -> Optional[np.ndarray]:
    """
```

```
    Per-frame importance from Transformer self-attention weights.
    Returns None for GRU / LSTM encoders (encoder.model_type != "transformer").
    """
    if encoder.model_type != "transformer":
        return None

    encoder.eval()
    with torch.no_grad():
        _, _, _, attn = encoder(tensor) # (1, n_heads, T', T') or None

    if attn is None:
        log.warning("attention_map: encoder returned attn=None")
        return None

    T_clip = tensor.shape[2]
    attn_np = attn[0].cpu().numpy() # (n_heads, T', T')
    T_prime = attn_np.shape[-1]

    # Uniformity diagnostic
    attn_std = float(attn_np.std(axis=-1).mean())
    threshold = 0.5 / max(T_prime, 1)
    if attn_std < threshold:
        log.warning(
            "attention_map: near-uniform attention (std=%.5f < thr=%.5f). "
            "Model may not be sufficiently trained. Map will be unreliable.",
            attn_std, threshold,
        )

    A = attn_np.mean(axis=0) # (T', T') head-averaged

    if mode == "rollout":
        I = np.eye(T_prime, dtype=np.float32)
        A_aug = A + I
        row_s = A_aug.sum(axis=-1, keepdims=True)
        A_aug = A_aug / np.where(row_s == 0, 1.0, row_s)
        importance = A_aug.sum(axis=0) # (T',) col sum
        # Subtract uniform baseline; keep above-average only
        importance = importance - importance.mean()
        importance = np.maximum(importance, 0.0)
    else: # "recv"
        importance = A.sum(axis=0) # (T',) raw col sum

    # Fallback if everything collapsed to zero
    if importance.max() < 1e-8:
        log.warning("attention_map: zero importance after aggregation - "
                    "falling back to raw column sums")
        importance = A.sum(axis=0)

    importance = norm01(importance)

    log.debug("attention_map T'=%d min=%.4f max=%.4f std=%.4f",
```

```
            T_prime, importance.min(), importance.max(), importance.std())

    # Upsample T' -> T_clip
    recv_t = torch.from_numpy(importance).float().view(1, 1, -1)
    recv_up = F.interpolate(recv_t, size=T_clip,
                            mode="linear", align_corners=False) \
              .squeeze().numpy() # (T_clip,)

    return recv_up.astype(np.float32)
```

Listing A.6: Temporal occlusion importance and attention map extraction.

“Non è il potere a fare la differenza, ma chi ti spinge a superare i tuoi limiti. I risultati non arrivano da soli, ma da tutto ciò che non si vede."

Oggi non è solo un traguardo, ma il risultato di un percorso fatto di persone, momenti e crescita. E proprio da qui voglio partire.

Il primo ringraziamento va ai miei genitori, che mi hanno dato l'opportunità di arrivare fin qui. Mi hanno sempre sostenuto in ogni aspetto, senza mai farmi mancare nulla. Questo traguardo non è soltanto mio, è nostro. Ogni piccolo gesto quotidiano, ogni forma d'amore e presenza costante negli anni hanno fatto la differenza più di quanto riesca a dire. Alla mia famiglia Anna, Ludovica e Rory, la nuova entrata che ormai non mi fa più andare nemmeno in bagno. Grazie alle mie sorelle per esserci sempre state. Un ringraziamento anche ai membri “non ufficiali ma ufficiosi” Dionisio, e un augurio al nuovo percorso imminente, e Sasy, per l'aiuto nella gestione della piccola “pazza” Rory.

Alle nonne Anna e Betta: so che aspettavate questo giorno forse più di me, e che sareste state felici forse più di me. Se oggi sono arrivato fin qui, è anche grazie alla vostra spinta costante e al desiderio di vedere il vostro sorriso, che immagino proprio in questo momento.

A nonno Pasquale "Celeste" va un grande grazie per tutti gli insegnamenti, tra serietà e momenti di leggerezza, come quella notte in Francia nella dependance di zio Francesco.

Un ringraziamento va a tutta la famiglia Lanzaro Cuomo: zia Rosaria, zio Enzo, Simona, Giacomo, zio Francesco, zia Adeline, Angela e leyla, e un pensiero affettuoso all'ultimo Natale passato insieme.

Un grazie sentito ai professori Ciaramella e Montella, che in questi cinque anni mi hanno sempre sostenuto, aiutato a risolvere ogni problema e offerto tante opportunità. Un ringraziamento particolare al prof. Di Nardo per aver seguito la mia tesi, e a tutto il corpo docente per questi anni indimenticabili. Un grazie anche al professor Maratea, per i caffè del quarto piano e probabilmente per molte altre cose che nemmeno lui sa.

È il turno di Fiorillo e Gallo. Le storie sarebbero infinite, forse troppe per essere credute. Quello che posso dire è che difficilmente si trova una compagnia così positiva, piacevole e piena di voglia di fare. A Fiorillo grazie per questi cinque anni vissuti quasi sempre insieme, e a Gallo per la sua energia unica e la

sua capacità di credere sempre in tutto. Non potevo chiedere di meglio.

A Franzese: mi sono sempre chiesto se ogni comitiva abbia un "Franzese". Poi ho capito che no, non è possibile... solo noi ce l'abbiamo. Al di là di tutto, grazie per esserci sempre stato, per le risate e anche per i momenti più difficili che, ancora oggi, riesci a farmi rivivere. E a Roberta, grazie per essere parte di tutto questo e per stare accanto a Franzese: ormai sei parte integrante del gruppo.

Alcuni direbbero che è mio cugino, io dico mio fratello, A Lele. Un grande ascoltatore, un consigliere, una presenza costante. Una di quelle persone rare che sai sempre di poter avere accanto. Grazie per tutto, ma soprattutto per ciò che sei diventato oggi: un uomo cambiato e cresciuto.

Al mio personal trainer del cuore, al mio compagno di serate, a colui che osserva ogni esecuzione in palestra con attenzione maniacale... e a quello che viene dipinto come dipingono anche me (non sempre in positivo): grazie Matteo per tutte le risate fatte e per quelle che ancora faremo.

Un grazie alle mitiche Alessandra, Federica Sav e Federica Pic: il trio magico. Anche se forse non lo sanno, mi hanno aiutato tantissimo in questo periodo. Ale, la mia consulente: ti voglio bene e grazie per tutti questi anni passati insieme. Federica Sav, la più bella e la più pazza, amante delle cose strane. Federica Pic, la più responsabile, sempre pronta a curarti anche quando non ne hai bisogno.

Un ringraziamento a Sasy, Feffa e Rebecca: ormai è cresciuta e tra poco mi chiamerà zio. È bello sapere che alla mia tesi triennale c'era già, e che oggi è qui ancora più felice. Grazie a voi per averci sempre accompagnato in questo percorso.

A Marchese e Curcione: le nostre strade stanno prendendo forma, ed è una delle cose più belle. Ricordo quando era tutto incerto, meno pensieri e più uscite. Oggi invece vedo tre ragazzi con tre strade diverse, forti e pronti ad andare avanti più di prima.

A Stefano Fiscale: grazie per tutte le esperienze condivise, dalla tesi triennale alla borsa, a Oxford, a Marsiglia e a tutto il resto. Grazie per gli allenamenti, le esperienze e le sfide affrontate insieme. Sono certo che ce ne saranno ancora tante.

Al mio bro Ettore Esposito: quel giorno per caso in palestra a Pianura è iniziato tutto. Dal centro direzionale al corso di informatica, a 30 secondi da casa

mia... da lì è nata un'amicizia importante fatta di chiacchierate e risate. Grazie bro.

A fratm Sabih: forse non ve lo aspettereste, ma è il più simpatico di tutti, il re dei pasti piccanti e una persona dal cuore d'oro, sempre al massimo. Grazie per tutti i regali che ci hai fatto, per le storie che ci hai raccontato e per tutte le risate che ci siamo fatti.

A fratm Sabih: You might not expect it, but he's the nicest guy in town, the king of spicy food, and a guy with a heart of gold, always on top of things. Thank you for all the gifts you've given us, for the stories you've told us, and for all the laughs we've had.

Un ringraziamento speciale va al mitico e unico gruppo insuperabile del Maratea Fan Club. Nessuno sa come sia nato, e forse è proprio questo il bello. Oggi è una garanzia: ogni giorno 999+ messaggi, almeno 10 tag, tra risate, pazzie, challenge, case distrutte, coppie finite e coppie create. Senza i marateoti non saprei come fare. Grazie a tutti.

Un grazie speciale va a Core e' Cafè: questi cinque anni non sarebbero stati gli stessi senza di voi. Ogni mattina il vostro sorriso ha reso le mie giornate più leggere e più belle, grazie alla vostra simpatia e alla vostra gentilezza. Spero di tornare presto per continuare le mie pause caffè insieme a voi.

Infine, grazie a tutti. Grazie a Borrelli, Forgione, Alterio, Iacovelli, Palmers, il gruppo delle serate a Tipladi e Cascetta, e i miei amici dell'università Carmine, Camilla, Vincenzo, Gaia, Ylenia, Mario, Matteo e Danilo. Grazie per tutte le esperienze, le serate e i momenti condivisi insieme.